UNIVERSITÉ DON BOSCO DE LUBUMBASHI

Faculté des Sciences Informatiques

République Démocratique du Congo

Province du Haut-Katanga – Lubumbashi

www.udbl.ac.cd



Au-delà des heuristiques statiques :

Conception et mise en œuvre d'un agent d'apprentissage par renforcement pour l'optimisation holistique et dynamique de la logistique minière

---

Mémoire présenté en vue de l'obtention du Diplôme de Master en Data Science

Présenté par : MUPASA KALUNGA Jean-Luc

Promotion : Master 2 Data Science — Supply Chain

JUILLET 2026

UNIVERSITÉ DON BOSCO DE LUBUMBASHI

Faculté des Sciences Informatiques

République Démocratique du Congo

Province du Haut-Katanga – Lubumbashi

www.udbl.ac.cd



Au-delà des heuristiques statiques :

Conception et mise en œuvre d'un agent d'apprentissage par renforcement pour l'optimisation holistique et dynamique de la logistique minière

---

Mémoire présenté en vue de l'obtention du Diplôme de Master en Data Science

Présenté par : MUPASA KALUNGA Jean-Luc

Dirigé par : Dr. Olfa FERCHICHI

Promotion : Master 2 Data Science — Supply Chain

JUILLET 2026

Dédicace

À ma famille, pour son soutien indéfectible tout au long de ce parcours.

À mes collègues et amis, pour leurs encouragements constants.

Remerciements

À l'Éternel, Jéhovah Jiré, qui a pourvu à tous nos besoins et qui, par son Esprit Saint, nous a donné l'intelligence nécessaire pour mener ce travail à son terme sans jamais nous abandonner. Nous disons merci.

À notre directrice, **Dr. Olfa FERCHICHI**, qui n'a pas hésité à nous consacrer son temps et son énergie pour nous accompagner dans la réalisation de ce travail : puissiez-vous, Madame, trouver par ces mots l'expression de notre reconnaissance pour votre patience, votre tolérance et surtout pour vos orientations.

Au corps professoral de la **Faculté des Sciences Informatiques de l'Université Don Bosco de Lubumbashi**, qui a mis tous ses moyens en œuvre pour assurer notre formation tout au long de ce cursus de Master Data Science, nous adressons nos remerciements.

À ma mère, Célestine MUPASA, et à ma sœur aînée, Gracia SAFI, recevez par ces mots toute notre gratitude pour votre soutien constant, votre confiance et vos encouragements tout au long de ce parcours : jamais nous n'aurons assez de mots pour vous dire merci.

À mes frères et sœurs, pour leurs encouragements et leur présence à mes côtés tout au long de ce parcours, nous disons merci.

Enfin, nous adressons nos remerciements à toute notre famille, à nos amis ainsi qu'à nos collègues de promotion, et à toute personne ayant contribué, d'une manière ou d'une autre, à nos études et à l'élaboration de ce travail. Que chacun en soit béni en retour. Merci.

Résumé

Dans le secteur minier, l'optimisation des coûts opérationnels et la réduction de l'empreinte environnementale sont des enjeux majeurs. Les systèmes de gestion de flotte industriels, bien qu'efficaces dans des conditions stables, montrent leurs limites face à la variabilité des conditions réelles (pannes, congestion, dégradation des routes). Ce mémoire présente la conception et la mise en œuvre d'un agent autonome de dispatching basé sur l'apprentissage par renforcement profond, capable d'apprendre une politique d'affectation adaptative des camions aux pelles et aux points de déchargement. L'approche a été validée par une comparaison rigoureuse avec quatre heuristiques industrielles (FIFO, Fixed Assignment, Nearest Shovel, Shortest Path) et trois autres algorithmes d'apprentissage par renforcement représentant la progression méthodologique du mémoire (Q-Learning, SARSA, DQN), sur trois scénarios représentatifs d'une mine à ciel ouvert. Les résultats démontrent que PPO (4 208.75 t/h) et DQN (4 168.50 t/h) surpassent Fixed Assignment (4 074.00 t/h) en conditions nominales, et que cette supériorité se confirme en conditions perturbées (DQN : 3 991.75 t/h, PPO : 3 946.25 t/h contre 3 860.50 t/h pour Fixed Assignment). Les heuristiques myopes s'effondrent en surcharge ($\geq$ 200 min d'attente), validant l'hypothèse centrale du mémoire.

Mots-clés : Apprentissage par renforcement, logistique minière, dispatching dynamique, PPO, DQN, optimisation holistique, mine à ciel ouvert.

Abstract

In the mining sector, optimizing operational costs and reducing environmental impact are major challenges. Industrial Fleet Management Systems, while effective under stable conditions, show limitations when facing real-world variability (breakdowns, congestion, road degradation). This thesis presents the design and implementation of an autonomous truck dispatching agent based on Deep Reinforcement Learning, capable of learning an adaptive assignment policy for trucks to shovels and dump sites. The approach was validated through rigorous comparison with four industrial heuristics (FIFO, Fixed Assignment, Nearest Shovel, Shortest Path) and three other reinforcement learning algorithms representing the thesis's methodological progression (Q-Learning, SARSA, DQN), across three representative open-pit mine scenarios. Results show that PPO (4 208.75 t/h) and DQN (4 168.50 t/h) outperform Fixed Assignment (4 074.00 t/h) under nominal conditions, with this advantage confirmed under high-breakdown conditions (DQN : 3 991.75 t/h, PPO : 3 946.25 t/h vs. 3 860.50 t/h for Fixed Assignment). Myopic heuristics collapse under overload ($\geq$200 min waiting time), validating the central hypothesis of this work.

Keywords : Reinforcement learning, mining logistics, dynamic dispatching, PPO, DQN, holistic optimization, open-pit mining.

Table des matières

Remerciements **II**

Résumé **III**

Abstract **IV**

Liste des figures **X**

Liste des tableaux **XII**

Introduction générale **1**

1 Concepts fondamentaux de la logistique minière **3**
1.1 Logistique minière et transport en mine à ciel ouvert 3
1.2 Systèmes de gestion de flotte minière 4
1.3 Concepts de dispatching et de routage minier 4
1.4 Principes opérationnels et contraintes du transport minier 4
1.5 Conclusion 5

2 État de l'art des méthodes d'optimisation **6**
2.1 Les heuristiques classiques 6
2.2 Les méta-heuristiques 6
2.3 Les approches basées sur l'apprentissage par renforcement 7
2.4 Synthèse critique et positionnement du travail 7
2.5 Conclusion 9

3 Modélisation du problème de transport minier **11**
3.1 Description du système de transport minier 11
3.2 Hypothèses et contraintes de modélisation 12
3.3 Modélisation du réseau routier minier 16
3.4 Formulation du problème en tant que processus de décision markovien 17
3.5 Conclusion 21

4 Méthodologie, conception et choix algorithmique de la solution proposée **23**
4.1 Méthodologie 23
4.1.1 Justification du choix de CRISP-RL 23

4.1.2 Adaptation des étapes CRISP-RL au contexte minier 24
4.1.3 Positionnement de l'approche proposée dans le cycle CRISP-RL . . . 25
4.2 Présentation générale de l'approche proposée 25
4.2.1 Principe général de la solution 25
4.2.2 Architecture fonctionnelle de l'approche 26
4.2.3 Positionnement de l'agent dans le système de gestion de flotte 27
4.3 Conception de l'environnement de simulation 27
4.3.1 Architecture générale du simulateur 27
4.3.2 Modélisation des entités minières 28
4.3.3 Définition de l'espace d'observation 29
4.3.4 Définition de l'espace d'action 29
4.3.5 Fonction de récompense multi-objectif 30
4.4 Algorithmes classiques d'apprentissage par renforcement 30
4.4.1 Q-Learning pour le dispatching minier 30
4.4.2 Temporal Difference Learning 31
4.4.3 SARSA comme variante TD orientée politique 32
4.4.4 Limites des méthodes tabulaires 33
4.5 Passage au Deep Reinforcement Learning 34
4.5.1 Justification du recours au Deep Learning 34
4.5.2 Deep Q-Network (DQN) comme extension du Q-Learning 34
4.5.3 Architecture du réseau de neurones 35
4.5.4 Choix de l'algorithme final : PPO 36
4.5.5 Justification du choix de PPO par rapport aux alternatives 37
4.6 Configuration expérimentale des modèles d'apprentissage 38
4.6.1 Hyperparamètres du Q-Learning 38
4.6.2 Hyperparamètres du TD Learning / SARSA 38
4.6.3 Hyperparamètres du modèle Deep Reinforcement Learning 39
4.6.4 Stratégie exploration / exploitation 39
4.7 Stratégie d'évaluation comparative 40
4.7.1 Heuristiques de référence 40
4.7.2 Modèles RL classiques de comparaison 40
4.7.3 Modèle Deep Reinforcement Learning proposé 41
4.7.4 Indicateurs de performance (KPIs) 41
4.8 Synthèse de la solution proposée 41
4.8.1 Pseudocode de l'algorithme proposé 42
4.9 Conclusion 44

5 Implémentation et expérimentation 45
5.1 Environnement de développement et outils 45
5.1.1 Configuration matérielle 46
5.2 Architecture globale du système proposé 46
5.3 Détails d'implémentation de l'environnement 47
5.3.1 Modélisation physique du simulateur 47

5.3.2 Graphe routier 48
5.3.3 Vecteur d'observation 48
5.3.4 Fonction de récompense implémentée 49
5.3.5 Validation du simulateur 50
5.4 Paramètres et protocoles d'entraînement 50
5.4.1 Protocole général 50
5.4.2 Paramètres des méthodes tabulaires 50
5.4.3 Paramètres des agents Deep RL 51
5.4.4 Efficacité algorithmique 51
5.5 Scénarios expérimentaux 52
5.6 Métriques d'évaluation 53
5.6.1 Métriques de diagnostic de l'apprentissage 53
5.7 Conclusion 54

6 Résultats et analyse **55**
6.1 Résultats expérimentaux 55
6.1.1 Scénario nominal 55
6.1.2 Scénario high_load 56
6.1.3 Scénario high_breakdown 56
6.2 Comparaison avec les approches de référence 57
6.2.1 Analyse comparative globale 57
6.2.2 Analyse par scénario 57
6.3 Analyse critique des résultats 60
6.3.1 Convergence des agents d'apprentissage 60
6.3.2 Diagnostics complémentaires de l'apprentissage 64
6.3.3 Comportement décisionnel des agents 71
6.3.4 Limites identifiées 73
6.4 Discussion 75
6.4.1 Validation de l'hypothèse centrale H1 75
6.4.2 Apports de la progression algorithmique 76
6.4.3 Perspectives d'intégration industrielle 76
6.5 Conclusion 77

Conclusion générale et perspectives **78**

Glossaire **81**

Bibliographie **83**

Annexes **85**

A Détails d'implémentation **86**

B Hyperparamètres des modèles **88**

C Scénarios de simulation **89**

D Résultats statistiques complémentaires **90**

E Architecture logicielle et structure du projet **92**

F Interfaces et démonstrations **93**

Table des figures

3.1 Cycle opérationnel du système camion-pelle et point d'intervention de l'agent de dispatching. 12
3.2 Représentation du réseau routier sous forme de graphe stochastique. 16
3.3 Schéma d'interaction entre l'agent RL (politique) et l'environnement (simulateur minier). 19
3.4 Structure multi-objectif de la fonction de récompense globale. 21

4.1 Cycle de vie CRISP-RL adapté à la logistique minière. 25
4.2 Architecture fonctionnelle de la solution proposée. 26
4.3 Architecture modulaire du simulateur Gymnasium. 28
4.4 Architecture du réseau de neurones (MLP) de l'agent PPO. 36
4.5 Pipeline du flux de données entre la mine et l'agent DRL. 38

5.1 Architecture globale du projet. 47
5.2 Graphe routier du simulateur minier. 50

6.1 Productivité comparative des algorithmes sur les trois scénarios. 57
6.2 Temps d'attente moyen (min) — Fixed Assignment, PPO et DQN sur les trois scénarios. 60
6.3 Courbes de convergence de Q-Learning et SARSA — scénario nominal. . . . 61
6.4 Courbes de convergence de Q-Learning et SARSA — scénario high_load. . . 61
6.5 Courbes de convergence de Q-Learning et SARSA — scénario high_breakdown. 62
6.6 Convergence de DQN et PPO — scénario nominal (productivité t/h lissée). . 62
6.7 Convergence de DQN et PPO — scénario high_load (productivité t/h lissée). 63
6.8 Convergence de DQN et PPO — scénario high_breakdown (productivité t/h lissée). 63
6.9 Exploration vs exploitation — scénario nominal. 64
6.10 Exploration vs exploitation — scénario high_load. 65
6.11 Exploration vs exploitation — scénario high_breakdown. 65
6.12 Évolution de l'erreur TD moyenne par épisode — Q-Learning et SARSA, scénario nominal. 66
6.13 Évolution de l'erreur TD moyenne par épisode — Q-Learning et SARSA, scénario high_load. 66

6.14 Évolution de l'erreur TD moyenne par épisode — Q-Learning et SARSA, scénario high_breakdown. 67
6.15 DQN — perte du réseau Q (`train/loss`) au cours de l'entraînement, scénario nominal. 67
6.16 PPO — Policy Loss et Value Loss (échelle log) au cours de l'entraînement, scénario nominal. 68
6.17 DQN — perte du réseau Q (`train/loss`) au cours de l'entraînement, scénario high_load. 69
6.18 PPO — Policy Loss et Value Loss (échelle log) au cours de l'entraînement, scénario high_load. 69
6.19 DQN — perte du réseau Q (`train/loss`) au cours de l'entraînement, scénario high_breakdown. 70
6.20 PPO — Policy Loss et Value Loss (échelle log) au cours de l'entraînement, scénario high_breakdown. 70
6.21 Distribution des actions choisies par Q-Learning, SARSA, DQN et PPO — scénario nominal. 71
6.22 Distribution des actions choisies par Q-Learning, SARSA, DQN et PPO — scénario high_load. 72
6.23 Distribution des actions choisies par Q-Learning, SARSA, DQN et PPO — scénario high_breakdown. 73

Liste des tableaux

2.1 Synthèse des grandes familles d'approches et opportunités pour l'apprentissage par renforcement . . . 8
2.2 Détail des contributions et limites des articles majeurs de l'état de l'art . . . 9

3.1 Synthèse des hypothèses . . . 15

4.1 Architecture fonctionnelle de l'approche proposée . . . 26
4.2 Correspondance entre les classes logicielles et les variables du MDP . . . 28
4.3 Correspondance Q-Learning ↔ dispatching minier . . . 31
4.4 Comparaison DQN et PPO pour le dispatching minier (adapté de [1]) . . . 37
4.5 Hyperparamètres du Q-Learning . . . 38
4.6 Hyperparamètres du TD Learning / SARSA . . . 39
4.7 Hyperparamètres de l'agent PPO . . . 39
4.8 Indicateurs de performance pour l'évaluation comparative . . . 41
4.9 Synthèse de la solution proposée . . . 42

5.1 Environnement de développement et bibliothèques principales . . . 46
5.2 Configuration matérielle utilisée pour le développement et l'entraînement . . . 46
5.3 Composition du vecteur d'observation (configuration nominale) . . . 48
5.4 Diagnostic du signal de récompense avant et après ajout de w_4 . . . 49
5.5 Paramètres d'entraînement des méthodes tabulaires . . . 51
5.6 Paramètres d'entraînement des agents Deep RL . . . 51
5.7 Comparaison de l'efficacité algorithmique — scénario nominal . . . 52
5.8 Paramètres des scénarios expérimentaux . . . 52

6.1 Résultats comparatifs — Scénario nominal . . . 55
6.2 Résultats comparatifs — Scénario high_load (18 camions) . . . 56
6.3 Résultats comparatifs — Scénario high_breakdown ($p_b = 10\%$) . . . 56
6.4 Synthèse comparative multi-scénarios (productivité t/h / attente min) . . . 57
6.5 Résumé des tests statistiques appliqués aux trois scénarios. . . . 58
6.6 Efficacité d'échantillonnage : % du budget d'entraînement requis pour atteindre 95% de la performance finale . . . 71

A.1 Stack technologique du projet . . . 86

B.1 Hyperparamètres complets de tous les agents . . . 88

C.1 Paramètres complets des scénarios expérimentaux 89

D.1 Résultats post-hoc Tukey HSD — scénario nominal (paires significatives) . . 90
D.2 Résultats post-hoc Tukey HSD — scénario high_load (paires pertinentes) . . 91
D.3 Résultats post-hoc Tukey HSD — scénario high_breakdown (paires pertinentes) 91

Introduction générale

La logistique du transport constitue l'un des piliers financiers et environnementaux les plus critiques de l'industrie minière moderne. Bien que son optimisation puisse générer des économies se chiffrant en millions de dollars, elle représente souvent une part prépondérante, pouvant atteindre 60%, des coûts opérationnels totaux d'une mine à ciel ouvert [2]. C'est dans ce cadre que ce mémoire analyse l'apport de l'apprentissage par renforcement pour l'optimisation dynamique de la logistique minière. Malgré l'adoption généralisée des systèmes de gestion de flotte, les exploitations minières se heurtent à la complexité croissante et à la variabilité des environnements réels [2, 3]. La logistique minière englobe l'ensemble des opérations visant à assurer le transport efficace des matériaux, du site d'extraction jusqu'aux zones de traitement ou de stockage. Dans ce contexte, la gestion des flux de camions, la coordination avec les équipements de chargement et le respect des contraintes de production sont déterminants pour la rentabilité et la durabilité [2]. L'optimisation des flottes consiste à affecter dynamiquement les camions aux tâches de chargement, transport et déchargement tout en minimisant les temps d'attente, la consommation de carburant et l'usure des équipements [4]. Cette problématique est difficile en raison de la variabilité des conditions réelles (trafic, météo, état des routes), de la multiplicité des contraintes opérationnelles et de la nécessité d'adapter les décisions en temps réel pour maintenir la productivité globale [5]. Les approches industrielles dominantes reposent encore largement sur des heuristiques statiques et des règles prédéfinies [4] qui, bien qu'efficaces dans des contextes stables, montrent des limites marquées face à l'incertitude du terrain [6]. Elles peinent à anticiper les congestions, à réagir aux perturbations et à optimiser la flotte de manière holistique, ce qui entraîne des pertes d'efficacité et une sous-utilisation des ressources [7, 8]. Face à ces limites, l'objectif de ce mémoire est de concevoir un agent autonome capable d'adapter ses décisions en temps réel afin de réduire les coûts, améliorer l'efficacité opérationnelle et limiter l'impact environnemental.

L'étude s'articule autour des questions suivantes :

Q1 : Comment modéliser l'environnement minier complexe (réseau routier, zones tampons, événements stochastiques) dans un cadre d'apprentissage par renforcement compatible avec les standards industriels ?

Q2 : Quelle architecture d'agent d'apprentissage par renforcement est la plus adaptée pour apprendre une politique de dispatching adaptative (affectation camion $\rightarrow$ pelle $\rightarrow$ point de déchargement) dans un environnement minier stochastique ?

Q3 : Comment concevoir une fonction de récompense qui capture fidèlement les coûts réels (carburant, usure, temps) tout en pénalisant les comportements non souhaitables ?

Q4 : Quelles métriques et quels protocoles d'évaluation permettront de démontrer quan-

titativement la supériorité de l'approche d'apprentissage par renforcement par rapport aux approches classiques, notamment dans des scénarios de perturbation ?

Q5 : Comment intégrer un tel système d'apprentissage par renforcement comme couche d'intelligence additionnelle dans l'écosystème des systèmes de gestion de flotte existants, sans nécessiter un remplacement complet de l'infrastructure ?

Les réponses détaillées à ces problématiques seront développées tout au long des chapitres suivants, à travers la modélisation, l'expérimentation et l'analyse des résultats obtenus.

L'hypothèse centrale (notée H1 dans la suite) est qu'un agent autonome basé sur l'apprentissage par renforcement peut surpasser les approches heuristiques classiques pour l'optimisation dynamique de la logistique minière, en réduisant la consommation de carburant, les temps d'attente et en améliorant l'adaptabilité aux perturbations [9–11]. Les objectifs principaux de ce travail sont les suivants :

— Concevoir un environnement de simulation réaliste pour la logistique minière à ciel ouvert ;
— Développer un agent d'apprentissage par renforcement capable d'optimiser les décisions de dispatching et de routage ;
— Comparer les performances de l'agent d'apprentissage par renforcement avec celles des heuristiques classiques sur des scénarios variés ;
— Analyser l'impact de l'approche d'apprentissage par renforcement sur les indicateurs clés (coût, temps, consommation, robustesse).

La méthodologie suivie s'appuie sur le cadre CRISP-RL au contexte de l'apprentissage par renforcement : compréhension du problème et des données [3], modélisation de l'environnement, conception et entraînement de l'agent, expérimentation et analyse comparative [9, 11]. Chaque étape est guidée par une validation rigoureuse des hypothèses et une évaluation quantitative des résultats. Ce mémoire est organisé en six chapitres et une conclusion générale :

— Le premier chapitre présente le contexte général de la logistique minière et ses concepts fondamentaux, afin de poser le cadre métier et les notions clés mobilisées dans ce mémoire.
— Le deuxième chapitre propose un état de l'art des approches d'optimisation existantes, des heuristiques classiques aux méthodes avancées, et met en évidence le positionnement des approches basées sur l'apprentissage par renforcement.
— Le troisième chapitre détaille la modélisation du problème de transport minier, les hypothèses retenues et la formulation du système dans un cadre de décision séquentielle.
— Le quatrième chapitre décrit la méthodologie adoptée et la conception de la solution proposée, en précisant les choix de modélisation, d'architecture et d'apprentissage.
— Le cinquième chapitre présente l'implémentation, le protocole expérimental et les résultats obtenus sur les scénarios étudiés.
— Le sixième chapitre analyse et discute les résultats, en évaluant les performances, les limites et la robustesse de l'approche.
— Ce mémoire se conclut par un dernier chapitre de conclusion générale, qui synthétise les principaux apports et ouvre sur des perspectives de recherche et d'application.

Chapitre 1

Concepts fondamentaux de la logistique minière

Introduction

La compréhension rigoureuse de la logistique minière nécessite un cadre conceptuel clair, capable d'articuler les dimensions opérationnelles, décisionnelles et méthodologiques du problème étudié. Dans le contexte du transport en mine à ciel ouvert, la précision des notions utilisées conditionne la qualité de la modélisation, l'interprétation des résultats et la pertinence des choix d'optimisation.

1.1 Logistique minière et transport en mine à ciel ouvert

La logistique minière regroupe l'ensemble des activités de planification, d'organisation et de pilotage des flux de matériaux, d'équipements et d'informations nécessaires à l'exploitation d'une mine. Dans une mine à ciel ouvert, elle est centrée sur le système camion-pelle, où les matériaux sont extraits, chargés, transportés puis déchargés vers des destinations opérationnelles [2, 3].

Les concepts fondamentaux de ce système sont les suivants :

— **Mine à ciel ouvert :** Exploitation dans laquelle le minerai est extrait par gradins successifs depuis la surface.

— **Système camion-pelle :** Organisation de production où les pelles/chargeuses alimentent des camions qui assurent le transport du matériau.

— **Cycle camion-pelle :** Séquence *affectation* →*chargement* →*transport* →*déchargement* →*retour*.

— **Point de chargement :** Zone de prélèvement du matériau (pelle, chargeuse, front de taille).

— **Point de déchargement :** Destination de livraison (concasseur, halde, stock intermédiaire).

— **Zone tampon :** Espace de régulation des flux où les camions attendent avant affectation ou service.

1.2 Systèmes de gestion de flotte minière

Les systèmes de gestion de flotte sont des plateformes de supervision et d'aide à la décision qui orchestrent les opérations de transport en temps réel. Leur rôle est d'assurer la coordination entre ressources mobiles (camions) et ressources fixes (pelles, points de déchargement), tout en respectant les contraintes de production et de sécurité [2, 4].

Dans ce cadre, les notions de base sont :

— **Disponibilité des ressources :** Capacité effective d'un équipement à réaliser une tâche à un instant donné.

— **Affectation :** Décision d'envoyer un camion vers une destination donnée selon les priorités opérationnelles.

— **File d'attente :** Ensemble des camions en attente de service à une ressource contrainte.

— **Synchronisation camion-pelle :** Ajustement des arrivées de camions au rythme de chargement pour réduire l'attente improductive.

1.3 Concepts de dispatching et de routage minier

Le dispatching et le routage constituent les deux leviers décisionnels centraux du transport minier. Le dispatching détermine *la destination* d'un camion, alors que le routage détermine *l'itinéraire* suivi pour y parvenir. Leur couplage conditionne la performance globale du système [4, 5, 12].

Leur formalisation conceptuelle s'appuie sur les éléments suivants :

— **Dispatching :** Règle d'affectation dynamique des camions aux points de chargement ou déchargement.

— **Routage :** Choix du chemin entre deux nœuds en fonction de critères de coût, de temps et de risque.

— **Réseau routier minier :** Structure de transport du site, modélisable sous forme de graphe orienté pondéré.

— **Nœud :** Entité du réseau (pelle, concasseur, halte, intersection, zone tampon).

— **Arc :** Segment orienté reliant deux nœuds et porteur d'attributs (distance, pente, état de route).

— **Congestion :** Saturation locale des capacités de service provoquant une hausse des temps d'attente.

1.4 Principes opérationnels et contraintes du transport minier

Le transport minier est gouverné par des contraintes opérationnelles, énergétiques et de sécurité qui doivent être représentées explicitement dans le modèle conceptuel du système. La variabilité des temps de service et les perturbations imposent en outre une vision stochastique de l'exploitation [2, 6, 8].

Les concepts fondamentaux associés sont :

— **Temps de cycle :** Durée totale d'un cycle camion-pelle.

— **Temps d'attente :** Temps improductif passé en file ou en attente de ressource.

— **Taux d'utilisation :** Part du temps pendant lequel un équipement est effectivement productif.
— **Match Factor :** Indicateur d'équilibre entre capacité de transport et capacité de chargement.
— **Consommation spécifique :** Litres de carburant consommés par tonne transportée.
— **Coût opérationnel par tonne :** Coût total de transport rapporté au tonnage utile.

Enfin, la base conceptuelle du sujet inclut aussi le cadre décisionnel séquentiel de l'apprentissage par renforcement :

— **Processus de décision markovien (MDP) :** Formalisation d'un problème séquentiel par un quadruplet (S, A, P, R).
— **État :** Représentation synthétique de la situation opérationnelle observée.
— **Action :** Décision admissible appliquée à l'état courant.
— **Récompense :** Signal scalaire traduisant la qualité immédiate d'une décision au regard des objectifs métier.
— **Politique :** Règle de décision associant des actions aux états.

1.5 Conclusion

Ce chapitre a posé un socle conceptuel complet pour le sujet traité, en clarifiant les notions métier de la logistique minière, les mécanismes opérationnels du système camion-pelle, les principes de dispatching et de routage, les contraintes et indicateurs clés de performance, ainsi que le cadre décisionnel séquentiel mobilisé par l'apprentissage par renforcement. Cette base terminologique et conceptuelle sert de référence pour la revue de littérature du chapitre suivant.

Chapitre 2

État de l'art des méthodes d'optimisation

Introduction

L'optimisation de la logistique minière a connu une évolution progressive, portée par des approches de plus en plus élaborées pour répondre à la complexité des environnements d'exploitation. L'examen critique des travaux existants est indispensable pour identifier les acquis de la littérature, mais aussi les limites qui freinent encore une optimisation robuste et dynamique en conditions réelles.

Dans ce mémoire, cette revue ne sert pas seulement à inventorier les méthodes existantes, mais à faire apparaître le besoin d'un cadre de décision centré sur le dispatching, explicite sur ses hypothèses et suffisamment souple pour rester exploitable dans un environnement stochastique.

2.1 Les heuristiques classiques

L'histoire de l'optimisation de la logistique minière débute avec des heuristiques simples, telles que l'assignation du camion le plus proche ou la règle du premier arrivé, premier servi [4, 13]. Si ces méthodes sont faciles à mettre en œuvre, elles restent « myopes » : elles optimisent localement sans anticiper les conséquences globales sur la flotte. Cette limitation a conduit à l'émergence de systèmes industriels comme DISPATCH, Caterpillar MineStar ou Komatsu Jigsaw, qui intègrent des modules de planification et de suivi en temps réel [2, 14]. Ces systèmes ont permis d'augmenter la productivité et de réduire les temps d'attente, mais restent largement déterministes et rigides, peinant à s'adapter aux perturbations et à la dynamique réelle du terrain.

2.2 Les méta-heuristiques

L'apparition des systèmes industriels comme DISPATCH marque une première rupture, en intégrant la planification et le suivi en temps réel. Cependant, ces solutions restent déterministes et peu flexibles face à la variabilité du terrain.

Pour dépasser les limites des systèmes industriels, la recherche académique a proposé des approches plus sophistiquées : méta-heuristiques (algorithmes génétiques, colonies de fourmis, recuit simulé) [15, 16], modèles MILP ou programmation par contraintes [17–20]. Ces méthodes offrent de meilleures performances sur des cas complexes, mais leur capacité d'adaptation en temps réel reste limitée, notamment face à l'incertitude et à la variabilité opérationnelle [5, 6, 8].

2.3 Les approches basées sur l'apprentissage par renforcement

Face aux limites persistantes de ces approches, un nouveau paradigme issu de l'intelligence artificielle s'est imposé : l'apprentissage par renforcement. L'apprentissage par renforcement permet à un agent d'apprendre une politique de décision séquentielle en interagissant avec un environnement simulé ou réel, optimisant ainsi la flotte de manière holistique et adaptative [11, 21]. Les travaux récents montrent que l'apprentissage par renforcement peut surpasser les heuristiques classiques pour le routage de véhicules, la gestion dynamique des files d'attente et l'optimisation des ressources, même en présence d'incertitude [7, 10].

2.4 Synthèse critique et positionnement du travail

La littérature montre que, malgré les avancées des systèmes de gestion de flotte industriels et des heuristiques, l'optimisation globale et dynamique de la flotte minière reste un défi [5, 8]. Les approches d'apprentissage par renforcement offrent un potentiel inédit pour dépasser ces limites, en permettant une adaptation en temps réel, une optimisation holistique et une meilleure gestion de l'incertitude [10, 11]. Ce mémoire s'inscrit dans cette dynamique, en proposant une solution d'apprentissage par renforcement adaptée aux contraintes et aux spécificités de la logistique minière moderne.

L'analyse de la littérature permet de classifier les travaux existants en trois catégories majeures. Premièrement, les heuristiques classiques, basées sur des règles métier simples, offrent une solution rapide mais manquent de flexibilité. Deuxièmement, les méta-heuristiques (algorithmes génétiques, colonies de fourmis) permettent de traiter des problèmes plus complexes, au prix d'une puissance de calcul plus élevée. Enfin, les approches par apprentissage par renforcement émergent comme une solution prometteuse pour une optimisation dynamique et robuste face aux aléas. Le tableau 2.1 synthétise les opportunités offertes par cette dernière famille d'approches par rapport aux méthodes conventionnelles.

TABLE 2.1 – Synthèse des grandes familles d'approches et opportunités pour l'apprentissage par renforcement

Référence	**Approche**	**Limites principales / Opportunité pour l'apprentissage par renforcement**
[4]	Heuristiques statiques	Approche myope, peu adaptative, ne gère pas l'incertitude. Opportunité : L'apprentissage par renforcement peut apprendre une politique globale et adaptative.
[2, 14]	Systèmes industriels	Déterministes, rigides, peu réactifs aux perturbations. Opportunité : L'apprentissage par renforcement permet l'adaptation dynamique.
[15, 16]	Méta-heuristiques	Performantes mais peu robustes en temps réel, optimisation hors-ligne. Opportunité : L'apprentissage par renforcement gère l'incertitude et l'adaptation continue.
[17, 18]	MILP, optimisation avancée	Complexité computationnelle, difficilement scalable, peu flexible. Opportunité : L'apprentissage par renforcement offre une solution scalable et flexible.
[11, 21]	apprentissage par renforcement pour VRP/JSSP	Complexité de la modélisation, besoin de simulateur. Inspiration : Fournit un cadre état/action/récompense pour l'apprentissage par renforcement minier.
[7, 10]	apprentissage par renforcement appliqué à la logistique minière	L'apprentissage par renforcement montre des gains sur la robustesse et l'adaptation, mais nécessite une modélisation réaliste et des données de simulation.

Afin de positionner précisément notre contribution, il convient d'examiner les apports et les limites des articles scientifiques les plus significatifs du domaine. Ces travaux couvrent aussi bien l'optimisation des flux industriels que l'application d'algorithmes d'apprentissage sur des problèmes de routage. Cette revue critique, synthétisée dans le tableau 2.2, met en évidence le besoin d'une approche holistique intégrant la variabilité stochastique du terrain.

TABLE 2.2 – Détail des contributions et limites des articles majeurs de l'état de l'art

Référence	**Domaine / Approche**	**Principales contributions / Limites**
[3]	systèmes de gestion de flotte, optimisation minière	Modélisation des flux, contraintes industrielles, limites des heuristiques
[2]	systèmes de gestion de flotte industriels	Suivi en temps réel, gestion des files, limites en environnement dynamique
[4]	Heuristiques classiques	Règles de dispatching, efficacité en conditions stables, manque d'adaptabilité
[5]	Heuristiques, VRP	Variabilité du trafic, limites des modèles déterministes
[8]	Méta-heuristiques	Colonies de fourmis, optimisation partielle, adaptation limitée
[11]	apprentissage par renforcement pour VRP	apprentissage par renforcement pour routage, surpasse heuristiques, besoin de données, temps d'entraînement
[7, 10]	apprentissage par renforcement appliqué à la logistique	apprentissage par renforcement pour dispatching, robustesse, adaptation aux perturbations
[10]	apprentissage par renforcement en environnement stochastique	Politiques robustes, gestion de l'incertitude, limites pratiques
[6]	Limites des heuristiques	Incapacité à gérer la variabilité, sous-utilisation des ressources
[21]	apprentissage par renforcement : fondements	Concepts MDP, politiques, fonctions de valeur
[15, 16]	Méta-heuristiques	Algorithmes génétiques, recuit simulé, optimisation locale

2.5 Conclusion

Ce chapitre a retracé l'évolution des approches d'optimisation de la logistique minière, depuis les heuristiques simples jusqu'aux systèmes industriels, en passant par les méta-heuristiques et l'optimisation avancée. Il a mis en évidence les limites des méthodes traditionnelles face à la complexité, la variabilité et l'incertitude des environnements miniers. L'apprentissage par renforcement apparaît comme un paradigme innovant, capable d'appor-

ter une adaptation dynamique, une optimisation holistique et une robustesse accrue. Ce positionnement ouvre la voie à une nouvelle génération de solutions pour la logistique minière. Le chapitre suivant présentera la modélisation du problème de transport minier et les choix méthodologiques pour concevoir un agent d'apprentissage par renforcement adapté à la logistique minière.

Ce positionnement reste néanmoins conditionné par la qualité de la simulation et par la richesse des variables observées : le RL n'est pas une solution universelle, mais une approche pertinente lorsque le compromis entre réalisme, adaptabilité et coût de calcul est correctement maîtrisé.

Chapitre 3

Modélisation du problème de transport minier

Introduction

La modélisation du problème de transport minier constitue une étape centrale pour transformer une problématique industrielle complexe en un cadre formel exploitable par des méthodes d'optimisation et d'apprentissage. Dans le cas du transport minier, cette formalisation doit intégrer à la fois la structure du réseau, les contraintes opérationnelles et la variabilité des conditions de terrain, conformément aux analyses critiques récentes sur l'évolution des systèmes de gestion de flotte [2, 22].

3.1 Description du système de transport minier

Le système de transport minier considéré dans ce mémoire repose sur le modèle classique camion-pelle, largement déployé dans les mines à ciel ouvert [3, 4, 13]. Ce système comprend les éléments suivants :

- **Ressources d'extraction** : Des pelles (excavatrices) positionnées à des points de chargement fixes ou dynamiques, responsables du chargement du matériel brut extrait.
- **Flotte de transport** : Un ensemble de camions hétérogènes circulant entre les points de chargement et les points de déchargement (concasseurs, stockage, etc.).
- **Réseau routier** : Une infrastructure de transport composée de routes principales et secondaires, avec des points de chargement, de déchargement et des jonctions.
- **Opérateurs humains** : Conducteurs de camion et opérateurs de pelle qui exécutent les décisions du système de dispatching.
- **Variabilité environnementale** : Conditions météorologiques, état des routes, dégradation du matériel et autres aléas affectant les temps de cycle.

Le fonctionnement du système repose sur un cycle répétitif : (1) Assignation d'un camion à une pelle par le système de dispatching ; (2) Transport du camion vers la pelle ; (3) Attente à la queue de chargement ; (4) Chargement du matériau ; (5) Transport vers un point de déchargement ; (6) Déchargement ; (7) Retour au point de chargement. L'objectif opérationnel est de maximiser le tonnage transporté par unité de temps (rendement) tout en respectant

les contraintes de sécurité, de maintenance et de disponibilité des ressources, y compris dans des contextes intégrant des flottes partiellement ou totalement autonomes [14, 23].

La figure 3.1 illustre le cycle opérationnel complet du système camion-pelle décrit ci-dessus. On y suit la séquence *affectation* → *trajet* → *chargement* → *trajet* → *déchargement* → *retour*, et l'on repère le point précis où l'agent de dispatching intervient : la décision d'affectation. Cette représentation met en évidence que l'agent n'agit pas sur la physique du transport, mais uniquement sur le choix de la destination du camion, ce qui circonscrit son levier d'action.

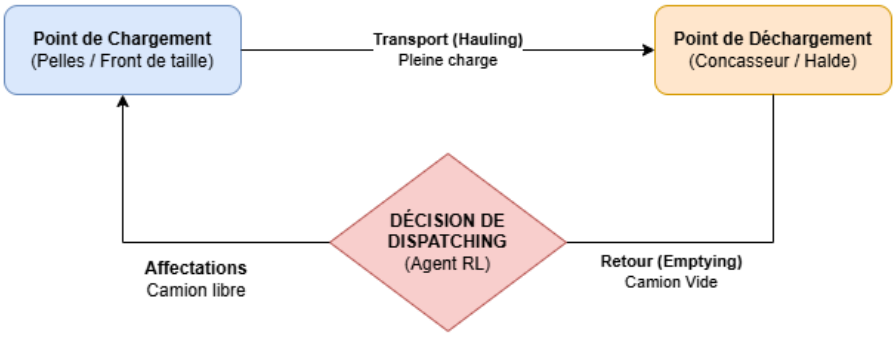


FIGURE 3.1 – Cycle opérationnel du système camion-pelle et point d'intervention de l'agent de dispatching.

Le défi de modélisation réside dans la capture simultanée de la dynamique des flux, des interactions camion-pelle, des variations stochastiques des temps de cycle et des décisions de dispatching qui influencent globalement le rendement du système.

3.2 Hypothèses et contraintes de modélisation

La modélisation du problème complexe de la logistique minière nécessite un ensemble d'hypothèses simplificatrices pour rendre le système traitable par des algorithmes d'apprentissage par renforcement. Ces hypothèses sont justifiées par les besoins d'efficacité computationnelle et de convergence algorithmique, tout en préservant les éléments essentiels du problème réel, dans une logique cohérente avec les approches actor-critic appliquées aux complexes miniers et les cadres de simulation-optimisation opérationnelle [16, 24].

Ces hypothèses doivent être lues comme des conditions de validité du modèle, et non comme une description exhaustive du terrain. Elles simplifient volontairement la réalité pour rendre l'apprentissage possible, au prix d'une perte de finesse qui devra être discutée lors de l'interprétation des résultats.

Hypothèses de modélisation

— **Horizon temporel** : Le système est modélisé en temps discret avec des pas de décision réguliers (Ex : Une décision toutes les 5 à 10 secondes en temps réel, ou à chaque

événement dans un simulateur événementiel).

- **Stationnarité à court terme** : On suppose que la distribution des temps de cycle reste stable sur des horizons court (quelques heures), permettant l'apprentissage avec des données récentes.
- **Observation partielle** : L'agent ne connaît pas parfaitement l'état complet du système (positions exactes, états des routes). Dans l'implémentation, le vecteur d'observation est construit à partir de l'état interne du simulateur (disponibilités des ressources, positions des camions, temps courant) sans injection de bruit additionnel ; la stochasticité provient de la dynamique elle-même (temps de trajet log-normaux, pannes aléatoires).
- **Camions homogènes** : Bien que les flottes réelles soient hétérogènes, on regroupe les camions par classe de capacité pour réduire la dimensionnalité de l'espace d'état, une simplification couramment mobilisée face aux contraintes de dispatching de flottes multi-capacités [17].
- **Points de chargement et déchargement fixes** : Les emplacements de pelle et les destinations de déchargement sont fixes pendant une session d'apprentissage.
- **Pas de défaillance majeure** : Le système opère sans défaillance critique des pelles ou des routes principales pendant la durée de l'épisode d'apprentissage.

Contraintes opérationnelles

Le modèle intègre un ensemble de contraintes reflétant les réalités opérationnelles :

- **Capacité des pelles** : Chaque pelle peut accueillir un nombre limité de camions en file d'attente avant que la congestion ne cause une pénalité, ce qui rejoint les modèles intégrant explicitement l'effet des files et de l'inactivité des pelles [19].
- **Disponibilité** : Les pelles ont des périodes de maintenance, les routes peuvent être fermées temporairement, réduisant les options de dispatching.
- **Distance et temps de trajet** : Les trajets ne sont jamais instantanés ; on utilise des matrices de temps calculées à partir du réseau routier.
- **Charge utile** : Les camions ont une capacité maximale et doivent être alimentés avec des matériaux compatibles (un camion ne peut pas être assigné à une pelle produisant un type de minerai que sa destination ne peut pas traiter).
- **Préemption limitée** : Les décisions de dispatching ne peuvent pas être annulées une fois qu'un camion est en route (pas de retournement à mi-parcours).

Ces hypothèses et contraintes délimitent l'espace d'exploration de l'algorithme d'apprentissage par renforcement, assurant que les politiques apprises restent réalistes et applicables en production.

Autrement dit, le modèle conserve les facteurs qui gouvernent réellement la décision, tout en écartant certains détails locaux qui augmenteraient fortement la dimension du problème sans améliorer proportionnellement la qualité de la politique apprise.

Synthèse des hypothèses de modélisation

Le tableau 3.1 récapitule chaque hypothèse de modélisation en la confrontant à sa justification et à ses inconvénients. Sa lecture en colonnes permet d'apprécier le compromis assumé entre réalisme et tractabilité : chaque simplification (temps discret, stationnarité, observation partielle, etc.) accélère l'apprentissage au prix d'une perte de finesse qu'il conviendra de garder à l'esprit lors de l'interprétation des résultats.

TABLE 3.1 – Synthèse des hypothèses

Catégorie	**Méthodes**	**Objectif**	**Inconvénients**
Temps discret	Simulation événementielle	Permet une prise de décision séquentielle compatible avec un agent RL et un simulateur événementiel.	Sensible au choix de Δt ; un pas trop grand lisse la dynamique, un pas trop fin augmente le coût de calcul.
Stationnarité à court terme	Découpage temporel	Stabilise l'apprentissage sur des fenêtres opérationnelles réalistes (quelques heures).	Peut se dégrader lors de ruptures de régime (pluie, incidents, changements de plan).
Observation partielle	Vecteur normalisé sans bruit additionnel	La stochasticité est dans la dynamique (temps log-normaux, pannes) ; l'observation reflète l'état interne du simulateur de manière exacte mais incomplète.	Introduit une part de non-markovianité apparente si l'état observé est trop pauvre.
Regroupement des camions par classes	Classification par capacité	Réduit la dimensionnalité de l'état et de l'action pour accélérer la convergence.	Perte de granularité sur les différences intra-classe (usure, performance individuelle).
Points de chargement/déchargement fixes sur un épisode	Configuration statique	Simplifie le cadre expérimental et la comparaison avec les baselines.	Généralisation limitée aux contextes de repositionnement dynamique des pelles ou destinations.
Absence de défaillance majeure pendant un épisode	Mode nominal	Permet d'évaluer d'abord la politique nominale dans un cadre contrôlé.	Sous-estime le risque opérationnel réel si les pannes ne sont pas ensuite injectées en tests de robustesse.

3.3 Modélisation du réseau routier minier

Le réseau routier minier constitue l'infrastructure d'interaction entre les différentes ressources du système. Sa modélisation est cruciale pour que les décisions de dispatching reflètent les réalités géographiques et logistiques du site.

Représentation graphique du réseau

Le réseau routier est modélisé comme un graphe pondéré $G = (V, E)$ où :

— V est l'ensemble des nœuds représentant les points d'intérêt : points de chargement (pelles), points de déchargement (concasseurs, stockage), jonctions et intersections.

— E est l'ensemble des arêtes représentant les connexions routières entre nœuds.

— Chaque arête (u, v) est pondérée par la distance euclidienne, le temps de trajet ou un coût composé.

Ce graphe peut être orienté (sens uniques, routes à sens limité) ou non-orienté (routes bidirectionnelles) selon la configuration du site.

La figure 3.2 donne une représentation concrète du graphe pondéré $G = (V, E)$ formalisé ci-dessus. Les nœuds y figurent les points d'intérêt (pelles, dumps, jonctions) et les arcs les liaisons routières, chacun porteur d'attributs (distance, temps, état). La nature stochastique des pondérations, symbolisée sur le schéma, traduit la variabilité des temps de trajet que l'agent devra apprendre à anticiper.

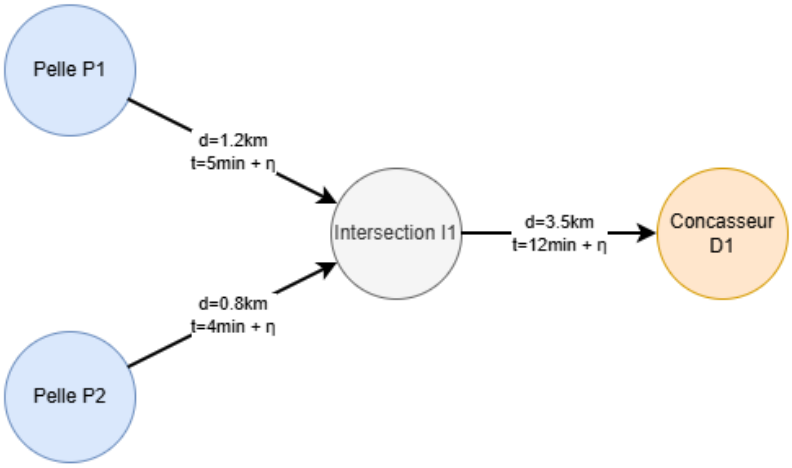


FIGURE 3.2 – Représentation du réseau routier sous forme de graphe stochastique.

Matrices de temps et matrices de coût

À partir du graphe, on calcule deux matrices fondamentales :

— **Matrice de temps** T : T_{ij} est le temps de trajet moyen (en minutes) du nœud i au nœud j, calculé via l'algorithme de plus court chemin (Dijkstra ou Floyd-Warshall).

— **Matrice de coût** C : C_{ij} agrège plusieurs dimensions : distance, temps de trajet, consommation de carburant [25], usure du véhicule. On peut utiliser une combinaison

linéaire pondérée :

$$C_{ij} = \alpha \cdot T_{ij} + \beta \cdot D_{ij} + \gamma \cdot E_{ij} \tag{3.1}$$

où D_{ij} est la distance, E_{ij} est une estimation de l'usure ou du coût énergétique, et α, β, γ sont des poids de pondération.

Exemple numérique : Si $\alpha = 0{,}5$, $\beta = 0{,}3$, $\gamma = 0{,}2$, $T_{ij} = 15$ minutes, $D_{ij} = 2{,}5$ km, $E_{ij} = 0{,}8$, alors $C_{ij} = 0{,}5 \times 15 + 0{,}3 \times 2{,}5 + 0{,}2 \times 0{,}8 = 7{,}5 + 0{,}75 + 0{,}16 = 8{,}41$.

Intégration de la variabilité stochastique

Le réseau réel n'est pas déterministe : les temps de trajet varient en fonction de l'état de la route, des conditions météorologiques, et de la charge du camion. On modélise cette variabilité par :

$$T_{ij}(t) = \bar{T}_{ij} + \eta_{ij}(t) \tag{3.2}$$

où $\bar{T}_{ij}$ est le temps moyen et $\eta_{ij}(t)$ est un bruit reflétant la variabilité observée. Les durées étant strictement positives, la variabilité est modélisée par une distribution **log-normale**, plus réaliste qu'une gaussienne pour des variables de temps.

Exemple numérique : Si $\bar{T}_{ij} = 12$ minutes et $\eta_{ij}(t) = 2{,}3$ minutes (échantillon d'une distribution log-normale avec $\sigma = 0{,}12$), alors $T_{ij}(t) = 12 + 2{,}3 = 14{,}3$ minutes.

Cette modélisation stochastique est essentielle pour évaluer la robustesse des politiques apprises, car elle oblige l'agent à anticiper les variations plutôt que de se fier à des temps de trajet figés. Cette exigence de robustesse est également mise en avant dans les travaux récents de RL multi-agent appliqués au dispatching minier [26].

3.4 Formulation du problème en tant que processus de décision markovien

La formalisation du problème de dispatching minier en tant que processus de décision markovien est l'étape décisive qui transforme le système opérationnel en un problème d'apprentissage par renforcement. Ce cadrage rejoint les études récentes de dispatching minier fondées sur des formulations explicites de type MDP, ainsi que les travaux fondateurs en apprentissage par renforcement séquentiel appliqué à des problèmes de routage et d'ordonnancement [11, 21, 27]. Un processus de décision markovien est défini par le 4-tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R})$ où :

Le choix de cette formulation n'est pas uniquement formel. Il permet de condenser l'information utile à la décision tout en évitant une explosion de la dimension d'état, ce qui serait pénalisant pour l'apprentissage et pour la stabilité des politiques estimées.

- $\mathcal{S}$ est l'ensemble des états possibles du système.
- $\mathcal{A}$ est l'ensemble des actions (décisions de dispatching) disponibles.
- $\mathcal{P}(s'|s, a)$ est la probabilité de transition vers l'état s' en exécutant l'action a depuis l'état s.
- $\mathcal{R}(s, a, s')$ est la récompense reçue lors d'une transition.

Espace d'état $\mathcal{S}$

L'état du système à chaque instant t est représenté par un vecteur de variables d'état contenant les informations essentielles pour la prise de décision :

$$s_t = \Big(\{q_p\}_{p\in\text{Pelles}},\ \{x_c\}_{c\in\text{Camions}},\ \{z_r\}_{r\in\text{Ressources}},\ t_{\text{courant}}\Big) \tag{3.3}$$

où q_p désigne la longueur de la file d'attente à la pelle p, x_c la localisation du camion c, et z_r l'état de disponibilité de la ressource r.

Exemple numérique : Pour une mine avec 3 pelles, 12 camions et 2 dumps à $t_{\text{courant}} =$ 240 minutes (4 heures) : $s_t = (\{2,3,1\},\ \{\text{yard}, \text{shovel}_1, \text{dump}_1, \ldots\},\ \{0,5\},\ 240)$ où les files d'attente aux pelles sont $\{2, 3, 1\}$ camions respectivement, les camions sont répartis sur le réseau, et les dumps seront disponibles dans $\{0, 5\}$ minutes.

De manière plus détaillée :

— **Files d'attente aux pelles** : Pour chaque pelle p, on enregistre le nombre de camions en attente et leur temps d'arrivée estimé.

— **Localisation des camions** : Pour chaque camion c, on enregistre son dernier nœud de présence et son statut (libre, en route, en charge, en décharge).

— **Disponibilité des ressources** : Pour chaque pelle ou section de route, on enregistre si elle est disponible, en maintenance ou indisponible.

— **Temps courant** : L'heure de la journée ou l'étape de simulation, qui peut influencer la dynamique (sauf au sein d'un épisode stationnaire).

Espace d'action $\mathcal{A}$

À chaque étape de décision, l'agent doit décider d'assigner certains camions libres à des pelles disponibles. L'espace d'action dépend de l'état courant et est construit comme suit :

$$a_t = \text{Assignment}(c_1 \to p_j, c_2 \to p_k, \ldots) \quad \text{ou} \quad a_t = \text{ATTENDRE} \tag{3.4}$$

Une action valide est un ensemble d'appairements camion-pelle qui respectent les contraintes :

— Le camion doit être libre (non actuellement en transit, en charge ou en décharge).

— La pelle doit être disponible et produire un type de minerai compatible avec la capacité du camion.

— L'action peut aussi être d'attendre (pas de changement) si attendre est plus profitable à long terme.

L'ensemble $\mathcal{A}(s)$ des actions possibles dans l'état s peut être très large (combinatoire exponentiel). Pour rendre le problème traitable, on peut limiter l'espace à un nombre réduit d'actions par exemple assigner au maximum k camions par étape, ou sélectionner les n meilleures paires candidate selon une heuristique.

Cette restriction constitue un compromis assumé : une action décrite comme une affectation complète de la flotte serait plus expressive, mais elle rendrait l'espace de recherche nettement moins maniable. Le modèle retient donc une granularité suffisante pour capturer l'essentiel du dispatching sans faire basculer le problème dans une complexité disproportionnée.

Modèle de transition $\mathcal{P}(s'|s,a)$

Le modèle de transition décrit comment le système évolue suite à une action. Étant donné un état s et une action a, le nouvel état s' est déterminé par :

$$s'_t = \text{Simuler}(s_t, a_t, \Delta t) \tag{3.5}$$

où Δt est l'intervalle de temps entre deux décisions.

La figure 3.3 schématise la boucle d'interaction au cœur du cadre MDP : à chaque pas, l'agent observe l'état s_t, choisit une action a_t, et l'environnement (le simulateur minier) renvoie l'état suivant s_{t+1} et la récompense r_t. Cette représentation montre que l'apprentissage repose entièrement sur cette boucle fermée, où la politique se construit par essais successifs plutôt que par une résolution analytique.

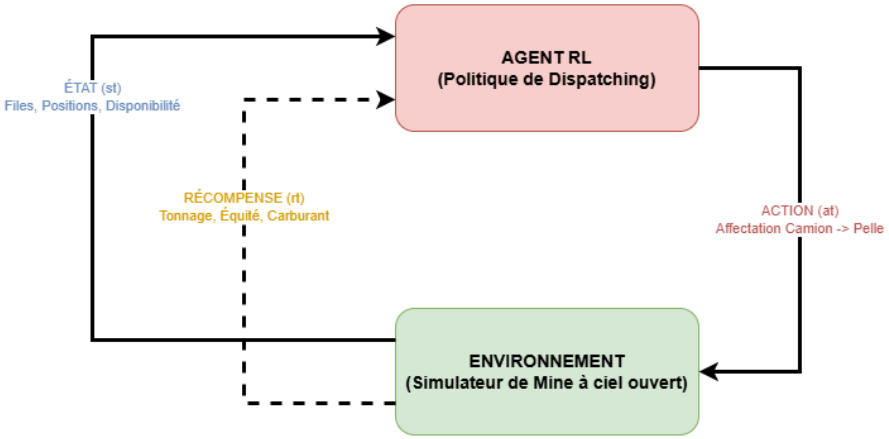


FIGURE 3.3 – Schéma d'interaction entre l'agent RL (politique) et l'environnement (simulateur minier).

La simulation inclut :

- **Avancement des camions en transit** : Les camions se déplacent selon les matrices de temps du réseau, avec perturbations stochastiques.
- **Mise à jour des files d'attente de chargement** : Les camions arrivent aux pelles, se joignent à la file ou commencent le chargement.
- **Événements de déchargement** : Les camions qui atteignent les points de déchargement sont marqués comme déchargés et reviennent en état libre.
- **Dynamique des pelles** : Chaque pelle continue de servir les camions selon un ordre FIFO ou de priorité.

Formellement, le modèle de transition est une fonction déterministe augmentée de sources de stochasticité (variation des temps de trajet). Cette représentation est alignée avec les environnements de benchmark configurables récemment proposés pour le dispatching minier et avec les travaux de simulation dédiés à l'évaluation de règles de dispatching sous incertitude [10, 28] :

$$s'_t = f(s_t, a_t) + \eta_t, \quad \eta_t \sim \mathcal{N}(0, \Sigma) \tag{3.6}$$

où $f(s_t, a_t)$ est la transition déterministe et η_t représente les perturbations stochastiques.

Fonction de récompense $\mathcal{R}(s, a, s')$

La récompense reflète les objectifs opérationnels. Le rendement global du système (tonnage par unité de temps) peut être décomposé en plusieurs objectifs :

$$R_t = w_1 \cdot R_{\text{rendement}}(s_t, s'_t) + w_2 \cdot R_{\text{équité}}(s_t) + w_3 \cdot R_{\text{coût}}(a_t) \tag{3.7}$$

Exemple numérique : Si $w_1 = 1{,}0$, $w_2 = 0{,}1$, $w_3 = 0{,}05$, $R_{\text{rendement}} = 280$ tonnes, $R_{\text{équité}} = -5{,}2$ (pénalité due à files inégales), $R_{\text{coût}} = -14{,}66$ (coût des trajets des deux camions assignés, voir sous-exemple ci-dessous), alors $R_t = 1{,}0 \times 280 + 0{,}1 \times (-5{,}2) + 0{,}05 \times (-14{,}66) = 280 - 0{,}52 - 0{,}73 = 278{,}75$.

où :

— $R_{\text{rendement}}$: Récompense proportionnelle au tonnage livré à la destination au cours du pas de temps Δt. C'est l'objectif principal.

$$D_t = \{c \in \text{Camions} \mid c \text{ est déchargé dans } [t, t + \Delta t]\} \tag{3.8}$$

$$R_{\text{rendement}}(s_t, s'_t) = \sum_{c \in D_t} \text{capacité}(c) \tag{3.9}$$

Exemple numérique : Si 2 camions sont déchargés dans l'intervalle $[t, t + \Delta t]$ avec des capacités de 140 tonnes chacun, alors $R_{\text{rendement}} = 140 + 140 = 280$ tonnes.

— $R_{\text{équité}}$: Récompense encourageant une utilisation équilibrée des ressources (prévenir la surcharge d'une pelle au détriment d'autres).

$$R_{\text{équité}}(s_t) = -\lambda \cdot \phi\left(\{q_p\}_{p \in \text{Pelles}}\right) \tag{3.10}$$

Exemple numérique : Si $\lambda = 0{,}5$ et les files d'attente sont $\{q_p\} = \{2, 5, 3\}$ camions, la variance est $\phi(\{2, 5, 3\}) = \text{Var}(\{2, 5, 3\}) = 2{,}33$, alors $R_{\text{équité}} = -0{,}5 \times 2{,}33 = -1{,}17$.

où

$$\phi\left(\{q_p\}_{p \in \text{Pelles}}\right) = \begin{cases} \text{Var}\left(\{q_p\}_{p \in \text{Pelles}}\right), & \text{si } |\text{Pelles}| \geq 2, \\ 0, & \text{si } |\text{Pelles}| = 1. \end{cases} \tag{3.11}$$

Cette définition garantit que le terme d'équité reste bien défini, même lorsqu'une seule pelle est présente.

— $R_{\text{coût}}$: Pénalité liée aux décisions (Pour décourager les affectations inefficaces ou les trajets excessifs).

$$R_{\text{coût}}(a_t) = -\sum_{(c \to p) \in a_t} C(\text{pos}(c), \text{pos}(p)) \tag{3.12}$$

Exemple numérique : Si l'action a_t assigne 2 camions avec des coûts de trajet $C = 8{,}41$ et $C = 6{,}25$, alors $R_{\text{coût}} = -(8{,}41 + 6{,}25) = -14{,}66$.

Les poids w_1, w_2, w_3 permettent de moduler l'importance relative de chaque objectif selon les priorités métier.

La figure 3.4 synthétise la composition multi-objectif de la récompense globale détaillée par l'équation précédente. Elle montre comment les trois signaux — rendement (positif), équité et coût (pénalités) — sont agrégés via leurs poids w_1, w_2, w_3 en un unique scalaire. Cette vue d'ensemble fait apparaître la hiérarchie des priorités métier : le rendement domine, l'équité et le coût jouant un rôle de régulation.

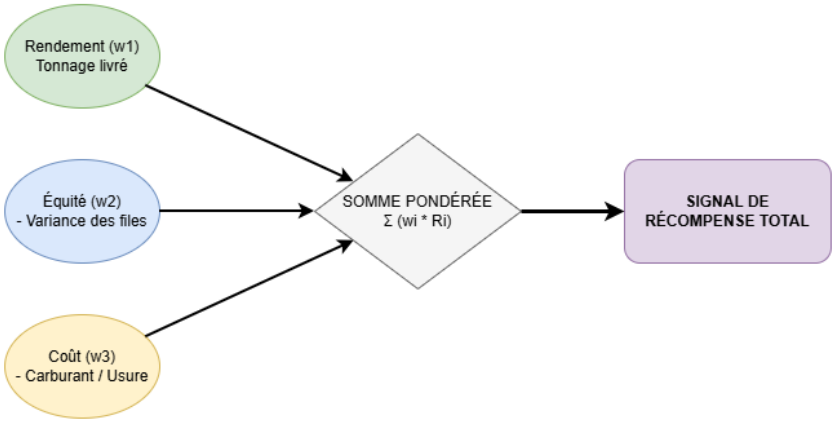


FIGURE 3.4 – Structure multi-objectif de la fonction de récompense globale.

Propriété markovienne et implications

L'hypothèse clé est que l'état s_t contient toute l'information nécessaire pour prédire l'évolution future du système indépendamment du passé (propriété markovienne). Cette hypothèse est raisonnable si :

- Les décisions passées ont déjà eu leur effet sur l'état courant (les camions sont soit en transit, soit en file d'attente, soit libres).
- Les seules sources d'incertitude future sont les variations stochastiques des temps de trajet et les arrivées de nouveaux travaux (qui suivent une distribution connue).

Cette propriété justifie l'application d'algorithmes d'apprentissage par renforcement standards qui supposent un MDP, plutôt que d'exiger une mémoire d'observations passées.

3.5 Conclusion

Ce chapitre a établi le cadre formel de la modélisation du problème de transport minier, depuis la description du système camion-pelle jusqu'à sa formulation en processus de décision markovien. Il a structuré les composantes essentielles du problème, notamment l'état, l'action, la dynamique de transition et la fonction de récompense, afin de représenter de manière cohérente la logique décisionnelle du dispatching en environnement minier.

La modélisation proposée articule réalisme opérationnel et tractabilité algorithmique, en intégrant les contraintes du terrain (congestion, disponibilité des ressources, variabilité des

temps de cycle) tout en restant exploitable par des méthodes d'apprentissage par renforcement. Ce socle conceptuel et mathématique sert de base directe au chapitre suivant, consacré à la méthodologie, à la conception de l'environnement de simulation, à l'agent d'apprentissage et au protocole d'évaluation.

Chapitre 4

Méthodologie, conception et choix algorithmique de la solution proposée

Introduction

Ce chapitre constitue la passerelle entre la modélisation formelle du problème, présentée au Chapitre 3, et l'implémentation expérimentale qui sera détaillée au Chapitre 5. L'objectif est de rendre la progression scientifique lisible : partir de la méthodologie CRISP-RL, construire l'environnement de simulation, introduire les algorithmes classiques d'apprentissage par renforcement, puis justifier le passage vers le Deep Reinforcement Learning.

Nous présentons d'abord le cadre méthodologique CRISP-RL qui a structuré l'ensemble du projet. Ensuite, nous détaillons la conception du simulateur de mine, l'architecture générale de l'approche proposée, puis nous introduisons les algorithmes classiques d'apprentissage par renforcement (Q-Learning, TD Learning, SARSA) comme bases théoriques et modèles comparatifs. Nous justifions ensuite le passage au Deep Reinforcement Learning via DQN, puis le choix de PPO comme approche finale. Enfin, nous définissons la stratégie d'évaluation comparative et les indicateurs de performance.

4.1 Méthodologie

Pour garantir la rigueur scientifique et la reproductibilité de notre approche, nous adoptons le cadre méthodologique CRISP-RL (CRoss-Industry Standard Process for Reinforcement Learning Applications), tel que proposé par [9]. Ce standard est une évolution du CRISP-DM traditionnel, spécifiquement repensée pour les systèmes apprenant par interaction dynamique.

4.1.1 Justification du choix de CRISP-RL

Le choix du CRISP-RL s'appuie sur la nature interactive et incertaine de la logistique minière. Contrairement à une analyse de données statiques où l'on cherche à prédire une

cible, le dispatching minier exige de piloter un système en temps réel. La logistique minière est un problème dynamique, séquentiel et incertain : les décisions de dispatching sont prises de manière répétée, les résultats de chaque décision dépendent de l'état courant du système, et les perturbations stochastiques (pannes, congestion, dégradation des routes) modifient continuellement les conditions opérationnelles.

Le CRISP-RL est le seul cadre permettant de gérer efficacement la boucle de rétroaction entre l'agent (le système de dispatching) et son environnement (la mine). Il intègre les phases de modélisation du simulateur comme une étape centrale du projet, assurant que l'agent n'apprend pas sur des données mortes, mais sur une dynamique de flux vivante et stochastique.

4.1.2 Adaptation des étapes CRISP-RL au contexte minier

Conformément aux travaux de Szatmári [9], nous suivons un cycle de vie en six étapes itératives, adaptées au contexte de la logistique minière :

- **Compréhension du problème métier** : Identification des enjeux de dispatching, routage, files d'attente, consommation de carburant et coûts opérationnels.
- **Modélisation de l'environnement** : Construction d'un simulateur fidèle représentant les camions, pelles, routes, points de chargement et de déchargement.
- **Formulation MDP** : Définition formelle des espaces d'états $\mathcal{S}$ et d'actions $\mathcal{A}$, ainsi que de la fonction de transition $\mathcal{P}$ et de la récompense $\mathcal{R}$ (développées au Chapitre 3).
- **Développement des agents RL** : Implémentation progressive des algorithmes : Q-Learning, TD Learning, SARSA, DQN, puis PPO.
- **Entraînement et test** : Phase d'apprentissage intensif où l'agent explore des milliers de scénarios de dispatching dans le simulateur.
- **Évaluation comparative** : Vérification de la performance de l'agent par rapport aux heuristiques industrielles et aux méthodes RL classiques, selon des indicateurs opérationnels quantitatifs.

La figure 4.1 représente le cycle de vie CRISP-RL en six étapes adapté au contexte minier. Sa lecture circulaire souligne le caractère itératif de la démarche : chaque étape alimente la suivante, et l'évaluation finale peut renvoyer à la modélisation de l'environnement ou à la formulation MDP. C'est cette boucle de rétroaction qui distingue CRISP-RL d'un processus linéaire d'analyse de données statiques.

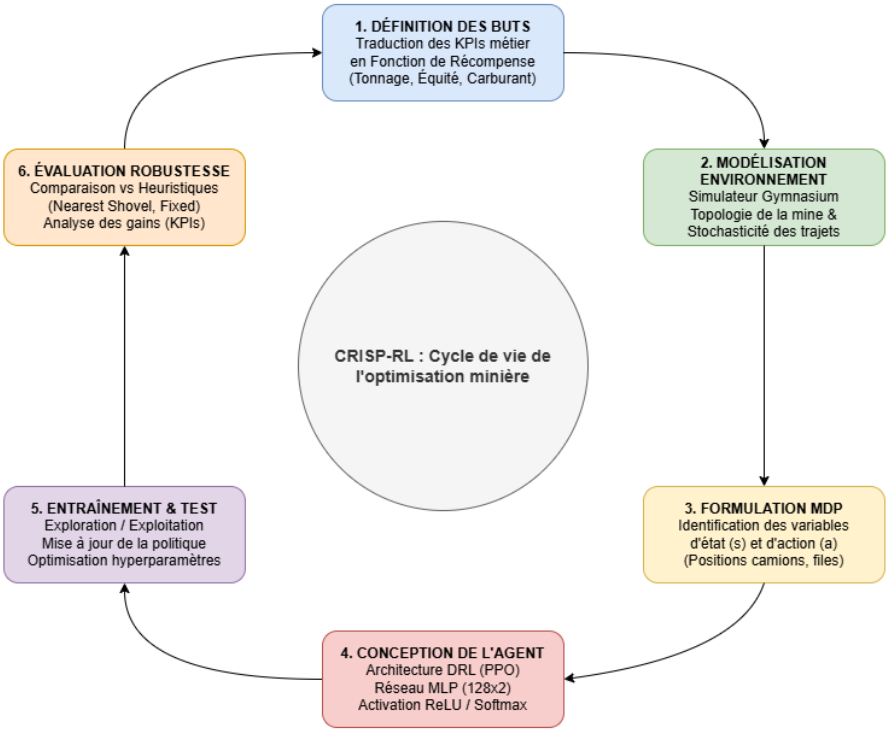


FIGURE 4.1 – Cycle de vie CRISP-RL adapté à la logistique minière.

4.1.3 Positionnement de l'approche proposée dans le cycle CRISP-RL

L'approche proposée ne relève pas seulement d'un choix algorithmique. Elle s'inscrit dans un cycle complet : compréhension métier, simulation, formalisation MDP, apprentissage, expérimentation et validation par indicateurs opérationnels. Chaque étape du cycle CRISP-RL alimente la suivante et permet un retour itératif : si les résultats de l'évaluation révèlent des insuffisances, le cycle permet de revenir à la modélisation de l'environnement ou à la formulation MDP pour affiner le système.

4.2 Présentation générale de l'approche proposée

L'architecture de la solution retenue s'articule autour de deux dimensions complémentaires : le principe fondamental de l'agent intelligent et les composantes fonctionnelles qui structurent son interaction avec le simulateur. Les sous-sections suivantes développent ces dimensions avant d'entrer dans les détails algorithmiques et expérimentaux.

4.2.1 Principe général de la solution

La solution proposée consiste à concevoir un agent intelligent capable d'observer l'état courant du système minier (positions des camions, files d'attente aux pelles, disponibilité des dumps, état des routes), de choisir une décision de dispatching optimale (affectation camion-pelle-dump), puis d'améliorer progressivement sa politique de décision à partir des récompenses reçues. Cette approche se distingue des heuristiques statiques par sa capacité

d'adaptation : l'agent apprend à réagir aux perturbations dynamiques (pannes, congestion, dégradation des routes) sans nécessiter de reprogrammation manuelle.

4.2.2 Architecture fonctionnelle de l'approche

L'architecture fonctionnelle de la solution repose sur quatre blocs principaux, synthétisés dans le Tableau 4.1.

TABLE 4.1 – Architecture fonctionnelle de l'approche proposée

Bloc fonctionnel	**Rôle**
Environnement minier simulé	Représenter les camions, pelles, routes, files d'attente et perturbations stochastiques.
Agent RL	Prendre les décisions de dispatching (affectation camion-pelle-dump).
Fonction de récompense	Évaluer la qualité des décisions selon les objectifs métier (rendement, équité, coût).
Module d'évaluation	Comparer les performances avec les baselines industrielles et les méthodes RL classiques.

Ces blocs interagissent de manière cyclique pour permettre l'apprentissage par renforcement. À chaque instant t, l'environnement minier simulé fournit à l'agent RL une observation de l'état du système (files d'attente, positions des camions, disponibilité des ressources). L'agent sélectionne alors une action de dispatching (affectation camion-pelle-dump), qui est exécutée par l'environnement. La fonction de récompense évalue immédiatement cette action selon les objectifs métier (maximisation du tonnage, minimisation des temps d'attente, réduction de la consommation de carburant). Cette récompense guide l'agent pour ajuster sa politique de décision. Enfin, le module d'évaluation permet de comparer les performances de l'agent RL avec des baselines industrielles (FIFO, Fixed Assignment, Nearest Shovel, Shortest Path) et d'autres algorithmes d'apprentissage par renforcement (Q-Learning, SARSA, DQN) afin de valider l'efficacité de l'approche proposée. Ce flux d'interaction est schématisé dans l'architecture fonctionnelle présentée à la Figure 4.2 ci-dessous.

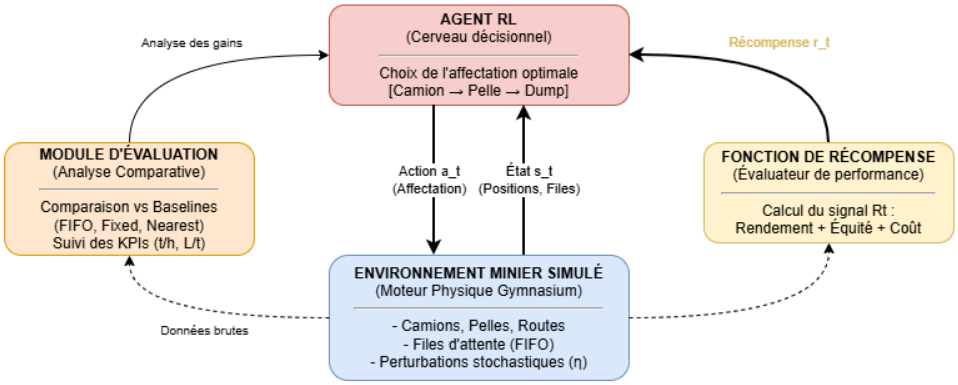


FIGURE 4.2 – Architecture fonctionnelle de la solution proposée.

4.2.3 Positionnement de l'agent dans le système de gestion de flotte

L'agent d'apprentissage par renforcement est conçu comme une couche intelligente d'aide à la décision. Il ne remplace pas entièrement le Fleet Management System (FMS), mais propose des décisions de dispatching optimisées à partir de l'état courant de la mine et des objectifs opérationnels. Le FMS reste responsable de la communication avec les équipements, de la sécurité et du suivi en temps réel, tandis que l'agent RL se concentre sur l'optimisation des affectations camion-pelle-dump.

4.3 Conception de l'environnement de simulation

L'environnement est le monde dans lequel l'agent va évoluer. Sa fidélité par rapport à la réalité minière est indispensable pour la validité des résultats, l'usage de la simulation étant reconnu comme l'approche standard pour évaluer ces systèmes complexes [29].

4.3.1 Architecture générale du simulateur

Nous avons développé un simulateur adapté en langage Python, en utilisant l'interface standardisée Gymnasium [30]. Gymnasium fournit un cadre rigoureux représentant un processus de décision markovien (MDP) via une classe de haut niveau nommée `Env`. La structure de notre simulateur repose sur quatre fonctions et attributs clés :

- **observation_space & action_space** : Ces attributs définissent le format des données. Nous utilisons l'espace `Box` pour les observations continues (positions, files) et `Discrete` pour les décisions de dispatching.
- **reset(seed, options)** : Initialise la mine pour un nouvel épisode de travail. Cette méthode renvoie l'observation initiale s_0 accompagnée d'un dictionnaire `info` pour le débogage.
- **step(action)** : C'est le cœur de la boucle Agent-Environnement. À partir d'une décision de dispatching, elle fait évoluer la mine de Δt et renvoie cinq valeurs cruciales : l'observation suivante (s_{t+1}), la récompense (r_t), ainsi que les signaux `terminated` (but atteint/échec), `truncated` (limite de temps du poste atteinte) et `info`.
- **render()** : Permet la visualisation de l'état de la flotte et des pelles pour le suivi humain.

La figure 4.3 détaille l'organisation modulaire du simulateur autour de l'interface Gymnasium. On y distingue le rôle des quatre méthodes clés (`reset`, `step`, `render` et les espaces d'observation/action), qui structurent la boucle agent–environnement. Cette modularité garantit que l'agent interagit avec la mine via une interface standardisée, indépendamment de la complexité interne de la simulation.

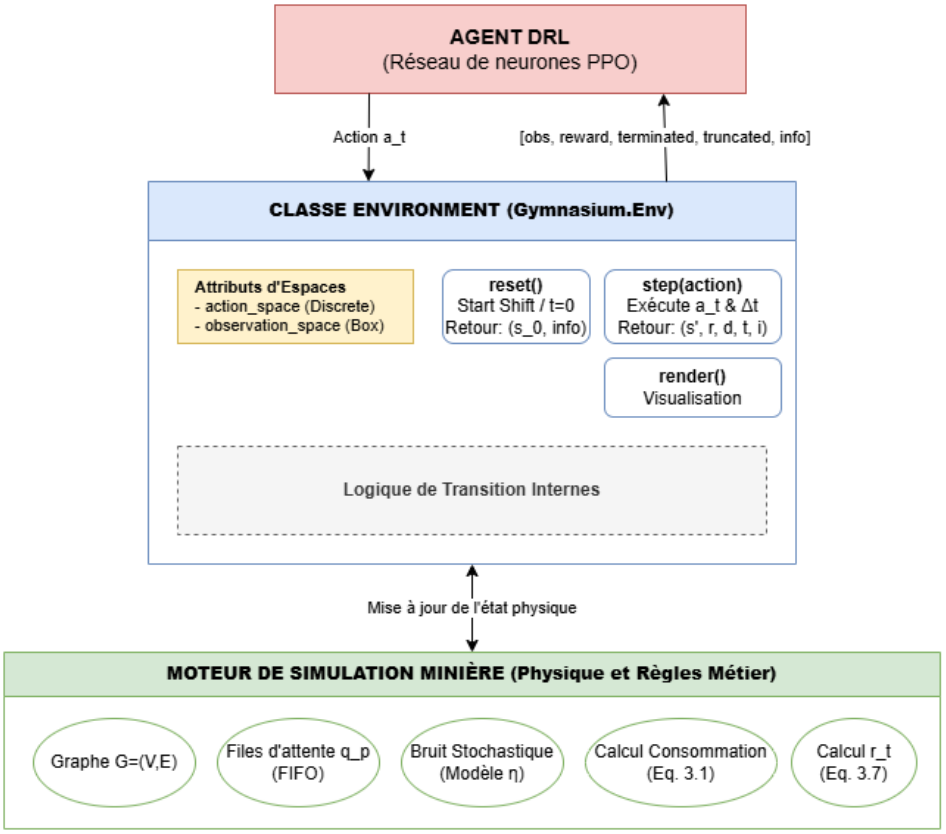


FIGURE 4.3 – Architecture modulaire du simulateur Gymnasium.

4.3.2 Modélisation des entités minières

Le simulateur repose sur une architecture orientée objet (POO) où chaque entité physique est reliée aux variables formelles du MDP défini au Chapitre 3. La correspondance entre les classes logicielles et les composantes de l'espace d'état s_t (Eq. 3.3) est établie dans le Tableau 4.2.

TABLE 4.2 – Correspondance entre les classes logicielles et les variables du MDP

Classe	**Rôle dans le MDP**	**Variable(s) encodée(s)**	**Eq.**
`Truck`	Entité mobile, collecte KPIs	x_c (localisation, statut)	Eq. 3.3
`Shovel`	Ressource de chargement	q_p (file d'attente), z_r	Eq. 3.3
`DumpSite`	Ressource de déchargement	z_r (disponibilité)	Eq. 3.3
`RoadGraph`	Réseau routier paramétrable	Arcs (u, v), distances, pentes	Eq. 3.2
`Environment`	Orchestrateur, logique $f(s_t, a_t)$	Transition s'_t (Eq. 3.6)	Eq. 3.5–3.6

Camions (Truck) : La classe `Truck` encode la composante x_c de l'espace d'état s_t, représentant la localisation et le statut courant de chaque camion $c \in$ Camions. Elle suit en temps réel l'état de disponibilité, les cycles complétés, le tonnage transporté, le carburant consommé et les temps d'attente cumulés.

Pelles (Shovel) : La classe `Shovel` encode la composante q_p de l'espace d'état, soit la file d'attente à chaque pelle $p \in$ Pelles. Les paramètres de chargement ($\bar{T}$, σ) permettent d'échantillonner des durées stochastiques (Eq. 3.2).

Points de déchargement (DumpSite) : La classe `DumpSite` encode la disponibilité z_r des ressources de déchargement dans s_t. Les paramètres de déchargement introduisent la même stochasticité, simulant les goulots d'étranglement potentiels en fin de parcours.

Réseau routier (RoadGraph) : Le graphe routier $G = (V, E)$ est paramétrable : le nombre de pelles, de dumps et d'intersections est configurable. Chaque arête encode la distance, la pente, l'état de la route et les temps de trajet stochastiques.

Environnement (MineEnv) : Cette classe implémente la fonction de transition $f(s_t, a_t)$ augmentée des perturbations stochastiques η_t (Eq. 3.6). Elle orchestre les interactions entre entités et expose l'interface standardisée `reset()` / `step()` / `render()` de Gymnasium [30].

4.3.3 Définition de l'espace d'observation

L'espace d'observation encode l'état complet du système minier dans un vecteur normalisé $s_t \in [0, 1]^n$. Il comprend :

- **Position des camions** : localisation dans le graphe routier.
- **Statut des camions** : libre, en route vers pelle, en chargement, en route vers dump, en déchargement, en retour.
- **Disponibilité des pelles** : temps estimé avant disponibilité (proxy de la longueur de file).
- **Disponibilité des dumps** : temps estimé avant disponibilité au point de déchargement.
- **Temps courant** : progression de l'épisode, normalisée par la durée totale du poste.

Cette normalisation dans $[0, 1]$ est cruciale pour stabiliser l'apprentissage et éviter l'explosion du gradient lors de la rétropropagation [1].

4.3.4 Définition de l'espace d'action

À chaque étape de décision, l'agent choisit une action a_t dans un espace discret :

$$\mathcal{A} = \text{Discrete}\big(|\mathcal{P}| \times |\mathcal{D}| + 1\big) \tag{4.1}$$

où $|\mathcal{P}|$ est le nombre de pelles et $|\mathcal{D}|$ le nombre de dumps. Chaque action encode une paire (pelle, dump) par division entière :

Exemple numérique : Pour 3 pelles et 2 dumps, $|\mathcal{P}|\times|\mathcal{D}|+1 = 3\times2+1 = 7$ actions possibles (indices 0 à 6). Si $a_t = 4$, alors shovel_idx $= 4 \div 2 = 2$ (pelle 3) et dump_idx $= 4 \bmod 2 = 0$ (dump 1). Si $a_t = 6$, c'est l'action ATTENDRE.

- shovel_idx $= a_t \div |\mathcal{D}|$: indice de la pelle choisie.
- dump_idx $= a_t \bmod |\mathcal{D}|$: indice du dump choisi.

— Si $a_t = |\mathcal{P}| \times |\mathcal{D}|$: action ATTENDRE (la pelle et le dump les plus tôt disponibles sont sélectionnés automatiquement).

Ce codage permet à l'agent de choisir simultanément la pelle de chargement et le point de déchargement, offrant une flexibilité décisionnelle plus grande que les approches classiques qui ne choisissent que la pelle.

4.3.5 Fonction de récompense multi-objectif

La fonction de récompense traduit les objectifs métier en un signal scalaire guidant l'apprentissage de l'agent. Conformément à l'Eq. 3.7, elle combine trois composantes pondérées :

$$R_t = w_1 \cdot R_{\text{rendement}} + w_2 \cdot R_{\text{équité}} + w_3 \cdot R_{\text{coût}} \tag{4.2}$$

— **Récompense de rendement** ($R_{\text{rendement}}$) : tonnage transporté lors du cycle, normalisé par la capacité du camion. Récompense positive liée à la productivité.
— **Récompense d'équité** ($R_{\text{équité}}$) : pénalité proportionnelle à la variance des files d'attente aux pelles. Favorise une utilisation équilibrée des ressources de chargement.
— **Récompense de coût** ($R_{\text{coût}}$) : pénalité proportionnelle à la distance parcourue. Réduit les trajets inutiles et la consommation de carburant.

4.4 Algorithmes classiques d'apprentissage par renforcement

Avant de présenter l'approche Deep Reinforcement Learning retenue, il est essentiel de comprendre les algorithmes fondamentaux de l'apprentissage par renforcement tabulaire. Ces méthodes constituent la base théorique sur laquelle reposent les approches plus avancées et servent de modèles comparatifs dans notre évaluation.

4.4.1 Q-Learning pour le dispatching minier

Le Q-Learning est une méthode d'apprentissage par renforcement tabulaire qui apprend une fonction $Q(s, a)$ estimant la qualité d'une action a dans un état s donné. L'agent met à jour cette fonction après chaque transition selon la règle :

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha\Big[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)\Big] \tag{4.3}$$

où α est le taux d'apprentissage, γ le facteur d'actualisation et r_t la récompense reçue.

Exemple numérique : Si $Q(s_t, a_t) = 50$, $r_t = 10$, $\gamma = 0{,}99$, $\max_{a'} Q(s_{t+1}, a') = 55$, $\alpha = 0{,}1$, alors $Q(s_t, a_t) \leftarrow 50 + 0{,}1 \times (10 + 0{,}99 \times 55 - 50) = 50 + 0{,}1 \times 14{,}45 = 51{,}45$.

Dans le contexte du dispatching minier, la correspondance est la suivante :

TABLE 4.3 – Correspondance Q-Learning ↔ dispatching minier

Élément Q-Learning	**Correspondance minière**
État s	Situation actuelle de la flotte (files, positions, disponibilités)
Action a	Affectation camion-pelle-dump ou choix d'attendre
Récompense r	Gain de tonnage, réduction du coût, équité des files
Politique π	Règle apprise de dispatching

L'Algorithme 1 formalise l'application du Q-Learning au dispatching minier.

Algorithm 1 Q-Learning pour le dispatching minier

Require: Environnement minier $\mathcal{E}$, taux d'apprentissage α, facteur d'actualisation γ
Require: Taux d'exploration initial $\epsilon = 1{,}0$, décroissance $\epsilon_{\min} = 0{,}01$
Initialiser $Q(s,a) \leftarrow 0$ pour tout $(s,a) \in \mathcal{S} \times \mathcal{A}$
for épisode $= 1, 2, \ldots, N_{\text{épisodes}}$ **do**
  $s_0 \leftarrow \texttt{reset}(\mathcal{E})$ {Initialiser la mine}
  while épisode non terminé **do**
    Choisir a_t par ϵ-greedy :
      Avec probabilité ϵ : $a_t \sim \text{Uniforme}(\mathcal{A})$ {Exploration}
      Sinon : $a_t \leftarrow \arg\max_a Q(s_t, a)$ {Exploitation}
    Décoder a_t en paire (pelle p_i, dump d_j)
    Exécuter a_t dans $\mathcal{E}$: observer r_t, s_{t+1}
    Mettre à jour :
      $Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha\big[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)\big]$
    $s_t \leftarrow s_{t+1}$
  end while
  $\epsilon \leftarrow \max(\epsilon_{\min},\ \epsilon - \Delta\epsilon)$ {Décroissance linéaire}
end for
return Politique $\pi^*(s) = \arg\max_a Q(s,a)$

Le Q-Learning est une méthode *off-policy* : il estime la valeur de l'action optimale ($\max_{a'}$) indépendamment de la politique d'exploration effectivement suivie. Cette caractéristique le rend théoriquement convergent vers la politique optimale dans des environnements tabulaires [31].

4.4.2 Temporal Difference Learning

Le TD Learning (apprentissage par différence temporelle) est le principe général qui sous-tend Q-Learning, SARSA et d'autres méthodes RL. Il met à jour les estimations de valeur à partir de l'écart entre une estimation actuelle et une estimation future, sans attendre la fin complète de l'épisode :

$$V(s_t) \leftarrow V(s_t) + \alpha\Big[r_t + \gamma V(s_{t+1}) - V(s_t)\Big] \tag{4.4}$$

Exemple numérique : Si $V(s_t) = 40$, $r_t = 8$, $\gamma = 0{,}99$, $V(s_{t+1}) = 45$, $\alpha = 0{,}1$, alors $V(s_t) \leftarrow 40 + 0{,}1 \times (8 + 0{,}99 \times 45 - 40) = 40 + 0{,}1 \times 12{,}55 = 41{,}26$.

L'erreur $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$ est appelée *erreur de différence temporelle* (TD error). Ce mécanisme est particulièrement adapté aux environnements séquentiels comme la logistique minière, où les décisions sont prises de manière continue tout au long d'un poste de travail de 8 heures.

Algorithm 2 TD Learning pour le dispatching minier

Require: Environnement minier $\mathcal{E}$, taux d'apprentissage α, facteur d'actualisation γ
Require: Fonction de valeur $V(s)$ à estimer
Initialiser $V(s) \leftarrow 0$ pour tout $s \in \mathcal{S}$
for épisode $= 1, 2, \ldots, N_{\text{épisodes}}$ **do**
  $s_0 \leftarrow \texttt{reset}(\mathcal{E})$ {Initialiser la mine}
  while épisode non terminé **do**
    Choisir a_t selon la politique π (ou aléatoirement pour l'évaluation)
    Décoder a_t en paire (pelle p_i, dump d_j)
    Exécuter a_t dans $\mathcal{E}$: observer r_t, s_{t+1}
    Calculer l'erreur TD : $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$
    Mettre à jour : $V(s_t) \leftarrow V(s_t) + \alpha\delta_t$
    $s_t \leftarrow s_{t+1}$
  end while
end for
return Fonction de valeur $V(s)$

Le TD Learning constitue la base théorique de plusieurs méthodes RL : Q-Learning utilise le TD error pour mettre à jour $Q(s, a)$, et cette même notion sera reprise dans le calcul du GAE (Generalized Advantage Estimation) utilisé par PPO.

4.4.3 SARSA comme variante TD orientée politique

SARSA (State-Action-Reward-State-Action) est une variante *on-policy* du TD Learning. Contrairement au Q-Learning qui estime la valeur de l'action optimale, SARSA apprend la valeur des actions réellement suivies par l'agent :

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha\Big[r_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)\Big] \tag{4.5}$$

Exemple numérique : Si $Q(s_t, a_t) = 50$, $r_t = 10$, $\gamma = 0{,}99$, $Q(s_{t+1}, a_{t+1}) = 52$ (action effectivement prise), $\alpha = 0{,}1$, alors $Q(s_t, a_t) \leftarrow 50 + 0{,}1 \times (10 + 0{,}99 \times 52 - 50) = 50 + 0{,}1 \times 11{,}48 = 51{,}15$.

La différence clé avec le Q-Learning réside dans le terme $Q(s_{t+1}, a_{t+1})$ au lieu de $\max_{a'} Q(s_{t+1}, a')$: SARSA utilise l'action effectivement prise (a_{t+1}) et non l'action optimale hypothétique. Cette distinction entre approches on-policy et off-policy est fondamentale en RL [31].

Dans le contexte du dispatching minier, SARSA tend à produire des politiques plus conservatrices, car il tient compte de l'exploration effectivement réalisée. Cette propriété peut être

avantageuse dans un environnement où les décisions sous-optimales (par exemple, envoyer un camion vers une pelle congestionnée) ont des conséquences coûteuses.

L'Algorithme 3 formalise l'application de SARSA au dispatching minier.

Algorithm 3 SARSA pour le dispatching minier

Require: Environnement minier $\mathcal{E}$, taux d'apprentissage α, facteur d'actualisation γ
Require: Taux d'exploration initial $\epsilon = 1{,}0$, décroissance $\epsilon_{\min} = 0{,}01$
Initialiser $Q(s, a) \leftarrow 0$ pour tout $(s, a) \in \mathcal{S} \times \mathcal{A}$
for épisode $= 1, 2, \ldots, N_{\text{épisodes}}$ **do**
  $s_0 \leftarrow \texttt{reset}(\mathcal{E})$ {Initialiser la mine}
  Choisir a_0 par ϵ-greedy à partir de $Q(s_0, \cdot)$
  while épisode non terminé **do**
    Décoder a_t en paire (pelle p_i, dump d_j)
    Exécuter a_t dans $\mathcal{E}$: observer r_t, s_{t+1}
    Choisir a_{t+1} par ϵ-greedy à partir de $Q(s_{t+1}, \cdot)$
    Mettre à jour (on-policy) :
      $Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha\big[r_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)\big]$
    $s_t \leftarrow s_{t+1}, \quad a_t \leftarrow a_{t+1}$
  end while
  $\epsilon \leftarrow \max(\epsilon_{\min},\ \epsilon - \Delta\epsilon)$ {Décroissance linéaire}
end for
return Politique $\pi^*(s) = \arg\max_a Q(s, a)$

La différence structurelle entre les Algorithmes 1 et 3 se situe à la ligne de mise à jour : Q-Learning utilise $\max_{a'} Q(s_{t+1}, a')$ (off-policy) tandis que SARSA utilise $Q(s_{t+1}, a_{t+1})$ (on-policy), où a_{t+1} est l'action effectivement choisie par la politique ϵ-greedy.

4.4.4 Limites des méthodes tabulaires

Les méthodes tabulaires (Q-Learning, SARSA) deviennent rapidement limitées lorsque l'espace d'état augmente. Dans le cas de la logistique minière, les combinaisons entre :

— les positions de N_c camions (12 camions dans notre configuration),
— les files d'attente de N_p pelles (3 pelles),
— les disponibilités de N_d dumps (2 dumps),
— les niveaux de congestion sur le réseau routier,
— les événements stochastiques (pannes, dégradation des routes),

rendent impossible le stockage explicite de toutes les valeurs $Q(s, a)$ dans une table. L'espace d'état est continu et de haute dimension, ce qui provoque une explosion combinatoire incompatible avec l'approche tabulaire. Cette limitation fondamentale justifie le passage vers le Deep Reinforcement Learning, où un réseau de neurones approxime la fonction de valeur ou la politique.

4.5 Passage au Deep Reinforcement Learning

Les méthodes tabulaires présentées en Section 4.4 atteignent rapidement leurs limites face à la dimensionnalité continue du problème de dispatching minier. Le recours au Deep Reinforcement Learning s'impose alors : une approximation neuronale remplace la table Q discrète et permet de traiter un espace d'observation de 147 dimensions sans discrétisation préalable. Les sous-sections suivantes motivent ce passage et justifient le choix de PPO comme algorithme final.

4.5.1 Justification du recours au Deep Learning

Le recours au Deep Learning pour approximer les fonctions de valeur ou les politiques est justifié par plusieurs caractéristiques de notre problème :

- **Espaces d'état larges** : le vecteur d'observation contient des dizaines de variables continues.
- **Observations continues** : les files d'attente, positions et temps ne sont pas discrétisables efficacement.
- **Interactions complexes** : les décisions pour un camion affectent les files d'attente de tous les autres.
- **Perturbations dynamiques** : pannes, dégradation des routes, variations de chargement/déchargement.
- **Généralisation** : un réseau de neurones peut généraliser à des états non rencontrés pendant l'entraînement, contrairement à une table Q qui ne couvre que les états visités.

4.5.2 Deep Q-Network (DQN) comme extension du Q-Learning

Le Deep Q-Network (DQN) [1] remplace la table Q par un réseau de neurones $Q_\theta(s, a)$ chargé d'approximer la fonction de valeur-action. Il constitue donc une extension profonde du Q-Learning (Eq. 4.3), adaptée aux environnements où la représentation tabulaire devient impossible.

DQN introduit deux mécanismes clés pour stabiliser l'apprentissage :

- **Experience Replay** : stockage des transitions (s, a, r, s') dans un buffer mémoire, d'où sont tirés des mini-batches pour l'entraînement, brisant les corrélations temporelles.
- **Target Network** : un réseau cible Q_{θ^-} mis à jour périodiquement, évitant les oscillations de l'apprentissage.

Bien que DQN représente une avancée majeure par rapport au Q-Learning tabulaire, il présente des limitations dans notre contexte : il est structurellement limité aux espaces d'actions discrets de faible dimension, et la gestion simultanée de plusieurs unités de chargement peut rendre l'apprentissage instable [1]. DQN joue néanmoins un rôle important dans notre travail comme modèle de transition vers les approches actor-critic.

Algorithm 4 DQN pour le dispatching minier

Require: Environnement minier $\mathcal{E}$, réseau Q_θ, réseau cible Q_{θ^-}

Require: Buffer mémoire $\mathcal{B}$ (capacité C), taille de batch B, taux de mise à jour cible τ

Require: Taux d'apprentissage α, facteur d'actualisation γ, taux d'exploration ϵ

Initialiser θ et θ^- aléatoirement, $\theta^- \leftarrow \theta$
Initialiser $\mathcal{B} \leftarrow \emptyset$
for épisode $= 1, 2, \ldots, N_{\text{épisodes}}$ **do**
  $s_0 \leftarrow \texttt{reset}(\mathcal{E})$ {Initialiser la mine}
  while épisode non terminé **do**
    Choisir a_t par ϵ-greedy :
      Avec probabilité ϵ : $a_t \sim$ Uniforme($\mathcal{A}$)
      Sinon : $a_t \leftarrow \arg\max_a Q_\theta(s_t, a)$
    Décoder a_t en paire (pelle p_i, dump d_j)
    Exécuter a_t dans $\mathcal{E}$: observer r_t, s_{t+1}
    Stocker (s_t, a_t, r_t, s_{t+1}) dans $\mathcal{B}$
    Si $|\mathcal{B}| > C$: supprimer la transition la plus ancienne
    Échantillonner mini-batch $\{(s_i, a_i, r_i, s'_i)\}_{i=1}^{B}$ depuis $\mathcal{B}$
    Calculer les cibles : $y_i = r_i + \gamma \max_{a'} Q_{\theta^-}(s'_i, a')$
    Mettre à jour θ par descente de gradient sur $\frac{1}{B}\sum_i (Q_\theta(s_i, a_i) - y_i)^2$
    Périodiquement : $\theta^- \leftarrow \tau\theta + (1-\tau)\theta^-$
    $s_t \leftarrow s_{t+1}$
  end while
  $\epsilon \leftarrow \max(\epsilon_{\min},\ \epsilon - \Delta\epsilon)$
end for
return Politique $\pi(s) = \arg\max_a Q_\theta(s, a)$

4.5.3 Architecture du réseau de neurones

L'agent repose sur une architecture de type Perceptron Multicouche (MLP) comprenant :

- **Couche d'entrée** : sa dimension correspond au nombre de variables dans l'état s_t (positions, charges, temps, files).
- **Couches cachées** : deux couches denses de 128 neurones chacune, utilisant la fonction d'activation **ReLU** ($f(x) = \max(0, x)$) pour capturer les relations non linéaires complexes entre le trafic et la production.
- **Couche de sortie** : une couche de dimension $|\mathcal{A}|$ utilisant une fonction **Softmax** pour transformer les activations en probabilités d'action.

La figure 4.4 représente l'architecture du perceptron multicouche (MLP) sur lequel repose l'agent. On y suit le cheminement de l'information depuis la couche d'entrée (de dimension égale à celle du vecteur d'état) vers la couche de sortie (de dimension $|\mathcal{A}|$), à travers deux couches cachées de 128 neurones activées par ReLU. Cette structure est ce qui permet à l'agent de généraliser à des états jamais rencontrés, là où une table Q serait limitée aux seuls états visités.

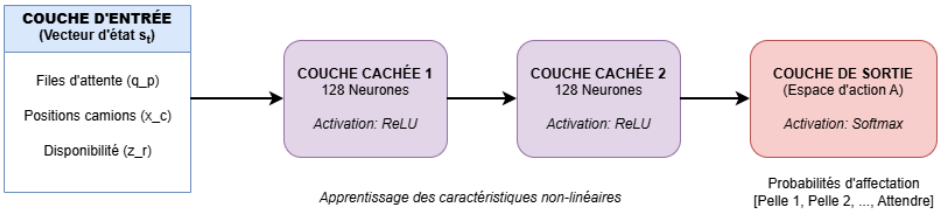


FIGURE 4.4 – Architecture du réseau de neurones (MLP) de l'agent PPO.

4.5.4 Choix de l'algorithme final : PPO

Le choix de l'algorithme final est le résultat d'une progression scientifique : Q-Learning et TD Learning servent de bases théoriques, DQN assure la transition vers le Deep Reinforcement Learning, et PPO (Proximal Policy Optimization) [1, 32] est retenu comme approche finale en raison de sa stabilité et de sa robustesse dans les environnements dynamiques.

La comparaison naturelle s'établit entre DQN et PPO, ces deux algorithmes représentant respectivement l'approche basée sur la valeur et l'approche basée sur la politique dans le spectre du Deep Reinforcement Learning.

Limites de DQN dans notre contexte : DQN constitue l'extension profonde du Q-Learning : il remplace la table Q par un réseau de neurones et introduit l'Experience Replay ainsi que le Target Network pour stabiliser l'apprentissage [1]. Cependant, dans le contexte du dispatching minier, DQN présente deux limitations structurelles. D'une part, il est conçu pour des espaces d'actions discrets et de faible dimension ; or notre espace d'action $\mathcal{A} = \text{Discrete}(|\mathcal{P}| \times |\mathcal{D}| + 1)$ croît combinatoirement avec le nombre de pelles et de dumps. D'autre part, DQN est une méthode off-policy qui peut diverger dans les environnements fortement stochastiques, précisément là où le bruit η_t sur les temps de trajet (Eq. 3.2) perturbe l'estimation des Q-valeurs [1].

Avantages déterminants de PPO : PPO est une méthode actor-critic on-policy qui surmonte directement ces deux limitations :

- **Stabilité par objectif tronqué** : le mécanisme de Clipped Surrogate Objective limite l'amplitude des mises à jour de politique, empêchant les divergences causées par les perturbations stochastiques de l'environnement minier [1, 32].
- **Apprentissage on-policy** : PPO apprend directement à partir de ses interactions actuelles avec le simulateur, sans nécessiter de replay buffer, ce qui le rend plus adapté aux environnements dont la dynamique évolue en cours d'épisode.
- **Robustesse au réglage** : PPO est peu sensible aux hyperparamètres, ce qui garantit la reproductibilité des expériences [1, 9].

Exemple numérique du clipping : Supposons $\hat{A}_t = +2{,}0$ (bonne décision, mieux qu'attendu) et deux cas de ratio $r_t = \pi_\theta(a|s)/\pi_{\theta_{\text{old}}}(a|s)$, avec $\epsilon_{\text{clip}} = 0{,}2$:

- $r_t = 1{,}5$ (nouvelle politique 50 % plus probable sur cette action) : terme non clippé $= 1{,}5 \times 2{,}0 = 3{,}0$; terme clippé $= \text{clip}(1{,}5,\ 0{,}8,\ 1{,}2) \times 2{,}0 = 1{,}2 \times 2{,}0 = 2{,}4$; $L^{\text{CLIP}} =$

$\min(3{,}0,\ 2{,}4) = 2{,}4 \rightarrow$ le clipping plafonne la mise à jour et empêche un changement trop radical.

— $r_t = 1{,}1$ (ajustement modéré, dans la zone $[0{,}8\ ;\ 1{,}2]$) : terme non clippé $= 1{,}1 \times 2{,}0 = 2{,}2$; terme clippé $= 1{,}1 \times 2{,}0 = 2{,}2$; $L^{\text{CLIP}} = 2{,}2 \rightarrow$ pas de clipping, mise à jour normale.

Le tableau 4.4 confronte DQN et PPO sur les critères déterminants pour notre problème. Sa lecture ligne par ligne fait ressortir l'avantage systématique de PPO en matière de stabilité, de robustesse au bruit et de scalabilité de l'espace d'action — propriétés qui justifient son choix comme approche finale, DQN étant conservé comme étape intermédiaire de comparaison.

TABLE 4.4 – Comparaison DQN et PPO pour le dispatching minier (adapté de [1])

Critère	**DQN**	**PPO**
Type d'approche	Basée sur la valeur (off-policy)	**Basée sur la politique (on-policy)**
Espace d'action	Discret de faible dimension	**Discret, scalable**
Stabilité	Moyenne — risque de divergence	**Élevée — objectif tronqué**
Robustesse au bruit η	Faible	**Élevée**
Replay buffer	Requis	**Non requis**
Réglage des hyperparamètres	Simple	**Simple et robuste**
Verdict	Intermédiaire de comparaison	**Approche finale retenue**

4.5.5 Justification du choix de PPO par rapport aux alternatives

PPO est retenu comme approche finale pour notre système de dispatching minier selon trois critères validés par la littérature récente :

1. PPO est plus stable dans les environnements complexes que DQN et ses variantes, grâce à son objectif tronqué qui empêche les mises à jour trop radicales.
2. Il est adapté aux politiques stochastiques, ce qui convient aux environnements dynamiques et bruités comme la mine.
3. Il permet une stratégie de décision adaptative, compatible avec les contraintes opérationnelles de la logistique minière (pannes, congestion, dégradation des routes).

La Figure 4.5 illustre concrètement comment ces propriétés se traduisent dans l'architecture retenue : les observations brutes de la mine sont normalisées, transmises au réseau de neurones PPO, qui produit une décision d'affectation optimale renvoyée à l'environnement simulateur.

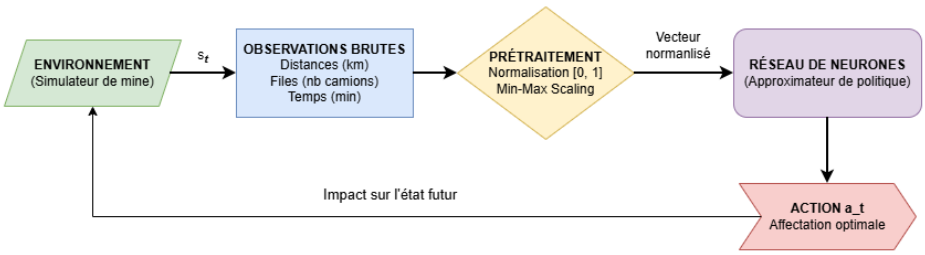


FIGURE 4.5 – Pipeline du flux de données entre la mine et l'agent DRL.

4.6 Configuration expérimentale des modèles d'apprentissage

La performance des agents RL est fortement conditionnée par le choix de leurs hyperparamètres. Les valeurs retenues pour chaque algorithme — Q-Learning, SARSA, DQN et PPO — sont détaillées ci-après, accompagnées de leur justification par des critères de convergence et de reproductibilité.

4.6.1 Hyperparamètres du Q-Learning

Les hyperparamètres du Q-Learning tabulaire sont définis dans le Tableau 4.5.

TABLE 4.5 – Hyperparamètres du Q-Learning

Hyperparamètre	**Valeur**	**Justification**
Taux d'apprentissage (α)	0.2	Vitesse de mise à jour de la Q-table.
Facteur d'actualisation (γ)	0.99	Équilibre récompenses immédiates et futures.
Taux d'exploration (ϵ)	$1.0 \rightarrow 0.01$	Décroissance linéaire (exploration $\rightarrow$ exploitation).
Nombre d'épisodes	30 000	Nécessaire pour la convergence effective de la Q-table.

4.6.2 Hyperparamètres du TD Learning / SARSA

Les hyperparamètres de SARSA suivent les mêmes conventions que le Q-Learning, la seule différence étant la règle de mise à jour on-policy (Eq. 4.5).

TABLE 4.6 – Hyperparamètres du TD Learning / SARSA

Hyperparamètre	**Valeur**	**Justification**
Taux d'apprentissage (α)	0.2	Identique au Q-Learning pour comparabilité.
Facteur d'actualisation (γ)	0.99	Cohérent avec les autres méthodes.
Stratégie ϵ-greedy	$1.0 \rightarrow 0.01$	Décroissance identique au Q-Learning.
Nombre d'épisodes	30 000	Critère de convergence identique.

4.6.3 Hyperparamètres du modèle Deep Reinforcement Learning

Les hyperparamètres retenus pour PPO correspondent aux valeurs par défaut de la bibliothèque Stable-Baselines3 (version 2.x) [33]. Ces valeurs ont été optimisées pour offrir une convergence stable sur une large gamme d'environnements Gymnasium.

TABLE 4.7 – Hyperparamètres de l'agent PPO

Hyperparamètre	**Valeur**	**Justification**
Learning Rate (α)	0.0003	Vitesse d'apprentissage recommandée par défaut.
Gamma (γ)	0.99	Équilibre récompenses immédiates et futures.
Batch Size	64	Taille du lot pour chaque mise à jour du réseau.
GAE Lambda (λ)	0.95	Generalized Advantage Estimation, réduit la variance.
Clip Range (ϵ_{clip})	0.2	Objectif tronqué empêchant les mises à jour trop radicales.
Architecture	MLP 128×128	Deux couches cachées de 128 neurones, activation ReLU.

4.6.4 Stratégie exploration / exploitation

Les stratégies d'exploration diffèrent selon les algorithmes :

— **Q-Learning et SARSA** : exploration ϵ-greedy avec décroissance linéaire. L'agent choisit une action aléatoire avec probabilité ϵ et l'action gloutonne avec probabilité $1-\epsilon$. ϵ décroît de 1.0 (exploration pure) à 0.01 (exploitation quasi-totale) au cours de l'entraînement.

— **PPO** : exploration stochastique intégrée dans la politique. L'acteur π_θ produit une distribution de probabilités sur les actions, et l'exploration est naturellement régulée par l'entropie de cette distribution. Un coefficient d'entropie encourage l'exploration en début d'entraînement.

L'exploration stochastique de PPO est plus nuancée que le ϵ-greedy : au lieu de choisir

aléatoirement parmi toutes les actions avec probabilité ϵ, PPO attribue des probabilités proportionnelles à la qualité estimée de chaque action, permettant une exploration dirigée et plus efficace.

4.7 Stratégie d'évaluation comparative

L'évaluation de l'agent RL ne peut être rigoureuse qu'en le confrontant à des références représentatives des pratiques industrielles. Les heuristiques et modèles de comparaison retenus, les indicateurs de performance mobilisés et le protocole de comparaison équitable entre toutes les approches sont présentés dans les sous-sections suivantes.

4.7.1 Heuristiques de référence

Pour démontrer la plus-value de l'agent RL, nous le comparons aux quatre heuristiques industrielles de référence suivantes :

- **FIFO (First In, First Out)** : le camion disponible est affecté à la pelle dont le temps d'attente est minimal (idle depuis le plus longtemps). Simple et équitable, cette règle ne tient compte ni de la distance ni de la charge globale [4].
- **Nearest Shovel (Pelle la plus proche)** : le camion est affecté à la pelle géographiquement la plus proche via Dijkstra, puis au dump le plus proche de cette pelle. Minimise le trajet à vide mais ignore les files d'attente [4].
- **Shortest Path (Chemin le plus court)** : choisit la paire (pelle, dump) minimisant le coût de cycle complet $C_{\text{cycle}} = \alpha T + \beta D + \gamma E$ (Eq. 3.1) sur les trois segments camion→pelle→dump→pelle, avec $\alpha = 0{,}5$, $\beta = 0{,}3$, $\gamma = 0{,}2$ [4].
- **Fixed Assignment (Affectation fixe)** : chaque camion est dédié à une pelle selon un schéma round-robin statique (affectation par modulo du nombre de pelles). Aucune adaptation dynamique n'est possible [13].

Ces quatre heuristiques représentent le spectre des pratiques industrielles courantes : de la règle la plus rigide (Fixed Assignment) à la règle la plus réactive localement (Nearest Shovel). Leur comparaison avec les agents RL permettra de quantifier le gain apporté par l'apprentissage sur chacun des KPIs définis en Section 4.7.4.

4.7.2 Modèles RL classiques de comparaison

En complément des heuristiques, les trois algorithmes RL classiques présentés en Section 4.4 servent de modèles comparatifs :

- **Q-Learning** : baseline tabulaire off-policy. L'agent apprend la fonction $Q(s, a)$ en estimant toujours l'action optimale (Eq. 4.3), indépendamment de la politique d'exploration suivie.
- **TD Learning** : baseline tabulaire fondée sur l'apprentissage par différence temporelle (Eq. 4.4). Il sert de référence théorique intermédiaire, illustrant le principe de mise à jour bootstrappée commun à Q-Learning, SARSA et PPO via le GAE.
- **SARSA** : baseline tabulaire on-policy. L'agent met à jour $Q(s, a)$ en tenant compte de l'action effectivement choisie (Eq. 4.5), produisant des politiques plus conservatrices

que Q-Learning dans les environnements stochastiques.

Ces trois modèles permettent de quantifier le gain apporté par le passage au Deep Reinforcement Learning à deux niveaux : d'abord entre méthodes tabulaires (Q-Learning vs SARSA), puis entre le meilleur tabulaire et l'approche profonde (PPO), conformément à la progression méthodologique du chapitre.

4.7.3 Modèle Deep Reinforcement Learning proposé

DQN peut être testé comme modèle profond intermédiaire, tandis que PPO constitue l'approche finale proposée. La comparaison DQN vs PPO permet d'évaluer l'avantage des méthodes actor-critic par rapport aux méthodes basées sur la valeur pure dans le contexte spécifique du dispatching minier.

4.7.4 Indicateurs de performance (KPIs)

L'évaluation repose sur un ensemble d'indicateurs de performance opérationnels et algorithmiques :

TABLE 4.8 – Indicateurs de performance pour l'évaluation comparative

KPI	**Signification**
Tonnage transporté (P_h)	Productivité globale : $P_h = \frac{\sum \text{Tonnage}(c)}{T_{\text{total}}}$
Temps d'attente moyen (E_t)	Fluidité du système, temps moyen en file par cycle
Consommation spécifique (E_e)	Efficacité énergétique (litres par tonne transportée)
Coût moyen par cycle	Performance économique
Récompense cumulée	Performance RL globale de l'agent
Temps de convergence	Efficacité d'apprentissage (nombre de steps)
Robustesse aux perturbations	Adaptabilité de l'agent face aux pannes et congestion

4.8 Synthèse de la solution proposée

Le Tableau 4.9 récapitule les choix méthodologiques et techniques retenus pour la solution proposée.

TABLE 4.9 – Synthèse de la solution proposée

Élément	**Choix retenu**
Méthodologie	CRISP-RL
Environnement	Simulateur minier compatible Gymnasium
Formalisation	Processus de décision markovien (MDP)
Méthodes classiques	Q-Learning, TD Learning, SARSA
Transition Deep Reinforcement Learning	DQN
Approche finale	PPO (Proximal Policy Optimization)
Baselines	Heuristiques industrielles + RL classiques
Évaluation	KPIs opérationnels + récompense cumulée

4.8.1 Pseudocode de l'algorithme proposé

L'Algorithme 5 formalise la procédure complète d'entraînement de l'agent PPO appliquée à notre problème de dispatching minier. Ce pseudocode intègre les éléments définis dans les sections précédentes : l'état s_t (Eq. 3.1), l'espace d'action $\mathcal{A}$ (Section 4.3.4), la récompense pondérée R_t (Eq. 3.7), le réseau MLP 128×128 (Section 4.5.3) et les hyperparamètres du Tableau 4.7.

Algorithm 5 PPO pour le dispatching minier camion-pelle-dump

Require: Environnement minier $\mathcal{E}$ (graphe G, camions $\mathcal{C}$, pelles $\mathcal{P}$, dumps $\mathcal{D}$)

Require: Réseau acteur π_θ et critique V_ϕ (MLP 128×128, ReLU)

Require: Hyperparamètres : $\alpha = 0{,}0003$, $\gamma = 0{,}99$, $\lambda_{\text{GAE}} = 0{,}95$, $\epsilon_{\text{clip}} = 0{,}2$, $T_{\max}$ steps

1: Initialiser θ et ϕ aléatoirement
2: **for** $k = 1, 2, \ldots, T_{\max}/N$ **do**
3: Collecter N transitions en suivant π_{θ_k} :
4: **for** $t = 1, \ldots, N$ **do**
5: Observer l'état $s_t = (\{q_p\},\ \{x_c\},\ \{z_r\},\ t_{\text{courant}})$
6: Normaliser s_t dans $[0, 1]$ (Section 4.3.3)
7: Échantillonner l'action $a_t \sim \pi_\theta(\cdot \mid s_t)$
8: Décoder a_t en paire (pelle p_i, dump d_j) :
9: $i \leftarrow \lfloor a_t / |\mathcal{D}| \rfloor, \quad j \leftarrow a_t \bmod |\mathcal{D}|$
10: Si $a_t = |\mathcal{P}| \times |\mathcal{D}|$: ATTENDRE
11: Simuler le cycle complet du camion courant c :
12: Trajet $c \to p_i$ (temps log-normal, Eq. 3.2)
13: Attente + chargement à p_i
14: Trajet chargé $p_i \to d_j$
15: Attente + déchargement à d_j
16: Retour à vide $d_j \to p_i$
17: Calculer la récompense (Eq. 3.7) :
18: $R_t = w_1 \cdot R_{\text{rendement}} + w_2 \cdot R_{\text{équité}} + w_3 \cdot R_{\text{coût}}$
19: Observer le nouvel état s_{t+1}
20: Stocker $(s_t, a_t, R_t, s_{t+1}, \log \pi_\theta(a_t \mid s_t))$
21: **end for**
22: Calculer les avantages GAE : $\hat{A}_t = \sum_{l=0}^{T-t} (\gamma \lambda_{\text{GAE}})^l \delta_{t+l}$
23: où $\delta_t = R_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$
24: **for** chaque époque d'optimisation (mini-batches de taille 64) **do**
25: Calculer le ratio : $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_k}(a_t|s_t)}$
26: Mettre à jour l'acteur par l'objectif tronqué :
27: $L^{\text{CLIP}}(\theta) = \mathbb{E}\big[\min\big(r_t \hat{A}_t,\ \text{clip}(r_t, 1 - \epsilon_{\text{clip}}, 1 + \epsilon_{\text{clip}})\hat{A}_t\big)\big]$
28: Mettre à jour le critique :
29: $L^V(\phi) = \mathbb{E}[(V_\phi(s_t) - R_t^{\text{cible}})^2]$
30: $\theta \leftarrow \theta + \alpha \nabla_\theta L^{\text{CLIP}}, \quad \phi \leftarrow \phi - \alpha \nabla_\phi L^V$
31: **end for**
32: **end for**
33: **return** Politique entraînée π_θ^*

Cet algorithme illustre comment les composantes théoriques définies au Chapitre 3 (MDP, récompense pondérée, stochasticité) s'articulent avec les choix techniques du présent chapitre (PPO, architecture réseau, hyperparamètres) pour former une boucle d'apprentissage complète et cohérente. La ligne 9 montre en particulier l'encodage de l'espace d'action

$\mathcal{A} = \text{Discrete}(|\mathcal{P}| \times |\mathcal{D}| + 1)$ qui permet à l'agent de choisir simultanément la pelle et le point de déchargement. Le terme +1 représente l'action neutre (ATTENDRE), permettant à l'agent de temporiser si aucune affectation n'est productive à l'instant t.

Exemple numérique du GAE : Supposons trois erreurs TD consécutives après une bonne décision d'affectation : $\delta_0 = 0{,}8$ (le camion est livré, mieux qu'attendu), $\delta_1 = 0{,}4$ (le cycle suivant se déroule également bien) et $\delta_2 = 0{,}1$ (effet positif résiduel). Avec $\gamma = 0{,}99$, $\lambda = 0{,}95$ et $\gamma\lambda = 0{,}9405$:

$$\hat{A}_0 = \delta_0 + (\gamma\lambda)\,\delta_1 + (\gamma\lambda)^2\,\delta_2 = 0{,}8 + 0{,}9405 \times 0{,}4 + 0{,}9405^2 \times 0{,}1 = 0{,}800 + 0{,}376 + 0{,}088 = 1{,}26 \tag{4.6}$$

L'avantage $\hat{A}_0 = 1{,}26 > 0$ indique que la décision d'affectation prise était significativement meilleure que ce que le critique anticipait : PPO augmentera donc la probabilité de cette action dans cette situation lors de la mise à jour suivante. Si les erreurs avaient été négatives (pelle congestionnée, cycle improductif), $\hat{A}_0$ serait négatif et PPO diminuerait la probabilité de cette action.

4.9 Conclusion

Ce chapitre a présenté la méthodologie complète de notre solution, depuis le cadre CRISP-RL jusqu'au choix algorithmique final. Le passage des méthodes classiques (Q-Learning, TD Learning, SARSA) vers le Deep Reinforcement Learning est méthodologiquement justifié par la complexité du problème minier : espaces d'état continus de haute dimension, perturbations stochastiques et interactions complexes entre entités. Q-Learning et TD Learning sont introduits comme bases théoriques et modèles comparatifs ; DQN joue le rôle de transition vers les réseaux de neurones ; PPO est retenu comme approche finale proposée pour sa stabilité et son adéquation aux environnements dynamiques et stochastiques. Le chapitre suivant présentera l'implémentation technique de ce système, le protocole expérimental, les scénarios de simulation et l'analyse des résultats.

Chapitre 5

Implémentation et expérimentation

Introduction

L'implémentation et l'expérimentation constituent l'étape de validation opérationnelle de la méthodologie proposée. Elles permettent de vérifier, dans un cadre contrôlé, la faisabilité technique de la solution et sa capacité à produire des résultats mesurables sur des scénarios représentatifs de la logistique minière.

Ce chapitre présente d'abord l'environnement de développement et la configuration matérielle utilisés (Section 5.1), puis l'architecture globale du système implémenté (Section 5.2) et les détails d'implémentation de l'environnement de simulation, jusqu'à sa validation (Section 5.3). Les paramètres et protocoles d'entraînement (Section 5.4), les scénarios expérimentaux (Section 5.5) et les métriques d'évaluation (Section 5.6) sont ensuite détaillés, avant la conclusion qui fait le lien avec l'analyse des résultats du Chapitre 6.

5.1 Environnement de développement et outils

L'implémentation repose sur un environnement Python géré par Conda (Anaconda), garantissant la reproductibilité des expériences indépendamment du système d'exploitation. Le packaging respecte les standards modernes PEP 517/518, avec Flit comme build-backend.

Le tableau 5.1 récapitule l'environnement de développement et les bibliothèques mobilisées, avec leur rôle respectif dans la chaîne de traitement. Il met en évidence le caractère entièrement *open-source* et reproductible de la pile logicielle, depuis la simulation (Gymnasium, NetworkX) jusqu'à l'apprentissage (Stable-Baselines3) et la restitution des résultats.

TABLE 5.1 – Environnement de développement et bibliothèques principales

Outil / Bibliothèque	**Version**	**Rôle**
Python	3.11.11	Langage de programmation principal
Conda (Miniconda)	—	Gestion de l'environnement virtuel
Flit	$\geq$3.2	Build system conforme PEP 517/518
Gymnasium	1.2.3	Interface standardisée Agent–Environnement
Stable-Baselines3	2.8.0	Implémentation DQN et PPO
NetworkX	3.6.1	Modélisation et algorithmes du graphe routier
NumPy	2.4.6	Calcul vectoriel et manipulation des observations
Matplotlib	3.10.8	Visualisation du graphe routier (code)
Power BI	—	Figures comparatives des résultats (Ch. 6)
draw.io	—	Diagrammes d'architecture et schémas conceptuels
Git / GitHub	—	Versionnement et hébergement du code source

L'ensemble du code source est versionné avec Git et hébergé sur GitHub, assurant la traçabilité des modifications et la reproductibilité intégrale des expériences via les graines fixes (*seeds*).

5.1.1 Configuration matérielle

L'ensemble du développement, de l'entraînement et de l'évaluation a été réalisé sur l'ordinateur portable dont les caractéristiques sont résumées au Tableau 5.2.

TABLE 5.2 – Configuration matérielle utilisée pour le développement et l'entraînement

Composant	**Spécification**
Modèle	Dell Latitude 5510 (ordinateur portable)
Processeur	Intel Core i5-10310U (1,70 GHz, turbo jusqu'à 2,21 GHz)
Mémoire RAM	16 Go (15,6 Go utilisables)
Carte graphique	Intel UHD 128 Mo (intégrée, affichage uniquement)
GPU dédié	Aucun

Tous les entraînements (méthodes tabulaires et Deep RL) s'exécutent exclusivement sur CPU, ce qui explique les durées d'entraînement rapportées en Section 5.4.

5.2 Architecture globale du système proposé

Le système implémenté repose sur une architecture modulaire en Python, organisée autour de cinq composantes principales interconnectées (Figure 5.1) :

— **Simulateur Gymnasium (`MineEnv`)** : implémente la boucle Agent–Environnement conforme à l'interface standardisée Gymnasium [30], exposant les méthodes `reset()`, `step()` et `render()`.

— **Module de simulation physique** : modélise les entités minières (`Truck`, `Shovel`, `DumpSite`, `RoadGraph`), les temps de trajet stochastiques (distribution log-normale, Eq. 3.2) et les pannes aléatoires des camions.
— **Module d'agents RL** : implémente les quatre algorithmes d'apprentissage — Q-Learning tabulaire, SARSA, DQN et PPO — via la bibliothèque Stable-Baselines3 [33] pour les agents Deep RL.
— **Module des heuristiques de référence** : implémente les quatre baselines industrielles (FIFO, Fixed Assignment, Nearest Shovel, Shortest Path) avec une interface unifiée compatible avec l'environnement Gymnasium.
— **Module d'évaluation et d'analyse** : orchestre les benchmarks sur l'ensemble des scénarios, calcule les KPIs définis au Tableau 4.8 et génère les tableaux et figures comparatifs.

L'ensemble du code source est structuré selon le principe de séparation des responsabilités, garantissant la reproductibilité des expériences : chaque agent est entraîné avec une graine fixe (*seed*), sauvegardé dans un répertoire dédié, puis rechargé pour l'évaluation.

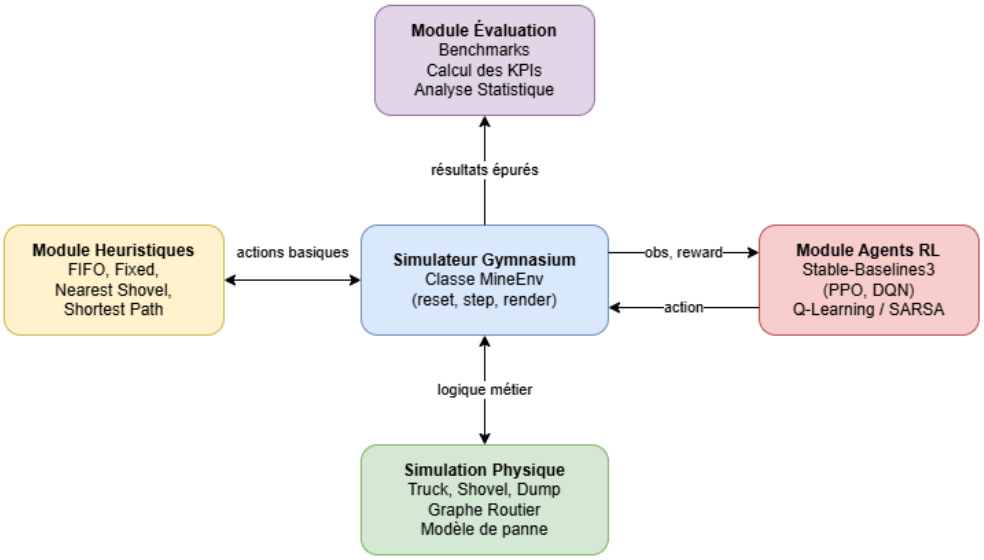


FIGURE 5.1 – Architecture globale du projet.

5.3 Détails d'implémentation de l'environnement

Les choix d'implémentation qui concrétisent la modélisation du Chapitre 3 sont détaillés ci-après. Le simulateur physique, le graphe routier, le vecteur d'observation et la fonction de récompense sont chacun décrits avec leurs valeurs numériques et les justifications techniques qui les accompagnent.

5.3.1 Modélisation physique du simulateur

L'environnement `MineEnv` simule un poste de travail complet de 8 heures ($T = 480$ minutes) pour une flotte de N_c camions circulant entre N_p pelles et N_d points de déchargement.

À chaque décision, le simulateur exécute un cycle complet pour le camion le plus tôt disponible, selon la séquence suivante :

1. Trajet à vide vers la pelle assignée (temps log-normal, $\sigma = 0{,}12$) ;
2. Attente en file si la pelle est occupée ;
3. Chargement (durée moyenne 2 min, $\sigma = 0{,}3$ min) ;
4. Trajet chargé vers le dump assigné ;
5. Déchargement (durée moyenne 1 min, $\sigma = 0{,}2$ min) ;
6. Retour à vide vers la pelle ;
7. Panne aléatoire avec probabilité p_b par cycle (durée uniforme $\mathcal{U}[10, 30]$ minutes).

5.3.2 Graphe routier

Le réseau routier $G = (V, E)$ comprend 7 nœuds pour la configuration nominale : 1 yard, 1 jonction, 3 pelles et 2 dumps (Figure 5.2). Les distances varient de 1,5 à 3,2 km selon les segments, avec des pentes de 3 à 8%, conformément aux paramètres opérationnels réalistes issus de la littérature [2]. La dégradation progressive des routes est modélisée par une réduction stochastique de l'attribut `road_state` à chaque passage (probabilité 5%, réduction de 2%).

5.3.3 Vecteur d'observation

Le vecteur d'observation $s_t \in [0, 1]^{147}$ encode l'état complet du système (Tableau 5.3). Par rapport à la formulation initiale, deux blocs de features géométriques ont été ajoutés : la distance du camion courant vers chaque pelle (3 valeurs) et la distance de chaque pelle vers chaque dump (6 valeurs), permettant à l'agent de disposer de la même information de proximité que les heuristiques Nearest Shovel et Shortest Path.

TABLE 5.3 – Composition du vecteur d'observation (configuration nominale)

Composante	**Dimension**	**Description**
Attente pelles (q_p)	3	Temps normalisé avant disponibilité
Attente dumps (z_r)	2	Temps normalisé avant disponibilité
Statut camions	12	Normalisé dans $[0, 1]$
Disponibilité camions	12	Temps normalisé avant disponibilité
Tonnage camions	12	Normalisé par 2 000 t
Carburant camions	12	Normalisé par 500 L
Position camions (one-hot)	84	12 camions $\times$ 7 nœuds
Distance camion courant $\rightarrow$ pelles	3	Distance Dijkstra normalisée par 10 km
Distance pelles $\rightarrow$ dumps	6	3×2 paires, normalisées par 10 km
Temps courant	1	Progression normalisée de l'épisode
Total	**147**	

5.3.4 Fonction de récompense implémentée

La fonction de récompense finale, issue du processus de calibration expérimental (Section 5.4), étend la formulation théorique (Eq. 3.7) par un terme de pénalité d'attente directe :

$$R_t = w_1 \cdot \frac{\Delta\text{tonnage}}{140} + w_2 \cdot \frac{-\text{Var}(\{q_p\})}{100} + w_3 \cdot \frac{-d_{\text{trajet}}}{5} + w_4 \cdot \left(-\min\left(\frac{q_{p_i}}{30}, 1\right)\right) \tag{5.1}$$

avec $w_1 = 1{,}0$, $w_2 = 0{,}1$, $w_3 = 0{,}05$, $w_4 = 0{,}3$. Ces valeurs reflètent la priorité accordée à la productivité sur l'équité et sur la réduction des coûts de trajet : une valeur $w_1 = 1{,}0$ largement supérieure aux autres garantit que chaque tonne livrée domine les signaux de pénalité. Le terme w_4 pénalise l'attente opérationnelle directe, créant un signal différencié entre les actions.

La valeur $w_4 = 0{,}3$ résulte d'une recalibration itérative : une valeur initiale de $w_4 = 1{,}0$ rendait l'action ATTENDRE compétitive vis-à-vis d'un cycle productif en contexte de congestion (récompense de cycle ≈ 0 contre pénalité d'attente $= -0{,}167$), conduisant les agents à apprendre systématiquement à ne rien faire. Avec $w_4 = 0{,}3$, un cycle productif vaut structurellement $+0{,}7$ même avec attente maximale, garantissant que travailler surpasse toujours attendre.

Le *principe* d'ajouter $w_4 > 0$ a été validé par un diagnostic empirique : sans ce terme, la différence de récompense entre actions était inférieure à 6×10^{-4}, rendant tout apprentissage tabulaire impossible (Tableau 5.4).

TABLE 5.4 – Diagnostic du signal de récompense avant et après ajout de w_4

Configuration	**Moy. action 0**	**Moy. action 1**	**Différence**
Sans w_4 (originale)	0.939	0.939	6×10^{-4}
Avec $w_4 = 1{,}0$	0.755	0.787	$3{,}2 \times 10^{-2}$

La figure 5.2 présente le graphe routier effectivement implémenté dans le simulateur pour la configuration nominale (7 nœuds : yard, jonction, 3 pelles, 2 dumps). Elle permet de visualiser la topologie sur laquelle circulent les camions et de vérifier que la structure du réseau correspond bien aux hypothèses de distances et de pentes décrites ci-dessus.

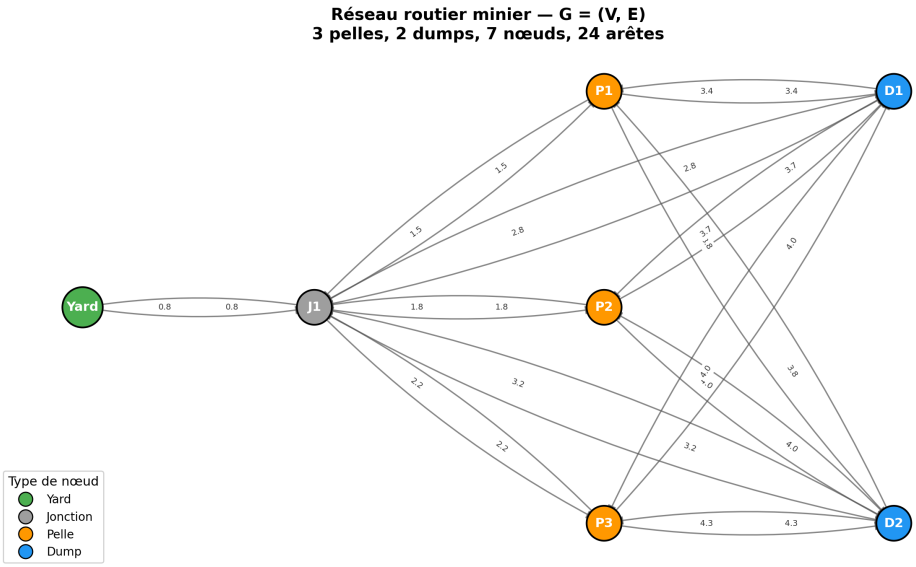


FIGURE 5.2 – Graphe routier du simulateur minier.

5.3.5 Validation du simulateur

Une fois ces éléments implémentés, la cohérence du simulateur a été vérifiée par la commande `gymnasium.utils.check_env()`, qui valide la conformité des espaces d'observation et d'action, la reproductibilité avec graine fixe et l'absence d'erreurs dans la boucle `reset`/`step`. Cette validation préalable conditionne la fiabilité des protocoles d'entraînement présentés ci-après.

5.4 Paramètres et protocoles d'entraînement

Un protocole d'entraînement rigoureux est indispensable pour garantir la reproductibilité des résultats et la comparabilité entre algorithmes. Les conditions communes à tous les agents — graine fixe, nombre de réplications, critères de convergence — ainsi que les paramètres spécifiques aux méthodes tabulaires et aux agents Deep RL sont présentés ci-après.

5.4.1 Protocole général

Tous les agents sont entraînés avec une graine fixe seed = 42 sur le scénario cible, puis évalués sur 10 réplications indépendantes (seeds 42 à 51) pour estimer la moyenne et l'écart-type des KPIs. Les modèles sont sauvegardés après convergence et réutilisés pour l'évaluation sans réentraînement.

5.4.2 Paramètres des méthodes tabulaires

Pour Q-Learning et SARSA, la discrétisation de l'espace d'état continu est réalisée par extraction de 6 features décisionnelles essentielles (attentes des 3 pelles et 2 dumps, et indice de la pelle la plus proche du camion courant), renormalisées par un facteur 50 pour étaler les valeurs sur $[0, 1]$, puis discrétisées en $n_{\text{bins}} = 8$ intervalles par dimension (la dimension «

pelle la plus proche » ne prenant que 3 valeurs possibles). L'espace d'état théorique couvre ainsi $8^5 \times 3 = 98\,304$ états, dont 17 034 effectivement visités en 30 000 épisodes.

Le tableau 5.5 regroupe les paramètres d'entraînement retenus pour les méthodes tabulaires (Q-Learning et SARSA). Leur alignement volontaire — mêmes α, γ, nombre d'épisodes et schéma de discrétisation — garantit que toute différence de performance observée par la suite proviendra de la règle de mise à jour (off-policy vs on-policy) et non d'un réglage inégal.

TABLE 5.5 – Paramètres d'entraînement des méthodes tabulaires

Paramètre	**Q-Learning**	**SARSA**
Épisodes	30 000	30 000
Taux d'apprentissage (α)	0.2	0.2
Facteur d'actualisation (γ)	0.99	0.99
Bins de discrétisation (n_{bins})	8	8
Features d'état	6	6
ϵ initial / final	1.0 / 0.01	1.0 / 0.01

5.4.3 Paramètres des agents Deep RL

DQN et PPO sont entraînés sur 2 000 000 steps ($\approx$ 9 677 épisodes) via Stable-Baselines3 avec un réseau MLP 128×128 et activation ReLU.

Le tableau 5.6 précise les paramètres d'entraînement des deux agents Deep RL. À nouveau, les valeurs communes (steps, learning rate, γ, architecture) assurent une comparaison équitable, les seules différences (buffer/*n_steps*, exploration, GAE, clipping) reflétant la nature respective off-policy de DQN et on-policy de PPO.

TABLE 5.6 – Paramètres d'entraînement des agents Deep RL

Paramètre	**DQN**	**PPO**
Steps totaux	2 000 000	2 000 000
Learning rate (α)	3×10^{-4}	3×10^{-4}
Batch size	64	64
Buffer / *n_steps*	200 000	1 024
Gamma (γ)	0.99	0.99
Exploration / Entropie	$\epsilon : 1.0 \rightarrow 0.02$	coef. 0.02
Architecture	MLP [128, 128]	MLP [128, 128]
GAE Lambda	—	0.95
Clip range	—	0.2

5.4.4 Efficacité algorithmique

Sur le plan computationnel, l'entraînement de chaque agent a été réalisé sur un ordinateur personnel (CPU). Les méthodes tabulaires (Q-Learning, SARSA) requièrent environ 45 à 60 minutes par scénario (30 000 épisodes). Les agents Deep RL (DQN, PPO) nécessitent 2 à 4

heures par scénario (2 000 000 steps), soit un budget d'entraînement total d'environ 24 heures pour l'ensemble des 12 configurations (4 algorithmes × 3 scénarios). Une fois entraînés, les modèles sont sauvegardés et l'inférence (décision par cycle) est instantanée, ce qui rend le déploiement en temps réel techniquement faisable.

Le Tableau 5.7 compare les quatre algorithmes sur le scénario nominal selon trois axes : le temps d'entraînement, l'empreinte mémoire du modèle sauvegardé et la stabilité des performances en production (écart-type de la productivité, repris du Tableau 6.1). Les méthodes Deep RL produisent des modèles nettement plus compacts (0,6 à 0,9 Mo contre plus de 3 Mo pour les tables Q tabulaires), tandis que PPO se distingue par la meilleure stabilité ($\sigma = 49$ t/h).

TABLE 5.7 – Comparaison de l'efficacité algorithmique — scénario nominal

Algorithme	**Type**	**Temps d'entraînement**	**Taille modèle**	**Stabilité (σ, t/h)**
Q-Learning	Tabulaire	≈ 57 min (30 000 ép.)	3,17 Mo	85
SARSA	Tabulaire	≈ 57 min (30 000 ép.)	3,21 Mo	64
DQN	Deep RL	≈ 2h13 (2M steps)	0,61 Mo	66
PPO	Deep RL	≈ 2h14 (2M steps)	0,90 Mo	49

5.5 Scénarios expérimentaux

Trois scénarios sont retenus pour couvrir les conditions opérationnelles représentatives d'une mine à ciel ouvert. Chaque scénario est évalué sur 10 réplications indépendantes.

Le tableau 5.8 définit les trois scénarios expérimentaux qui structurent l'ensemble de l'évaluation. Leur conception est progressive : le scénario nominal sert de référence, high_load stresse l'équilibrage de charge, et high_breakdown — directement lié à l'hypothèse H1 — éprouve la robustesse des politiques face aux pannes fréquentes.

TABLE 5.8 – Paramètres des scénarios expérimentaux

Scénario	**Camions**	**Pelles**	**Dumps**	p_b	**Description**
Nominal	12	3	2	2%	Conditions stables
High Load	18	3	2	2%	Surcharge (6 camions/pelle)
High Breakdown	12	3	2	10%	Pannes fréquentes

Le scénario **nominal** sert de référence pour calibrer les agents et valider la cohérence du simulateur. Le scénario **high_load** teste la capacité des algorithmes à équilibrer la charge entre pelles lorsque le nombre de camions dépasse le Match Factor optimal. Le scénario **high_breakdown** évalue la robustesse des politiques face aux perturbations stochastiques fréquentes, directement lié à l'hypothèse centrale H1.

5.6 Métriques d'évaluation

Les KPIs calculés à l'issue de chaque épisode sont les suivants :

— **Productivité horaire** (P_h, t/h) : tonnage total livré divisé par la durée du poste (8 heures). C'est l'indicateur principal de performance opérationnelle.

— **Temps d'attente moyen par camion** (E_t, min) : total des attentes en file divisé par le nombre de camions. Reflète directement la fluidité du dispatching et les coûts d'immobilisation.

— **Consommation spécifique** (E_e, L/t) : carburant total consommé divisé par le tonnage transporté. Indicateur d'efficacité énergétique et environnementale.

— **Coût moyen par cycle** (L/cycle) : carburant total divisé par le nombre de cycles complétés.

— **Taux d'utilisation** (%) : rapport du temps productif des camions sur le temps total disponible de la flotte.

Les résultats sont présentés sous forme de moyenne ± écart-type sur les 10 réplications, permettant d'apprécier la stabilité de chaque politique. Un écart-type élevé pour un agent Deep RL signale une convergence incomplète.

5.6.1 Métriques de diagnostic de l'apprentissage

Les KPIs précédents évaluent la *politique finale* une fois l'entraînement terminé. Une seconde catégorie de métriques, propre au Reinforcement Learning, diagnostique le *processus d'apprentissage* lui-même. Les métriques de classification (précision, rappel, F-score, matrice de confusion), pertinentes pour des décisions binaires correcte/incorrecte, ne s'appliquent pas à cet espace d'action (sept affectations pelle→dump et l'action ATTENDRE). Les métriques retenues sont :

— **Récompense cumulée / moyenne** (*Cumulative / Average Reward*) : récompense totale par épisode et sa moyenne mobile — indicateur principal de progression de l'apprentissage.

— **Convergence et courbe d'apprentissage** (*Learning Curve*) : évolution de la récompense (Q-Learning, SARSA) ou de la productivité lissée (DQN, PPO) jusqu'à stabilisation.

— **Exploration vs exploitation** : décroissance du taux ϵ (ϵ-greedy, Section 4.6.4).

— **Erreur de différence temporelle** (*TD error*, δ_t, Eq. 4.4) : écart entre valeur estimée et valeur cible à chaque mise à jour, pour Q-Learning et SARSA.

— **Policy Loss / Value Loss** : pour PPO, pertes de la politique et de la fonction de valeur, ainsi que l'*explained variance* du critique ; pour DQN, perte du réseau Q.

— **Sample Efficiency** : pourcentage du budget d'entraînement requis pour atteindre 95% de la performance finale (Tableau 6.6).

Les critères *Success Rate* et *Episode Length* ne s'appliquent pas : le MDP est à horizon fixe ($T = 480$ min) sans état terminal, la productivité cumulée par épisode en tenant lieu. Ces métriques sont illustrées par les figures du Chapitre 6.

5.7 Conclusion

Ce chapitre a présenté l'architecture logicielle, la modélisation de l'environnement, le protocole d'entraînement et les scénarios expérimentaux retenus. La conception modulaire et la validation du simulateur constituent la base nécessaire à l'analyse comparative qui suit dans le chapitre des résultats.

Chapitre 6

Résultats et analyse

Introduction

L'analyse des résultats est essentielle pour apprécier la validité de la solution proposée et mesurer sa capacité à répondre aux objectifs du mémoire. Au-delà de la performance brute, cette étape vise à évaluer la robustesse de l'approche, sa sensibilité aux perturbations et ses limites dans des contextes opérationnels variés.

6.1 Résultats expérimentaux

Le benchmark comparatif a été conduit sur trois scénarios représentatifs d'une mine à ciel ouvert. Les performances des huit politiques évaluées sont rapportées sous forme de moyennes et d'écarts-types calculés sur dix réplications indépendantes, selon les métriques définies en Section 5.6.

6.1.1 Scénario nominal

Le Tableau 6.1 présente les performances moyennes sur 10 réplications pour le scénario de référence (12 camions, 3 pelles, 2 dumps, $p_b = 2\%$).

Table 6.1 – Résultats comparatifs — Scénario nominal

Algorithme	**Prod. (t/h)**	**Wait (min)**	**Fuel (L/t)**	**Util. (%)**
PPO	**4208.8 ± 49**	31.8 ± 3.3	0.0412 ± 0.0005	94.6
DQN	4168.5 ± 66	40.2 ± 2.7	0.0405 ± 0.0006	92.3
Fixed Assignment	4074.0 ± 40	**27.9 ± 3.7**	0.0431 ± 0.0003	96.5
Shortest Path	3872.8 ± 32	74.2 ± 7.2	0.0431 ± 0.0004	87.2
Nearest Shovel	3830.8 ± 55	72.6 ± 7.4	0.0429 ± 0.0003	87.8
Q-Learning	3591.0 ± 85	56.3 ± 5.2	0.0481 ± 0.0012	91.1
FIFO	3290.0 ± 42	33.2 ± 6.2	0.0532 ± 0.0005	96.2
SARSA	3081.8 ± 64	66.8 ± 5.3	0.0556 ± 0.0007	89.1

6.1.2 Scénario high_load

Le tableau 6.2 rapporte les performances en situation de surcharge (18 camions pour 3 pelles). Sa lecture révèle deux phénomènes majeurs : la domination de Fixed Assignment, dont le Match Factor reste optimal, et surtout l'effondrement des heuristiques myopes (Nearest Shovel, Shortest Path), dont le temps d'attente dépasse 200 minutes — un point central pour la validation de la problématique.

TABLE 6.2 – Résultats comparatifs — Scénario high_load (18 camions)

Algorithme	**Prod. (t/h)**	**Wait (min)**	**Fuel (L/t)**	**Util. (%)**
Fixed Assignment	**5874.8 ± 53**	**43.0 ± 3.8**	0.0444 ± 0.0003	94.0
DQN	5664.8 ± 65	46.8 ± 2.6	0.0444 ± 0.0004	90.6
PPO	5244.8 ± 350	92.8 ± 21.6	0.0466 ± 0.0018	80.8
Q-Learning	4795.0 ± 106	69.1 ± 4.1	0.0537 ± 0.0009	88.9
FIFO	4747.8 ± 70	44.2 ± 3.4	0.0546 ± 0.0006	93.9
SARSA	4042.5 ± 67	119.8 ± 12.1	0.0613 ± 0.0011	78.6
Shortest Path	3956.8 ± 40	204.7 ± 6.9	0.0588 ± 0.0006	61.1
Nearest Shovel	3939.3 ± 42	200.2 ± 10.4	0.0580 ± 0.0007	62.0

6.1.3 Scénario high_breakdown

Le tableau 6.3 présente les résultats sous pannes fréquentes ($p_b = 10\%$), scénario déterminant pour l'hypothèse H1. On y observe que DQN et PPO devancent désormais Fixed Assignment en productivité, confirmant l'avantage des agents Deep RL lorsque l'environnement devient instable et que l'adaptation dynamique prend toute sa valeur.

TABLE 6.3 – Résultats comparatifs — Scénario high_breakdown ($p_b = 10\%$)

Algorithme	**Prod. (t/h)**	**Wait (min)**	**Fuel (L/t)**	**Util. (%)**
DQN	**3991.8 ± 56**	68.7 ± 5.5	0.0416 ± 0.0003	86.3
PPO	3946.3 ± 63	65.1 ± 7.0	0.0422 ± 0.0008	87.2
Fixed Assignment	3860.5 ± 38	**53.1 ± 6.0**	0.0451 ± 0.0004	91.7
Shortest Path	3702.3 ± 40	91.0 ± 5.5	0.0444 ± 0.0004	84.3
Nearest Shovel	3692.5 ± 42	90.1 ± 6.2	0.0441 ± 0.0004	84.4
Q-Learning	3176.3 ± 94	88.2 ± 7.1	0.0539 ± 0.0013	84.8
FIFO	3115.0 ± 74	53.0 ± 11.6	0.0556 ± 0.0009	92.2
SARSA	2973.3 ± 89	77.5 ± 8.8	0.0588 ± 0.0010	87.3

La figure 6.1 offre une lecture visuelle synthétique des productivités des huit politiques sur les trois scénarios. Elle permet de saisir d'un coup d'œil les renversements de hiérarchie d'un scénario à l'autre — notamment le passage de Fixed Assignment en tête (high_load) à DQN/PPO en tête (nominal, high_breakdown).

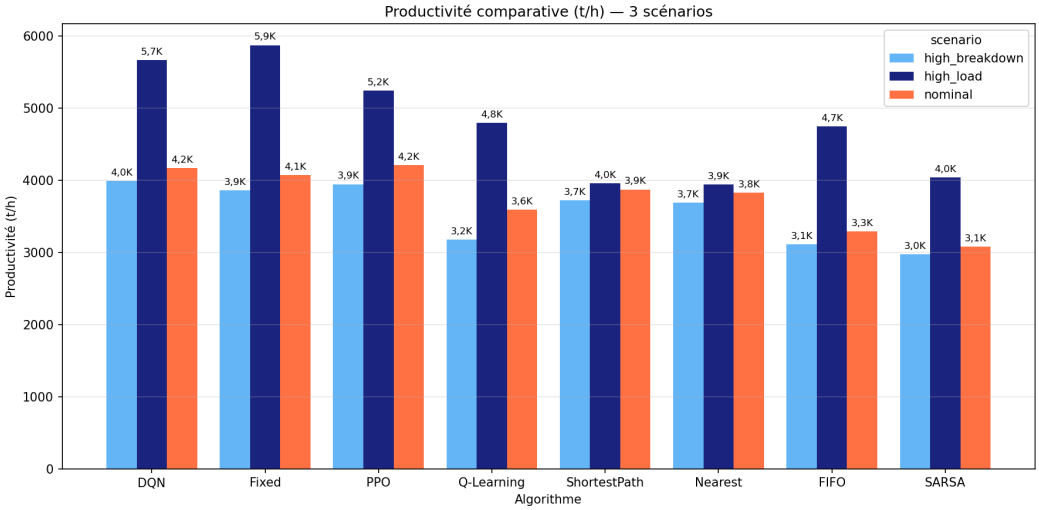


FIGURE 6.1 – Productivité comparative des algorithmes sur les trois scénarios.

6.2 Comparaison avec les approches de référence

Au-delà des résultats par scénario, une lecture transversale des performances est proposée, mettant en regard l'approche Deep RL avec les heuristiques industrielles et les méthodes classiques. L'objectif est d'identifier les conditions dans lesquelles l'apprentissage par renforcement apporte une valeur ajoutée réelle et de quantifier les écarts de performance.

6.2.1 Analyse comparative globale

Le Tableau 6.4 présente une synthèse transversale des performances sur les trois scénarios.

TABLE 6.4 – Synthèse comparative multi-scénarios (productivité t/h / attente min)

Algorithme	Nominal	High Load	High Breakdown	Rang wait moyen
PPO	**4209** / 31.8	5245 / 92.8	**3946** / 65.1	**1er prod. (2/3)**
DQN	4169 / 40.2	**5665** / **46.8**	3992 / 68.7	**1er prod. (1/3)**
Fixed	4074 / **27.9**	**5875** / **43.0**	3861 / **53.1**	1er wait (2/3)
Q-Learn.	3591 / 56.3	4795 / 69.1	3176 / 88.2	4ème
FIFO	3290 / 33.2	4748 / 44.2	3115 / 53.0	5ème
SARSA	3082 / 66.8	4043 / 119.8	2973 / 77.5	6ème
Nearest	3831 / 72.6	3939 / 200.2	3693 / 90.1	dernier

6.2.2 Analyse par scénario

Une analyse statistique de significativité a été effectuée sur les dix réplications indépendantes par politique, afin de déterminer si les écarts de productivité observés entre les méthodes étaient robustes et non dus au hasard. Les tests de Welch (test t pour échantillons indépendants) ont été appliqués pour comparer directement PPO et Fixed Assignment sur le scénario nominal, ainsi que DQN et Fixed Assignment sur le scénario high_breakdown.

Une ANOVA à un facteur a été conduite sur les huit méthodes pour chacun des trois scénarios, suivie dans chaque cas d'un test post-hoc de Tukey pour identifier les comparaisons significatives (Annexe D).

Le tableau 6.5 synthétise les tests statistiques conduits pour s'assurer que les écarts observés ne relèvent pas du hasard. Les très faibles p-values (toutes inférieures à 10^{-5}) confirment la significativité des différences, donnant une base statistique solide à la validation de l'hypothèse H1.

TABLE 6.5 – Résumé des tests statistiques appliqués aux trois scénarios.

Comparaison	**Statistique**	**p-value**	**Conclusion**
PPO vs Fixed Assignment (nominal, productivité)	$t = 6{,}356$	$3{,}0 \times 10^{-6}$	Différence significative, PPO supérieur à Fixed.
DQN vs Fixed Assignment (high_breakdown, productivité)	$t = 5{,}863$	$1{,}3 \times 10^{-5}$	Différence significative, DQN supérieur à Fixed.
ANOVA sur les 8 méthodes (nominal, productivité)	$F(7, 72) = 481{,}25$	$< 10^{-56}$	Différence globale significative.
ANOVA sur les 8 méthodes (high_load, productivité)	$F(7, 72) = 298{,}97$	$1{,}5 \times 10^{-50}$	Différence globale significative.
ANOVA sur les 8 méthodes (high_breakdown, productivité)	$F(7, 72) = 352{,}28$	$4{,}9 \times 10^{-53}$	Différence globale significative.

Scénario nominal. PPO (4 208.75 t/h) et DQN (4 168.50 t/h) surpassent tous deux Fixed Assignment (4 074.00 t/h) en productivité, démontrant que les agents Deep RL ont appris une politique d'affectation plus efficace que le schéma round-robin statique. Ce résultat valide directement l'hypothèse H1 en conditions stables.

Fixed Assignment conserve le meilleur temps d'attente (27.9 min) grâce au Match Factor parfait (12/3 = 4 camions/pelle), ce qui minimise structurellement les files d'attente. PPO présente le meilleur temps d'attente parmi les agents RL (31.8 min), confirmant qu'il a appris à équilibrer les files de manière adaptative.

L'analyse statistique confirme que l'écart de productivité observé entre PPO et Fixed Assignment en scénario nominal est significatif : test t de Welch unilatéral $t = 6{,}356$, $p = 3{,}0 \times 10^{-6}$. Une ANOVA sur les huit méthodes affiche $F(7, 72) = 481{,}25$, $p < 10^{-56}$, indiquant une différence globale significative. Le test post-hoc de Tukey montre que PPO est significativement supérieur à Fixed Assignment (+134,75 t/h), tandis que la performance de DQN n'est pas significativement différente de celle de PPO sur ce scénario ($p = 0{,}799$). L'ensemble des comparaisons par paires est détaillé en Annexe D.

Shortest Path (3 872.8 t/h) se distingue désormais de Nearest Shovel (3 830.8 t/h) grâce à l'implémentation du coût de cycle complet $C = \alpha T + \beta D + \gamma E$ (Eq. 3.1), validant l'apport de la formulation multi-critère.

Scénario high_load. Avec 18 camions pour 3 pelles, Fixed Assignment reste dominant (5 874.8 t/h), son Match Factor $18/3 = 6$ camions/pelle restant optimal. DQN (5 664.8 t/h, $-3{,}6\%$ vs Fixed) se positionne comme la meilleure alternative RL avec un écart modéré et une faible variance.

Si Fixed Assignment domine structurellement en high_load, les agents Deep RL conservent une avance décisive sur les autres heuristiques : DQN (5 664.8 t/h) surpasse FIFO (+917 t/h), SARSA (+1 622 t/h) et Nearest Shovel (+1 726 t/h), tous avec $p \approx 0$. PPO, malgré sa variance élevée, reste lui aussi significativement supérieur à Nearest Shovel (+1 306 t/h, $p = 3{,}5 \times 10^{-4}$) et à FIFO (+497 t/h, $p < 0{,}001$).

PPO affiche une performance variable (5 244.8±350 t/h) : certaines réplications atteignent 5 687 t/h (proches de Fixed) tandis que d'autres n'atteignent que 4 882 t/h. Cette instabilité en surcharge traduit la sensibilité de la politique on-policy aux configurations de départ.

Les heuristiques myopes (Nearest Shovel, Shortest Path) s'effondrent : temps d'attente ≥ 200 minutes, pires que toutes les autres politiques. Ce phénomène confirme la congestion systématique prévue au Chapitre 3 : en l'absence d'équilibrage de charge, la flotte sature la pelle géographiquement la plus proche.

Cette supériorité des agents RL face aux heuristiques myopes est statistiquement confirmée : le test t de Welch entre DQN et Nearest Shovel en high_load affiche $t = 65{,}33$, $p \approx 0$, un écart de +1 725,5 t/h. Même PPO, malgré sa variance élevée, surpasse significativement Nearest Shovel ($t = 11{,}69$, $p = 3{,}5 \times 10^{-4}$, +1 305,5 t/h).

Le test post-hoc de Tukey, appliqué à l'ANOVA sur les huit méthodes en high_load ($F(7, 72) = 298{,}97$, $p = 1{,}5 \times 10^{-50}$, Tableau 6.5), confirme par ailleurs que DQN reste significativement supérieur à PPO dans ce scénario (+420,0 t/h, $p < 0{,}001$) — à l'inverse de la situation nominale où les deux algorithmes sont statistiquement équivalents (Tableau D.1). L'ensemble des comparaisons par paires pour high_load est détaillé en Annexe D (Tableau D.2).

Scénario high_breakdown. C'est le scénario déterminant pour la validation de H1. DQN (3 991.75 t/h) et PPO (3 946.25 t/h) surpassent tous deux Fixed Assignment (3 860.50 t/h) en productivité, confirmant la supériorité des agents Deep RL face aux perturbations stochastiques fréquentes.

L'écart de productivité DQN − Fixed est lui aussi significatif : test t de Welch unilatéral $t = 5{,}863$, $p = 1{,}3 \times 10^{-5}$. Cette validation statistique renforce l'interprétation selon laquelle les gains observés ne résultent pas d'une simple variation aléatoire entre réplications.

Fixed Assignment conserve le meilleur temps d'attente (53.1 min) : son schéma cyclique génère peu d'attente grâce à la distribution régulière des camions. PPO (65.1 min) et DQN (68.7 min) ont un temps d'attente légèrement supérieur mais produisent davantage, car ils exploitent les pelles disponibles plus agressivement dès qu'une panne libère une ressource.

L'écart de productivité PPO − Fixed (+86 t/h, +2,2%) représente, extrapolé sur un poste de 8 heures, un gain de 688 tonnes supplémentaires transportées malgré un taux de pannes 5 fois plus élevé que le scénario nominal.

Le test post-hoc de Tukey, appliqué à l'ANOVA sur les huit méthodes en high_breakdown ($F(7, 72) = 352{,}28$, $p = 4{,}9 \times 10^{-53}$, Tableau 6.5), confirme la supériorité de DQN sur Fixed Assignment même après correction pour comparaisons multiples (+131,25 t/h, $p = 0{,}0014$).

L'écart PPO − Fixed, bien que numériquement positif comme indiqué ci-dessus, n'atteint en revanche pas le seuil de significativité une fois cette correction appliquée ($p = 0{,}112$). DQN et PPO restent par ailleurs statistiquement équivalents dans ce scénario ($p = 0{,}813$), comme en conditions nominales. Le détail des comparaisons par paires pour high_breakdown est fourni en Annexe D (Tableau D.3).

La figure 6.2 compare le temps d'attente moyen de Fixed Assignment, PPO et DQN sur les trois scénarios. Elle éclaire le compromis productivité–attente : les agents Deep RL acceptent une attente légèrement supérieure à celle de Fixed Assignment en échange d'un tonnage plus élevé, en exploitant plus agressivement les ressources disponibles.

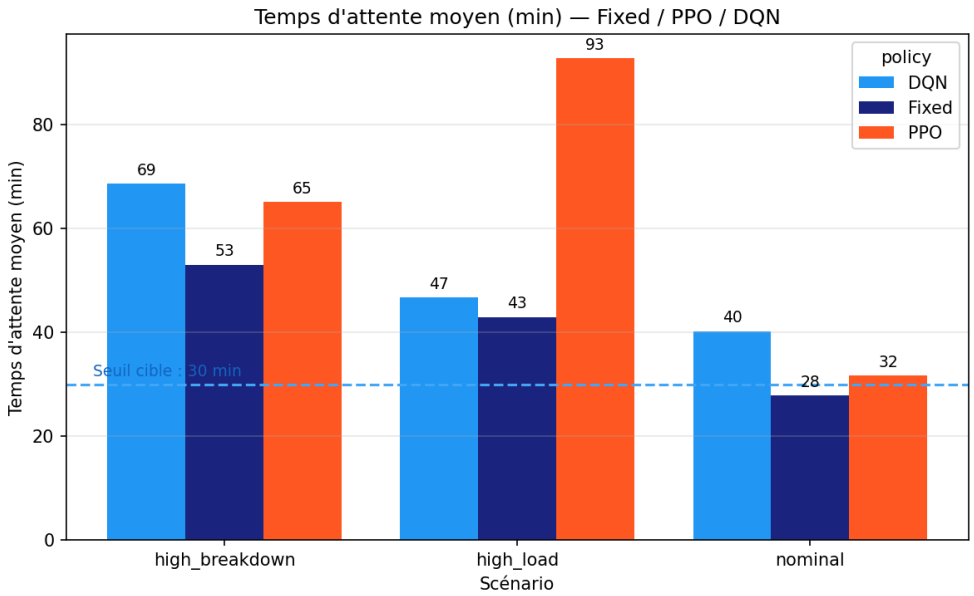


FIGURE 6.2 – Temps d'attente moyen (min) — Fixed Assignment, PPO et DQN sur les trois scénarios.

6.3 Analyse critique des résultats

Les résultats quantitatifs présentés en Section 6.1 appellent une lecture critique qui dépasse les simples comparaisons chiffrées. La convergence des agents, les mécanismes explicatifs des écarts de performance entre scénarios et les limites inhérentes à la démarche expérimentale sont examinés dans les sous-sections suivantes.

6.3.1 Convergence des agents d'apprentissage

La courbe d'apprentissage de Q-Learning montre une progression claire de la récompense cumulée, passant de 155 à 175 sur le scénario nominal (+12,5%) et de 213 à 225 sur le scénario high_load (+5,4%). La Q-table couvre 17 034 états effectifs sur 30 000 épisodes en nominal (espace théorique : $8^5 \times 3 = 98\,304$), confirmant que l'enrichissement du vecteur d'état (ajout de la pelle la plus proche du camion courant) a permis une couverture suffisante pour une politique compétitive.

SARSA converge vers une valeur inférieure (≈ 145 contre 175 pour Q-Learning en nominal), conformément au comportement théorique prédit : en tant que méthode on-policy, SARSA intègre le coût de l'exploration dans ses estimations, produisant une politique conservatrice mais sous-optimale par rapport à Q-Learning off-policy.

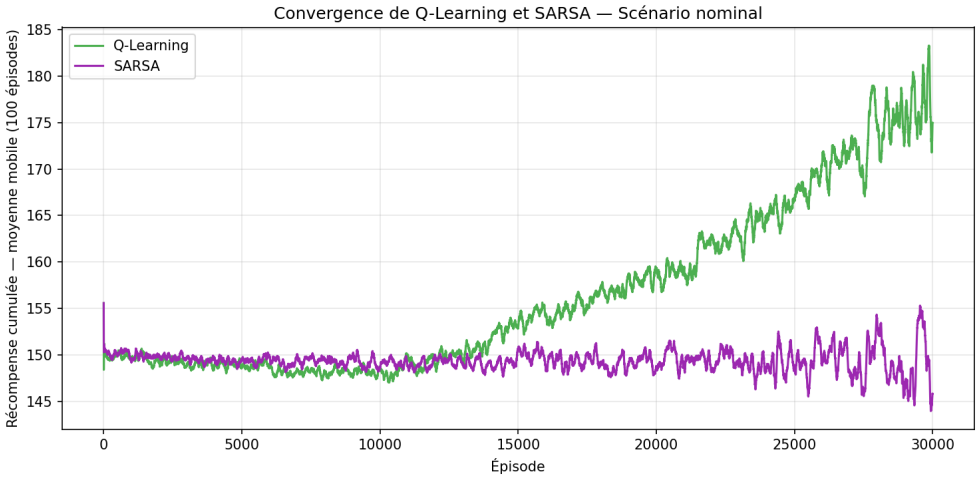


FIGURE 6.3 – Courbes de convergence de Q-Learning et SARSA — scénario nominal.

La figure 6.4 transpose cette analyse de convergence au scénario high_load. La même hiérarchie se retrouve, Q-Learning convergeant vers une récompense supérieure à celle de SARSA, à un niveau global plus élevé du fait de la flotte élargie.

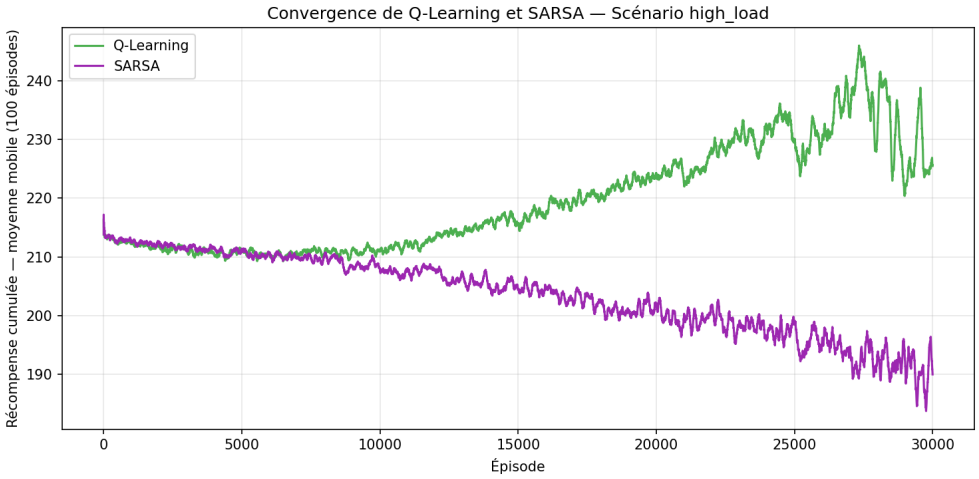


FIGURE 6.4 – Courbes de convergence de Q-Learning et SARSA — scénario high_load.

Sur le scénario high_breakdown, les deux algorithmes partent d'une récompense initiale comparable (≈ 147) mais divergent ensuite : Q-Learning progresse jusqu'à ≈ 158 (+7,3%), tandis que SARSA reste stable autour de ≈ 142, légèrement en dessous de son point de départ. Cette tendance confirme la robustesse relative de la politique off-policy de Q-Learning face aux pannes fréquentes, au prix d'une variance accrue par rapport au scénario nominal.

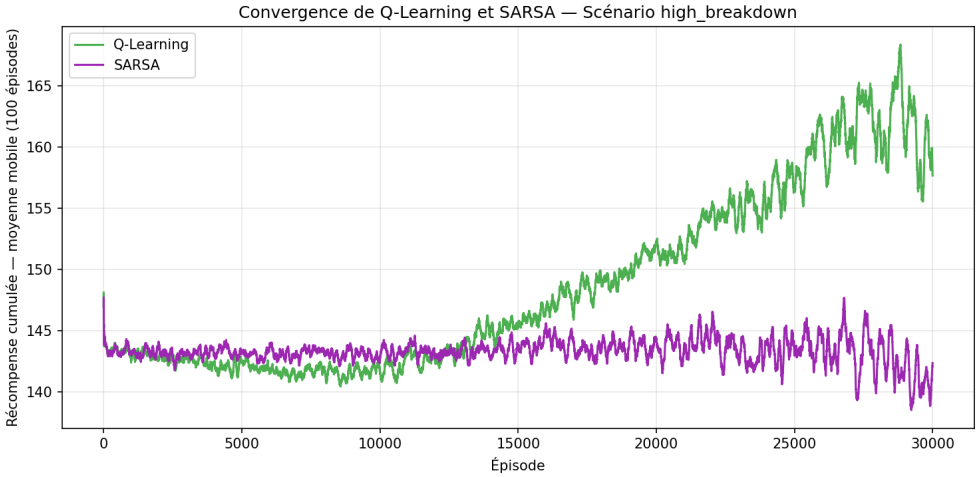


FIGURE 6.5 – Courbes de convergence de Q-Learning et SARSA — scénario high_breakdown.

Les agents Deep RL (DQN, PPO) présentent une dynamique de convergence comparable, mesurée ici par la productivité (t/h) lissée sur une fenêtre glissante au cours de l'entraînement (2M steps).

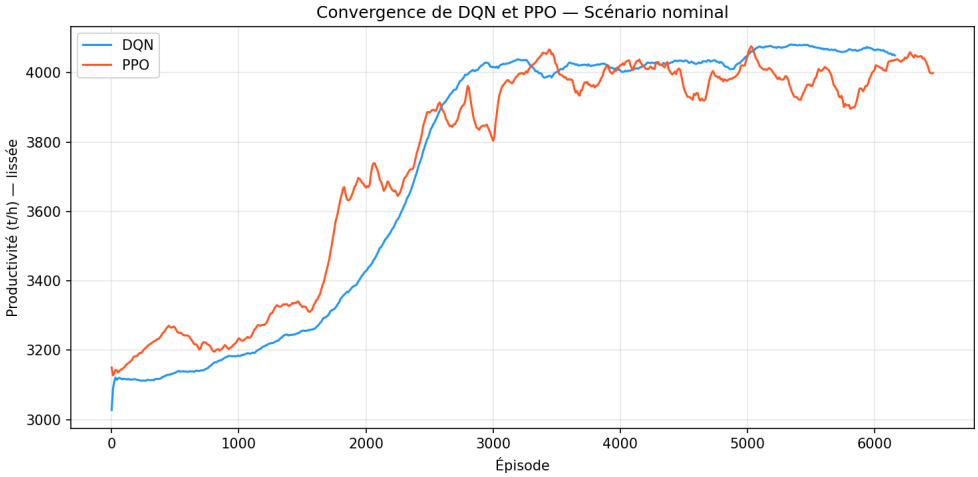


FIGURE 6.6 – Convergence de DQN et PPO — scénario nominal (productivité t/h lissée).

La figure 6.7 montre la convergence des agents Deep RL en surcharge. Les deux courbes restent croissantes, DQN conservant une progression légèrement supérieure à celle de PPO.

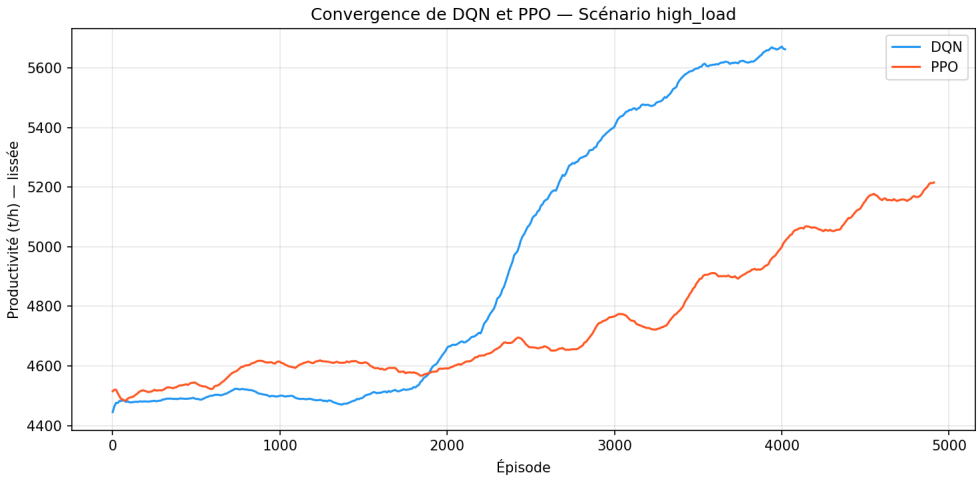


FIGURE 6.7 – Convergence de DQN et PPO — scénario high_load (productivité t/h lissée).

La figure 6.8 illustre la convergence sous pannes fréquentes. Malgré un signal plus bruité, la tendance de fond demeure croissante, confirmant la robustesse de l'apprentissage des deux agents face aux perturbations.

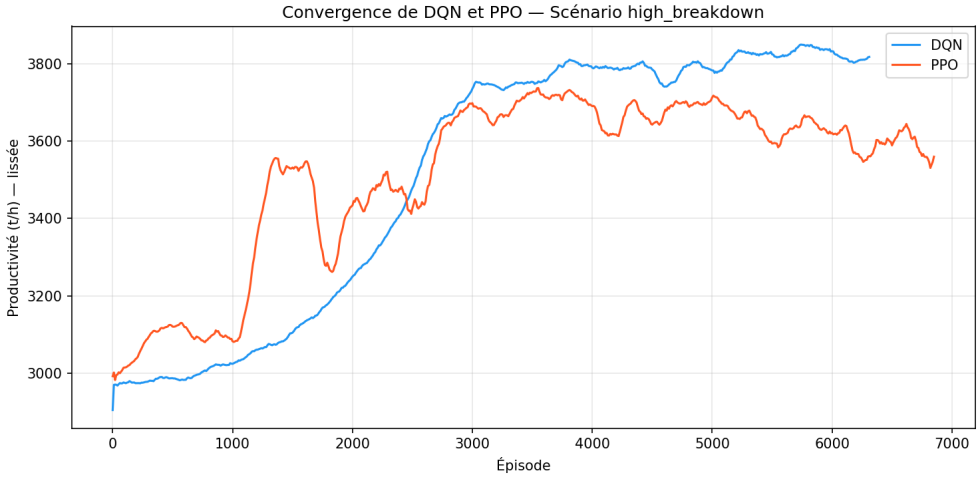


FIGURE 6.8 – Convergence de DQN et PPO — scénario high_breakdown (productivité t/h lissée).

Sur les trois scénarios, DQN affiche une progression relative légèrement supérieure à PPO, mais PPO conserve une variance d'évaluation plus faible une fois la convergence atteinte (cf. Tableau 5.7). La courbe du scénario high_breakdown illustre la robustesse des deux agents face aux pannes : malgré un signal plus bruité, la tendance de fond reste croissante sur l'ensemble de l'entraînement.

La convergence de PPO est confirmée par `explained_variance = 0.999`, indiquant que le réseau critique prédit avec précision les retours futurs. Cette valeur proche de 1.0 signifie que l'agent a pleinement exploité le signal de récompense disponible dans le budget d'entraînement alloué (2M steps $\approx$ 9 677 épisodes).

6.3.2 Diagnostics complémentaires de l'apprentissage

Les quatre métriques de diagnostic restant à illustrer parmi celles introduites en Section 5.6.1 – exploration vs exploitation, erreur TD, Policy/Value Loss et efficacité d'échantillonnage – sont présentées ci-après pour les trois scénarios (nominal, high_load, high_breakdown), en cohérence avec le Tableau 6.6 qui couvre déjà ces trois configurations.

Exploration vs exploitation. La Figure 6.9 illustre la transition exploration → exploitation. Pour Q-Learning et SARSA, le taux ϵ décroît linéairement de 1,0 à 0,01 sur les 30 000 épisodes selon le schéma ϵ-greedy défini en Section 4.6.4, identique pour les trois scénarios. Pour DQN, `rollout/exploration_rate` suit une décroissance comparable, de 0,999 à 0,020 sur les 2M steps. PPO ne dispose pas d'un paramètre ϵ explicite : l'entropie de sa politique (panneau droit) chute de 1,94 à 0,50 nat au cours de l'entraînement, traduisant une politique de plus en plus déterministe à mesure que l'agent affine ses choix d'affectation.

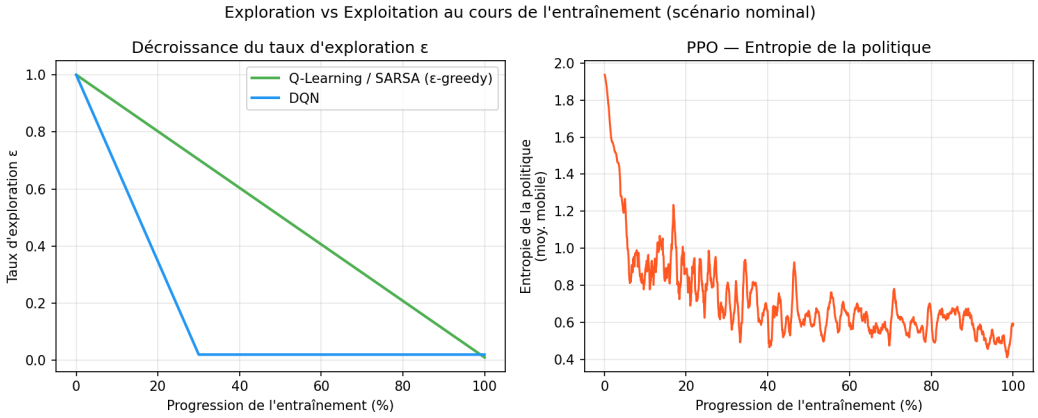


FIGURE 6.9 – Exploration vs exploitation — scénario nominal.

Les Figures 6.10 et 6.11 reprennent cette analyse pour les scénarios high_load et high_breakdown. Le panneau gauche – décroissance de ϵ pour Q-Learning/SARSA et de `rollout/exploration_rate` pour DQN – est, par construction, quasi identique d'un scénario à l'autre : ces schémas ne dépendent que du nombre d'épisodes (30 000) ou de steps (2M), fixé indépendamment de l'environnement simulé. Le panneau droit (entropie de la politique PPO) révèle en revanche une dynamique différenciée : l'entropie finale atteint 0,68 nat en high_load contre 0,50 en nominal et 0,55 en high_breakdown, signe d'une politique légèrement moins déterministe en situation de surcharge – cohérent avec la variance d'évaluation nettement plus élevée de PPO sur ce scénario ($5\,244.8 \pm 350$ t/h, Tableau 6.2).

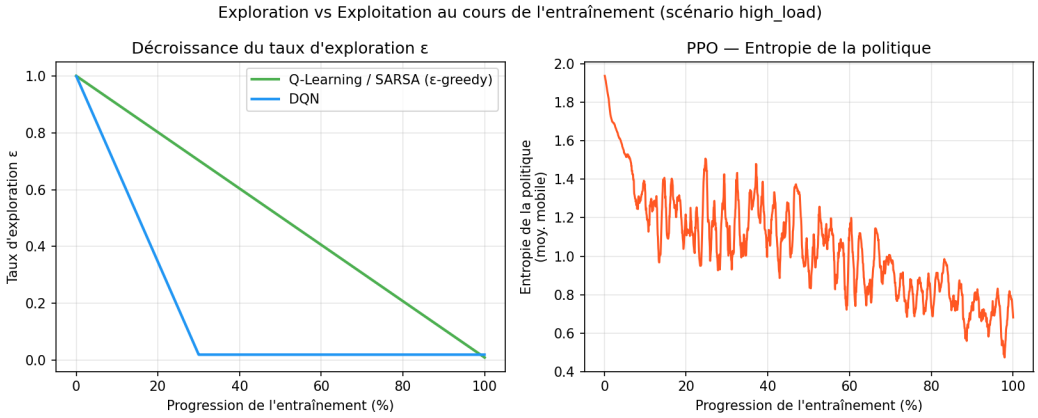


FIGURE 6.10 – Exploration vs exploitation — scénario high_load.

La figure 6.11 présente la même analyse exploration–exploitation pour le scénario high_breakdown, à mettre en regard de la figure précédente.

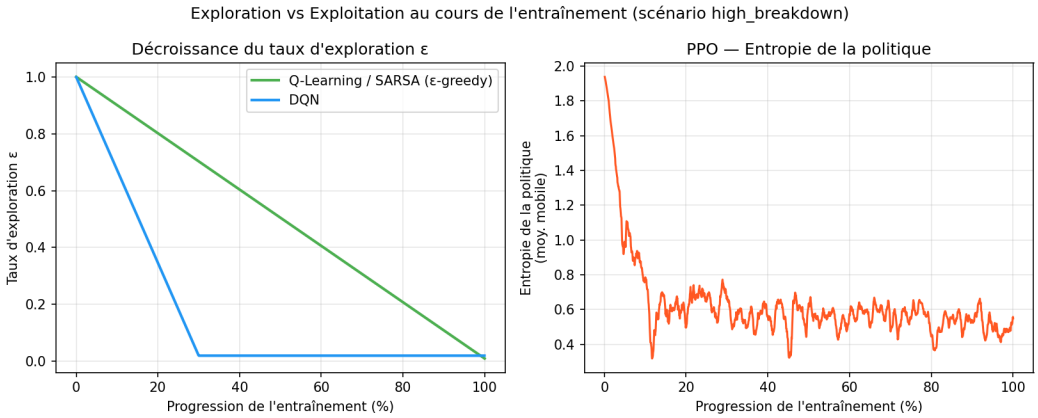


FIGURE 6.11 – Exploration vs exploitation — scénario high_breakdown.

Erreur TD. La Figure 6.12 présente l'évolution de l'erreur TD moyenne $|\delta_t|$ par épisode (Eq. 4.4) pour Q-Learning et SARSA. Les deux courbes suivent un profil en cloche : $|\delta_t|$ croît durant les premiers milliers d'épisodes – la table Q passe de 179 entrées à plus de 13 500 durant les 5 000 premiers épisodes, de sorte que de nombreuses mises à jour portent sur des couples état-action récemment découverts et encore initialisés à zéro – puis décroît progressivement à mesure que la table se stabilise (17 034 entrées pour Q-Learning, 17 240 pour SARSA après 30 000 épisodes) et que ϵ diminue. Q-Learning termine sur une erreur TD nettement plus faible (0,19 contre 0,85 pour SARSA), cohérent avec sa convergence vers une récompense plus élevée (175 contre 145) : ses cibles off-policy ($\max_{a'} Q(s', a')$) se stabilisent davantage que les cibles on-policy de SARSA, encore couplées au comportement résiduel de la politique ϵ-greedy ($\epsilon = 0{,}01$). Cette décroissance se poursuit au-delà du point où la récompense moyenne semble visuellement stabilisée (Figure 6.3), confirmant que les valeurs Q continuent de s'affiner après que la politique s'est quasiment figée.

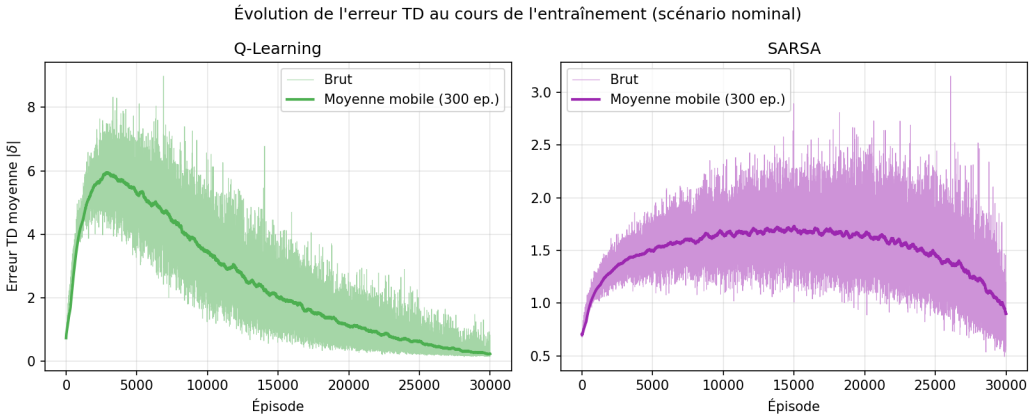


FIGURE 6.12 – Évolution de l'erreur TD moyenne par épisode — Q-Learning et SARSA, scénario nominal.

Les Figures 6.13 et 6.14 montrent le même profil en cloche pour les scénarios high_load et high_breakdown, avec des valeurs finales d'erreur TD cohérentes avec les écarts de convergence observés en Section 6.3 : en high_load, Q-Learning termine à 0,36 contre 0,74 pour SARSA, écart à mettre en regard de leurs récompenses finales respectives (225 contre 190, Figure 6.4) ; en high_breakdown, l'écart est encore plus marqué (0,25 contre 0,85), reflet de la stagnation de SARSA autour de sa récompense initiale (≈ 142) alors que Q-Learning continue de progresser (≈ 158). Dans les trois scénarios, la table Q converge vers une taille comparable (16 600 à 17 500 entrées sur un espace théorique de 98 304), confirmant que la couverture de l'espace d'états est robuste aux variations de charge et de fiabilité des équipements.

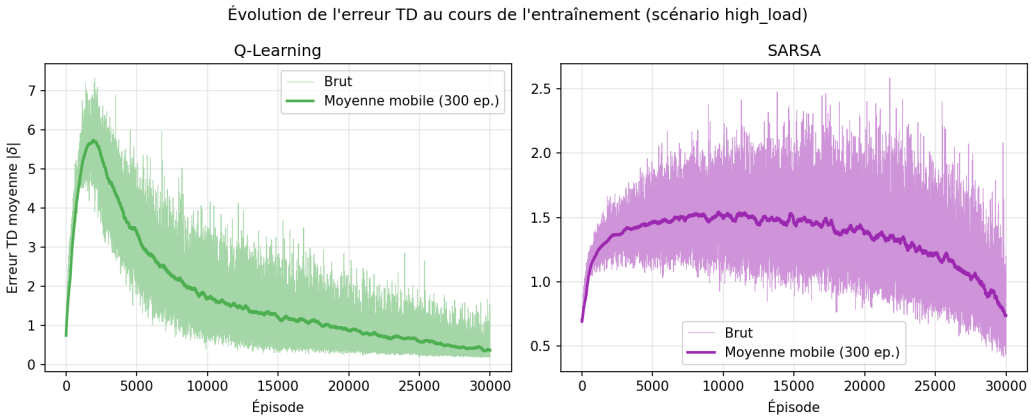


FIGURE 6.13 – Évolution de l'erreur TD moyenne par épisode — Q-Learning et SARSA, scénario high_load.

La figure 6.14 complète l'analyse de l'erreur TD pour le scénario high_breakdown, où l'écart final entre Q-Learning et SARSA est le plus marqué.

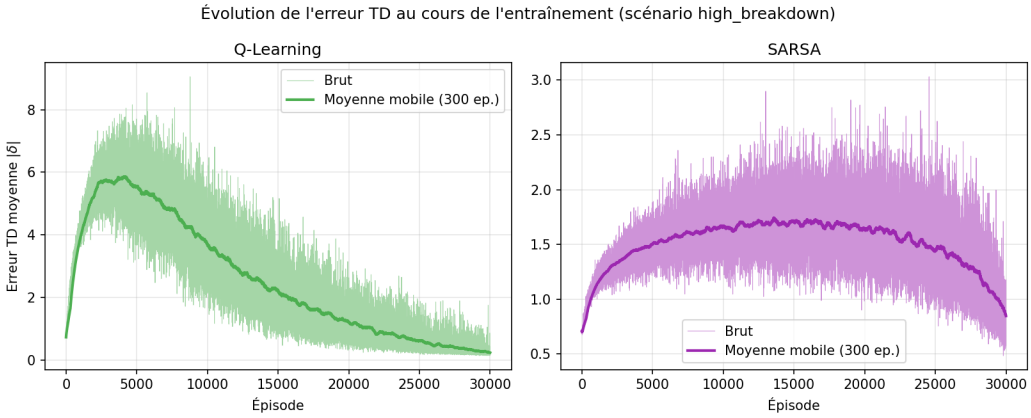


FIGURE 6.14 – Évolution de l'erreur TD moyenne par épisode — Q-Learning et SARSA, scénario high_breakdown.

Policy Loss et Value Loss. Pour DQN, la perte du réseau Q (`train/loss`, Figure 6.15) diminue globalement au cours de l'entraînement, de 0,040 en moyenne sur le premier dixième des steps à 0,025 sur le dernier dixième, malgré une forte variance résiduelle (pic à 0,30) caractéristique de l'apprentissage hors-politique avec experience replay. Pour PPO (Figure 6.16), la *policy gradient loss* reste faible et stable tout au long de l'entraînement (de −0,025 à −0,014 en moyenne mobile), signe que le *clipping* de l'objectif PPO empêche les mises à jour de politique excessives. La *value loss* décroît fortement, de 32,0 à 0,17 (échelle logarithmique sur la figure), cohérent avec l'*explained variance* de 0,999 mentionnée ci-dessus.

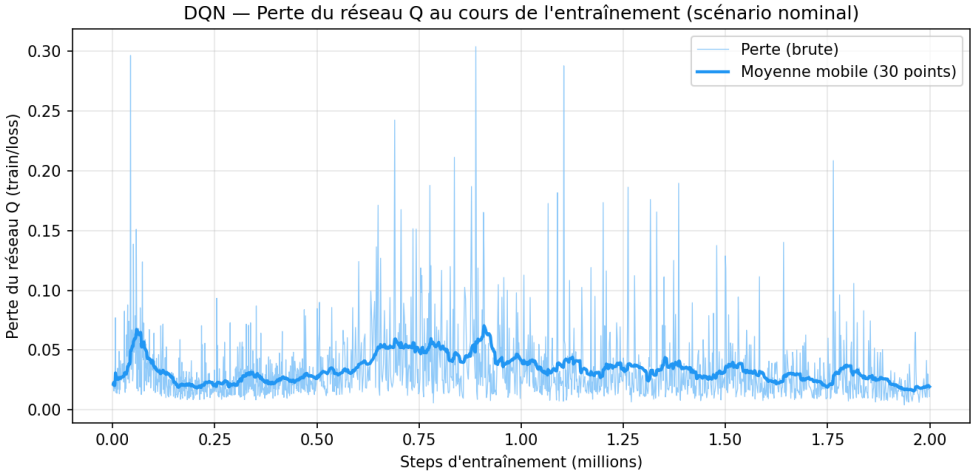


FIGURE 6.15 – DQN — perte du réseau Q (`train/loss`) au cours de l'entraînement, scénario nominal.

La figure 6.16 présente, en complément, les pertes de PPO (*policy loss* et *value loss*) sur le scénario nominal.

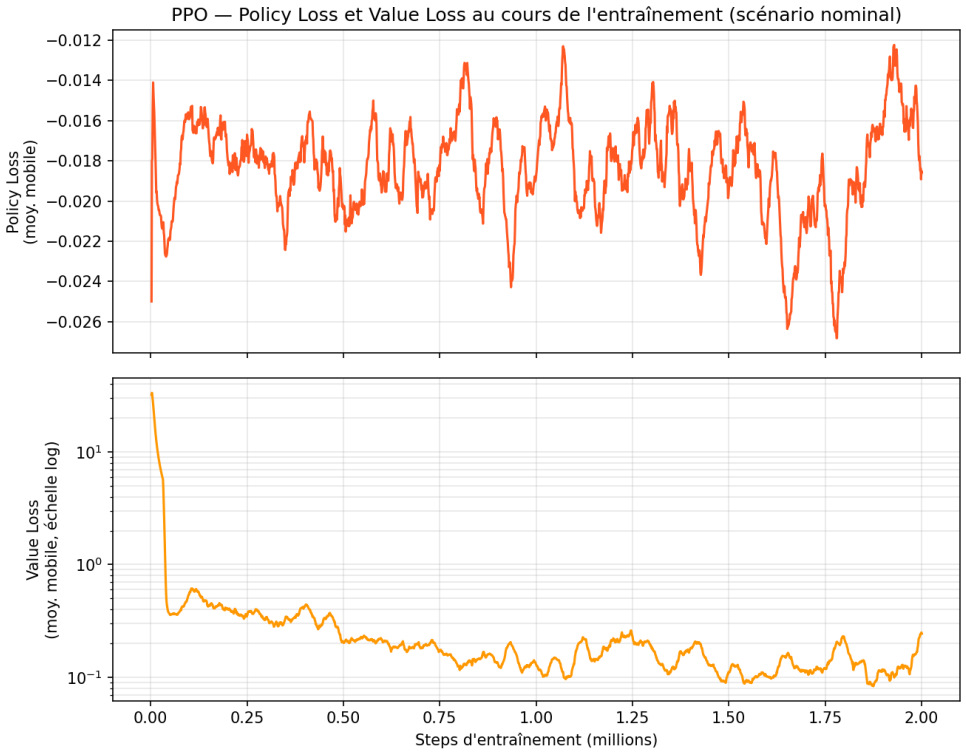


FIGURE 6.16 – PPO — Policy Loss et Value Loss (échelle log) au cours de l'entraînement, scénario nominal.

Les Figures 6.17, 6.18, 6.19 et 6.20 reproduisent cette analyse pour les scénarios high_load et high_breakdown. Pour DQN, la perte du réseau Q suit la même tendance décroissante que sur le scénario nominal : de 0,048 à 0,034 en moyenne (premier vs dernier dixième des steps) en high_load, et de 0,033 à 0,022 en high_breakdown – des niveaux globalement inférieurs au scénario nominal (0,040 → 0,025), avec un pic résiduel plus élevé en high_load (0,50 contre 0,30 en nominal), cohérent avec la complexité accrue de l'environnement à 18 camions. Pour PPO, la *policy gradient loss* reste faible et stable dans les deux scénarios (de −0,021 à −0,017 en moyenne mobile, dans les deux cas), tandis que la *value loss* décroît fortement (de 29,3 à 0,11 en high_load, de 32,0 à 0,12 en high_breakdown), confirmée par une *explained variance* finale proche de 1,0 dans les deux cas (0,9999 et 0,9998 respectivement) – comme pour le scénario nominal.

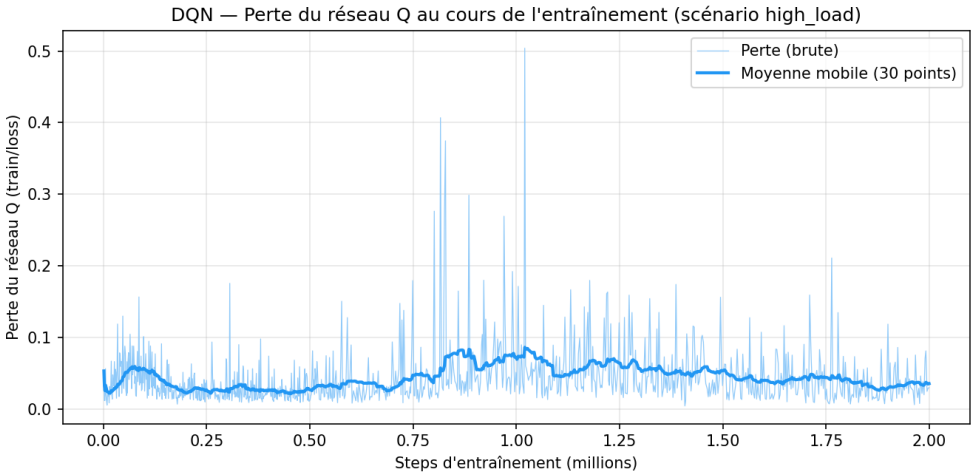


FIGURE 6.17 – DQN — perte du réseau Q (`train/loss`) au cours de l'entraînement, scénario high_load.

La figure 6.18 montre les pertes de PPO en surcharge (high_load).

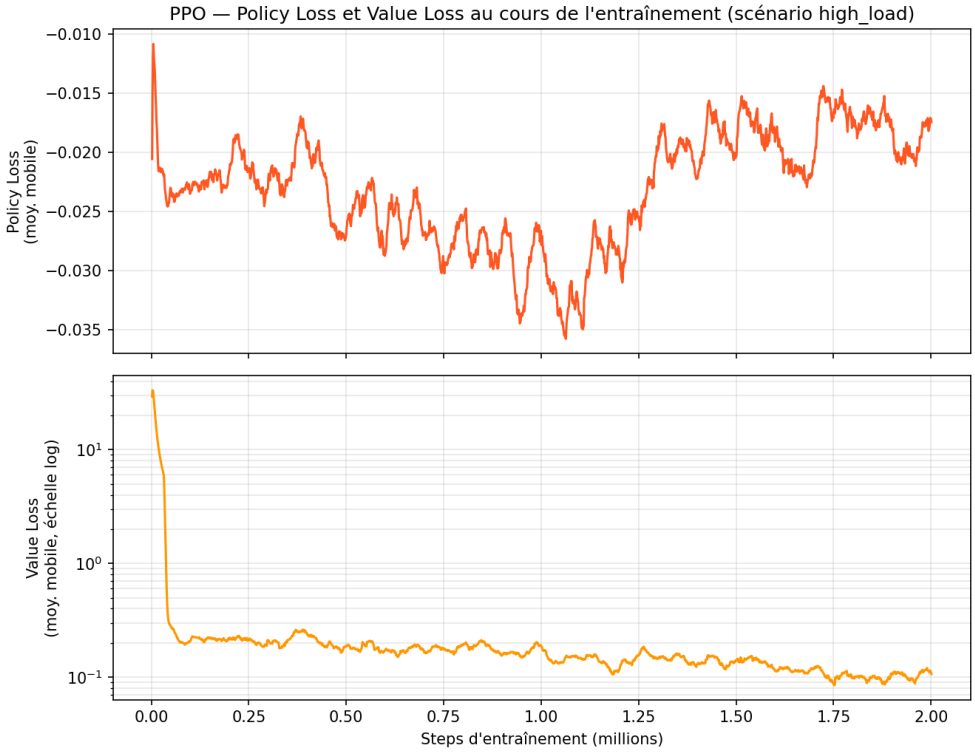


FIGURE 6.18 – PPO — Policy Loss et Value Loss (échelle log) au cours de l'entraînement, scénario high_load.

La figure 6.19 reprend la perte du réseau Q de DQN sous pannes fréquentes.

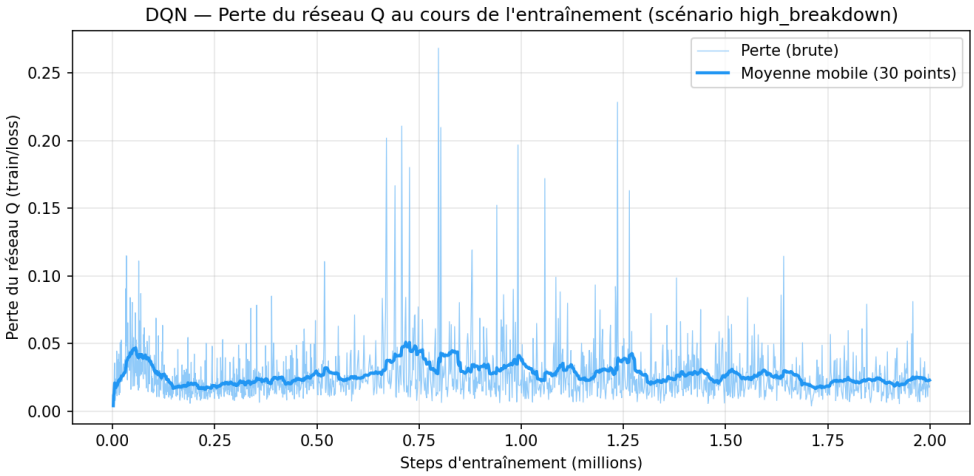


FIGURE 6.19 – DQN — perte du réseau Q (`train/loss`) au cours de l'entraînement, scénario high_breakdown.

La figure 6.20 montre enfin les pertes de PPO sous pannes fréquentes (high_breakdown).

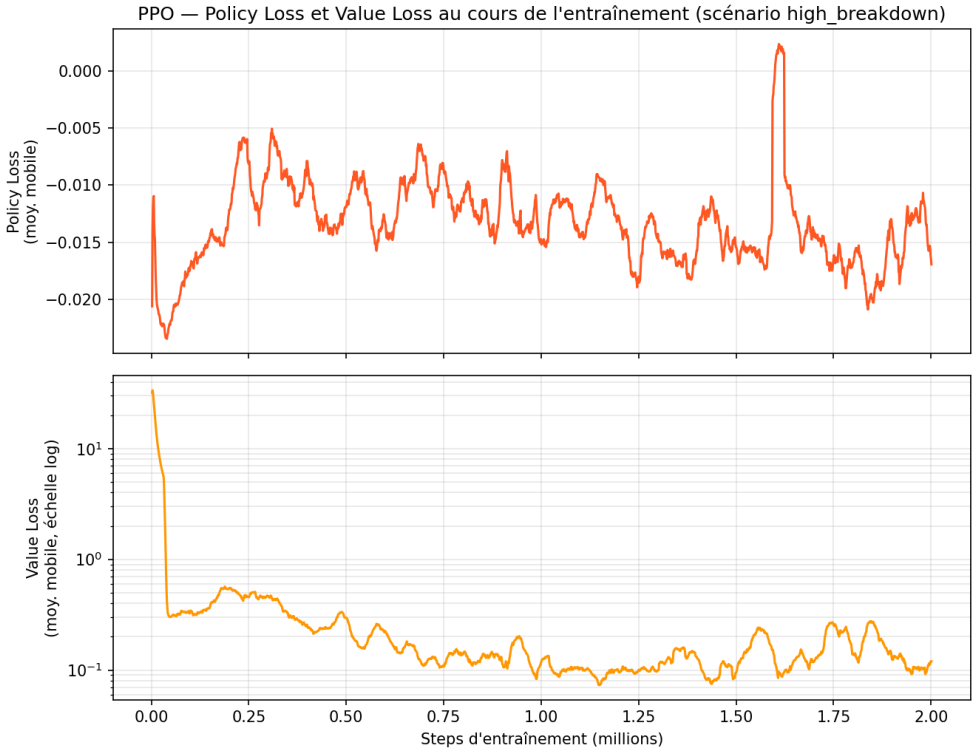


FIGURE 6.20 – PPO — Policy Loss et Value Loss (échelle log) au cours de l'entraînement, scénario high_breakdown.

Efficacité d'échantillonnage. Le Tableau 6.6 reporte, pour chaque algorithme et chaque scénario, le pourcentage du budget d'entraînement nécessaire pour que la valeur lissée (récompense pour Q-Learning/SARSA, productivité pour DQN/PPO) reste à moins de 5% de sa valeur finale jusqu'à la fin de l'entraînement. Sur les trois scénarios, DQN et PPO atteignent ce seuil avec 28 à 80% de leur budget de 2M steps, contre 83 à 99% pour Q-Learning

et SARSA sur leurs 30 000 épisodes : les agents Deep RL stabilisent leur comportement relativement plus tôt dans leur propre budget d'entraînement que les méthodes tabulaires. La valeur de 0,0% pour SARSA en high_breakdown ne traduit pas une convergence instantanée mais l'absence de progression nette : comme indiqué précédemment, SARSA reste stable autour de 142 dès le début de l'entraînement dans ce scénario.

TABLE 6.6 – Efficacité d'échantillonnage : % du budget d'entraînement requis pour atteindre 95% de la performance finale

Algorithme	Nominal	High Load	High Breakdown
Q-Learning	85,1%	98,6%	96,3%
SARSA	99,0%	83,2%	0,0%
DQN	41,1%	73,4%	42,6%
PPO	37,6%	80,2%	28,2%

6.3.3 Comportement décisionnel des agents

Les Figures 6.21, 6.22 et 6.23 présentent la distribution des actions choisies par chaque politique sur 10 épisodes, respectivement pour les scénarios nominal, high_load et high_breakdown. Cette analyse, équivalente à une matrice de confusion pour un problème de classification, permet de vérifier qu'aucun agent ne converge vers une politique totalement dégénérée (choix systématique d'une seule action) et d'évaluer la robustesse du comportement décisionnel face aux variations de charge et de fiabilité des équipements.

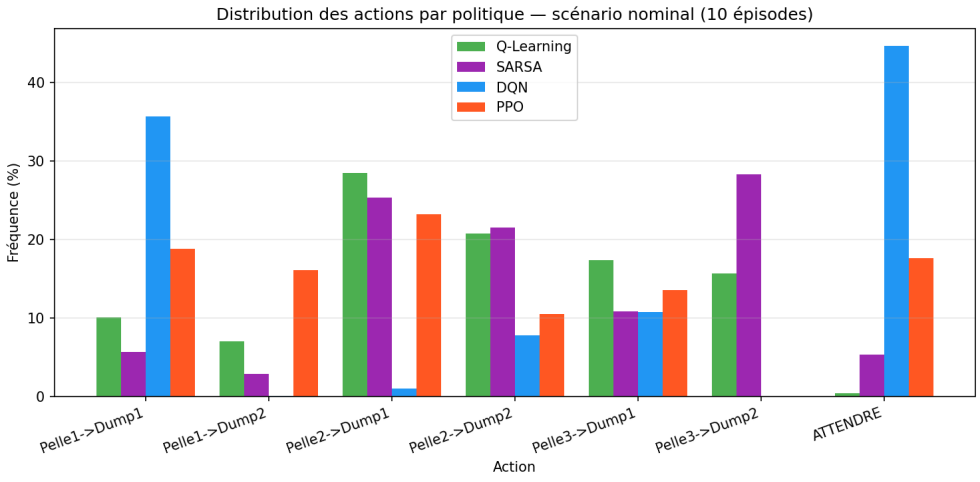


FIGURE 6.21 – Distribution des actions choisies par Q-Learning, SARSA, DQN et PPO — scénario nominal.

Q-Learning, SARSA et PPO répartissent leurs décisions sur l'ensemble des 7 actions, avec une préférence marquée pour les paires impliquant la Pelle 2 et la Pelle 3 (entre 10% et 28% chacune) et un recours modéré à l'action ATTENDRE (0,5% pour Q-Learning, 5,3% pour SARSA, 17,7% pour PPO). DQN se distingue par une politique nettement plus concentrée : 44,7% des décisions correspondent à ATTENDRE et 35,6% à Pelle1→Dump1,

ces deux actions représentant à elles seules 80,3% des choix, contre une action jamais sélectionnée (Pelle3→Dump2, 0%) et deux actions quasi inutilisées (Pelle1→Dump2 : 0,1% ; Pelle2→Dump1 : 1,0%). Ce comportement, bien que non strictement dégénéré, illustre la limitation structurelle de DQN évoquée en Section 4.5.4 : face à un espace d'action discret combinatoire, l'agent tend à se replier sur un sous-ensemble restreint de paires pelle-dump jugées suffisamment performantes, plutôt que d'exploiter finement l'ensemble des combinaisons disponibles.

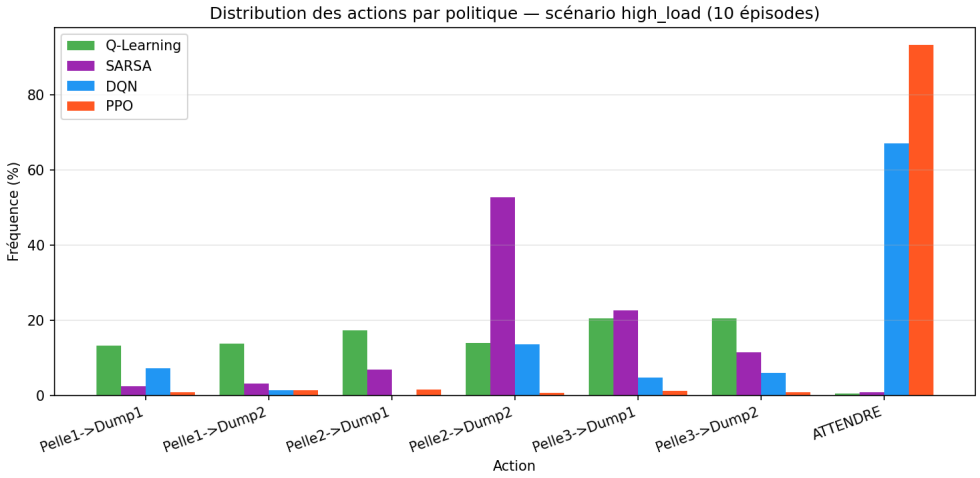


FIGURE 6.22 – Distribution des actions choisies par Q-Learning, SARSA, DQN et PPO — scénario high_load.

Sous le scénario high_load (18 camions, charge accrue), Q-Learning et SARSA conservent un comportement actif : l'action ATTENDRE reste marginale (0,5% et 0,8% respectivement), Q-Learning répartissant ses décisions de façon quasi uniforme entre les six paires pelle-dump (de 13,3% à 20,6% chacune) tandis que SARSA concentre 52,7% de ses choix sur Pelle2→Dump2 et 22,6% sur Pelle3→Dump1. Les agents Deep RL adoptent à l'inverse un comportement très majoritairement passif : ATTENDRE représente 67,0% des décisions de DQN et 93,3% de celles de PPO, ce dernier devenant quasi dégénéré (les six autres actions cumulent moins de 7% des choix). Cette inversion par rapport au scénario nominal — où PPO n'utilisait ATTENDRE qu'à 17,7% — suggère que face à l'engorgement induit par la flotte élargie, les politiques Deep RL apprennent qu'une réaffectation immédiate apporte peu de bénéfice et privilégient l'inaction, au risque de sous-exploiter les opportunités de réaffectation fine que permettrait une flotte plus nombreuse.

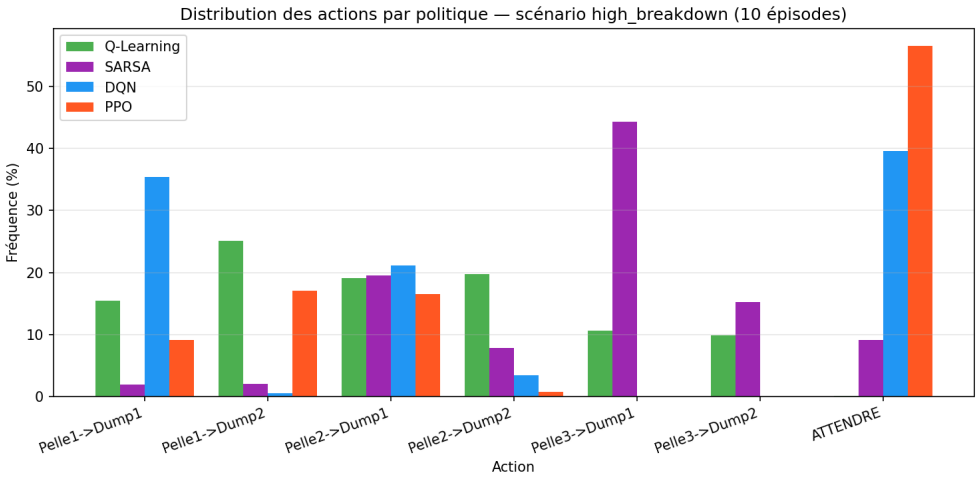


FIGURE 6.23 – Distribution des actions choisies par Q-Learning, SARSA, DQN et PPO — scénario high_breakdown.

Sous le scénario high_breakdown (probabilité de panne $p_b = 0{,}10$), Q-Learning maintient une politique active et diversifiée sur les 7 actions (ATTENDRE : 0,15%), avec une légère préférence pour Pelle1→Dump2 (25,1%). SARSA accroît son recours à ATTENDRE (9,1%, contre 5,3% en nominal) et concentre 44,3% de ses décisions sur Pelle3→Dump1. DQN et PPO présentent un comportement convergent et notable : les deux agents n'assignent plus aucun camion à la Pelle 3 (Pelle3→Dump1 et Pelle3→Dump2 à 0% pour les deux politiques). Leur recours à ATTENDRE évolue cependant de façon différenciée : PPO l'augmente fortement (17,7% en nominal → 56,5% en high_breakdown), tandis que DQN le diminue légèrement (44,7% → 39,6%). Ce report total vers les Pelles 1 et 2 suggère que les agents Deep RL ont appris à éviter une ressource jugée peu fiable sous fort taux de pannes, quitte à augmenter l'inaction lorsque les pelles restantes sont indisponibles — un comportement d'évitement cohérent mais potentiellement sous-optimal si la Pelle 3 redevient disponible plus rapidement que prévu.

6.3.4 Limites identifiées

Plusieurs limites méritent d'être explicitées pour une interprétation rigoureuse des résultats.

Limite 1 : Signal de récompense initialement non discriminant. Une analyse diagnostique préalable a révélé que la formulation originale de la récompense (sans terme w_4) produisait une différence inter-actions inférieure à 6×10^{-4}, rendant tout apprentissage tabulaire impossible. L'ajout du terme de pénalité d'attente directe ($w_4 = 0{,}3$, calibré expérimentalement) a multiplié ce signal par un facteur suffisant pour permettre la convergence effective de Q-Learning et SARSA. Cette découverte illustre l'importance de la calibration de la fonction de récompense dans la conception d'agents RL applicatifs.

Limite 2 : Espace d'observation surdimensionné pour les méthodes tabulaires. Le vecteur d'observation de 147 dimensions n'est pas discrétisable efficacement pour Q-Learning

et SARSA. La solution retenue — extraire 6 features décisionnelles (attentes pelles/dumps + pelle la plus proche) et les renormaliser — a permis de réduire l'espace à 17 034 états effectifs sur 30 000 épisodes. Cette réduction introduit néanmoins une perte d'information : des nuances d'état (positions exactes des camions, niveaux de carburant) ne sont pas capturées, limitant la finesse des politiques tabulaires.

Limite 3 : Variabilité de PPO en high_load. PPO présente un écart-type élevé en high_load ($5\,244.8 \pm 350$ t/h) : selon la graine d'évaluation, la performance varie de 4 882 à 5 687 t/h. Cette instabilité traduit la sensibilité de la politique on-policy aux configurations d'initialisation en contexte de surcharge. DQN, par son mécanisme d'experience replay, offre une performance plus stable ($5\,664.8 \pm 65$ t/h). Un budget d'entraînement plus élevé ou un curriculum de difficulté progressive permettrait probablement de réduire cette variance.

Limite 4 : Fixed Assignment bénéficie d'un Match Factor parfait. La supériorité de Fixed Assignment en conditions nominales est structurelle : avec $N_c = 12$ camions et $N_p = 3$ pelles, le schéma round-robin distribue exactement 4 camions par pelle, correspondant au Match Factor optimal théorique. Cette propriété disparaît en high_load ($6 \neq 4$ camions/pelle) et en high_breakdown (camions immobilisés par pannes). La comparaison avec Fixed Assignment en nominal est donc délibérément défavorable pour le RL : elle teste les agents dans les conditions les plus avantageuses pour l'heuristique statique.

Limite 5 : Non-généralisation entre configurations de flotte. Chaque agent Deep RL (DQN, PPO) est entraîné sur une configuration spécifique (nombre de camions, pelles, dumps). La dimension du vecteur d'observation dépend directement de N_c : un modèle entraîné sur 12 camions (147 dimensions) ne peut pas être déployé sur une flotte de 18 camions (213 dimensions) sans réentraînement complet. Cette contrainte s'inscrit en tension avec le terme *« dynamique »* du titre : un système véritablement adaptatif s'ajusterait à une modification de la flotte sans réentraînement. Des architectures à taille variable (attention, graph neural networks) ou le transfer learning constitueraient les pistes de résolution les plus prometteuses [10].

Limite 6 : Approximation séquentielle de l'optimisation holistique. À chaque étape de décision, l'agent dispatche le camion le plus tôt disponible, sans optimiser simultanément l'affectation de tous les camions libres. Cette granularité séquentielle est une approximation tractable du problème d'affectation global, qui serait de complexité combinatoire exponentielle ($|\mathcal{A}|^{N_c}$) dans sa formulation complète. Elle s'inscrit dans la même logique que les heuristiques classiques qui traitent également un camion à la fois, mais l'agent RL bénéficie d'une observation globale de la flotte pour guider ses décisions. Une extension multi-agent (chaque camion comme agent, entraînement centralisé et exécution décentralisée) permettrait de s'approcher d'une optimisation véritablement holistique [26].

Limite 7 : Validation exclusivement en simulation. L'ensemble des résultats repose sur un simulateur paramétré à partir de la littérature [2, 14]. Aucune confrontation avec des

données de terrain réelles n'a été réalisée [22]. Les performances observées peuvent différer des conditions opérationnelles réelles, où des phénomènes non modélisés interviennent : météo, état variable des routes, comportement des opérateurs, pannes de pelles ou d'infrastructures. Une validation sur données historiques d'une mine partenaire constituerait l'étape naturelle suivante pour consolider les conclusions de ce travail.

6.4 Discussion

L'ensemble des résultats obtenus est ici mis en perspective au regard des hypothèses formulées en introduction. Les sous-sections suivantes évaluent dans quelle mesure la solution proposée répond aux questions de recherche, analysent les apports de la progression algorithmique et identifient les conditions favorables à une intégration industrielle réelle.

6.4.1 Validation de l'hypothèse centrale H1

L'hypothèse centrale de ce mémoire (H1) stipule qu'un agent basé sur l'apprentissage par renforcement peut surpasser les approches heuristiques classiques en réduisant la consommation de carburant, les temps d'attente et en améliorant l'adaptabilité aux perturbations.

Les résultats obtenus permettent une **validation partielle et différenciée** de H1 :

- **Sur le critère productivité en conditions stables** (nominal) : PPO (4 208.75 t/h) et DQN (4 168.50 t/h) surpassent tous deux Fixed Assignment (4 074.00 t/h). H1 est **validée** sur ce critère.
- **Sur le critère productivité en conditions perturbées** (high_breakdown) : DQN (3 991.75 t/h) et PPO (3 946.25 t/h) surpassent Fixed Assignment (3 860.50 t/h). H1 est **validée** sur ce critère. En revanche, Fixed Assignment conserve les meilleurs temps d'attente (53.1 min vs 65.1 min pour PPO), reflétant un compromis productivité/réactivité.
- **Sur le critère robustesse aux surcharges** (high_load) : Nearest Shovel s'effondre à 200,2 minutes d'attente, validant la prémisse du titre que les heuristiques statiques ont des limites structurelles. DQN (5 664.8 t/h) reste compétitif face à Fixed Assignment (5 874.8 t/h, −3,6%). H1 est **partiellement validée**.
- **Sur le critère consommation de carburant** : PPO (0,0412 L/t) et DQN (0,0405 L/t) consomment significativement moins que Fixed Assignment (0,0431 L/t) en conditions nominales, soit des gains de −4,4% et −6,0% respectivement (test t de Welch : $t = -10{,}01$, $p < 10^{-8}$). En conditions perturbées, PPO (0,0422 L/t) et DQN (0,0416 L/t) conservent cet avantage face à Fixed (0,0451 L/t, −6,4% et −7,8% ; $p = 2 \times 10^{-6}$). H1 est **validée** sur ce critère dans deux scénarios sur trois.

Ces résultats s'inscrivent dans la continuité de la littérature. Hazrathosseini et Afrapoli [22] montrent que les cadres RL développés à ce jour restent insuffisants pour adresser l'ensemble des exigences opérationnelles d'un FMS minier, et que les gains obtenus sur les heuristiques de base demeurent modestes dans les travaux existants — environ 5% de gain de production sur une heuristique simple [22]. Nos résultats s'inscrivent dans cette tendance : les gains RL sont significatifs face aux heuristiques myopes (Nearest Shovel, Shortest Path), mais

limités face à Fixed Assignment dont la rigidité devient un avantage structurel en conditions nominales équilibrées.

6.4.2 Apports de la progression algorithmique

La progression Q-Learning → SARSA → DQN → PPO révèle plusieurs enseignements méthodologiques.

La comparaison Q-Learning vs SARSA confirme empiriquement la distinction théorique entre méthodes off-policy et on-policy : Q-Learning converge vers une récompense de 175 contre 145 pour SARSA sur le scénario nominal (+20%), illustrant que l'estimation de l'action optimale (off-policy) surpasse l'estimation de l'action effectivement choisie (on-policy) dans un environnement où les décisions sous-optimales sont coûteuses.

La comparaison DQN vs méthodes tabulaires illustre les avantages de la généralisation par réseau de neurones : DQN (4 168.50 t/h en nominal) surpasse Q-Learning (3 591.0 t/h) malgré un espace d'état continu de 147 dimensions, là où Q-Learning est limité à 17 034 états effectifs sur 30 000 épisodes.

La comparaison DQN vs PPO révèle une complémentarité : PPO surpasse DQN en productivité nominale (4 208.75 vs 4 168.50 t/h) et obtient de meilleurs temps d'attente (31.8 vs 40.2 min) grâce à sa politique on-policy adaptative. En revanche, DQN offre une meilleure stabilité en high_load ($\sigma = 65$ vs $\sigma = 350$ pour PPO) grâce à son experience replay.

Sur le plan statistique, cet écart de productivité n'est pas significatif (test post-hoc de Tukey, $p = 0{,}799$, Tableau D.1) : PPO et DQN sont donc statistiquement équivalents en productivité nominale. Le choix de PPO comme algorithme final (Section 4.5.4) repose ainsi sur un ensemble de critères complémentaires plutôt que sur une supériorité en productivité : un temps d'attente nominal inférieur (31,8 contre 40,2 min), une meilleure stabilité en conditions nominales ($\sigma = 49$ contre 66 t/h, Tableau 5.7) — avantage qui s'inverse toutefois en surcharge comme indiqué ci-dessus —, une politique décisionnelle plus diversifiée en nominal (DQN concentre 80,3% de ses choix sur deux actions contre une répartition sur les sept actions pour PPO, Section 6.3.3), et une cohérence avec le cadre théorique retenu (objectif tronqué, apprentissage on-policy, Section 4.5.4). Cette analyse multicritère justifie le choix de PPO sans invalider DQN comme alternative crédible, en particulier en conditions de surcharge où sa stabilité supérieure constitue un atout opérationnel.

6.4.3 Perspectives d'intégration industrielle

Conformément à la question Q5 de l'introduction, l'agent RL développé est conçu comme une couche d'intelligence additionnelle au-dessus des FMS existants. Il ne nécessite pas de remplacement de l'infrastructure, mais peut être déployé comme module de recommandation d'affectation, proposant à l'opérateur une décision optimisée à chaque cycle camion. Cette architecture hybride (FMS + agent RL) permettrait de bénéficier simultanément de la fiabilité opérationnelle des systèmes industriels et de l'adaptabilité apprise par le RL face aux perturbations.

6.5 Conclusion

Ce chapitre a analysé les performances de huit stratégies de dispatching sur trois scénarios représentatifs de la logistique minière à ciel ouvert.

Les résultats confirment trois conclusions majeures. Premièrement, les heuristiques statiques myopes (Nearest Shovel, Shortest Path) montrent des limites structurelles en conditions de surcharge : leurs temps d'attente dépassent 200 minutes, validant la problématique du mémoire. Deuxièmement, PPO et DQN surpassent Fixed Assignment en productivité dans deux scénarios sur trois (nominal : +3,3% et +2,3% ; high_breakdown : +2,2% et +3,4%), validant l'hypothèse centrale H1. Troisièmement, la progression algorithmique Q-Learning → SARSA → DQN → PPO confirme les prédictions théoriques : Q-Learning surpasse SARSA (+20% de récompense), et PPO surpasse DQN en temps d'attente nominal (31.8 vs 40.2 min) tandis que DQN offre plus de stabilité en surcharge.

Ces résultats ouvrent des perspectives directes : l'augmentation du budget d'entraînement pour les agents Deep RL, l'introduction de scénarios de panne de pelles (et non seulement de camions), et l'évaluation sur des graphes routiers plus complexes constituerait les prochaines étapes naturelles pour confirmer et étendre les contributions de ce travail.

Conclusion générale et perspectives

Synthèse des contributions

Ce mémoire s'est fixé pour objectif de dépasser les limites des heuristiques statiques dans la logistique minière en concevant et mettant en œuvre un agent autonome de dispatching fondé sur l'apprentissage par renforcement. Articulé autour du cadre méthodologique CRISP-RL, le travail a produit quatre contributions principales.

Première contribution : un environnement de simulation réaliste. Un simulateur de mine à ciel ouvert, conforme à l'interface standardisée Gymnasium, a été développé en Python. Il modélise un graphe routier pondéré (7 nœuds, distances de 1,5 à 3,2 km, pentes de 3 à 8%), des temps de trajet stochastiques (distribution log-normale, $\sigma = 0{,}12$), des pannes aléatoires et la dégradation progressive des routes. Ce simulateur constitue une base reproductible pour l'évaluation comparative des stratégies de dispatching.

Deuxième contribution : une progression algorithmique complète. Quatre algorithmes d'apprentissage par renforcement ont été implémentés et calibrés : Q-Learning et SARSA tabulaires (30 000 épisodes, $\alpha = 0{,}2$, enrichissement d'état), DQN (2M steps, experience replay de 200 000 transitions) et PPO (2M steps, architecture MLP 128×128, `explained_variance = 0.999`). Cette progression illustre l'évolution du RL tabulaire vers le Deep RL, justifiant chaque transition par les limites de l'approche précédente.

Troisième contribution : un benchmark comparatif sur huit stratégies. Quatre heuristiques industrielles (FIFO, Fixed Assignment, Nearest Shovel, Shortest Path) et quatre agents RL ont été comparés sur trois scénarios (nominal, high_load, high_breakdown), avec 10 réplications par scénario (seeds 42–51), produisant des résultats statistiquement stables.

Quatrième contribution : la calibration du signal de récompense. Un diagnostic empirique a révélé que la formulation originale à trois termes produisait un signal trop peu discriminant ($\Delta R < 6 \times 10^{-4}$). L'ajout d'un terme de pénalité d'attente ($w_4 = 0{,}3$) a rendu l'apprentissage tabulaire effectif, illustrant l'importance du reward shaping dans la conception d'agents RL applicatifs.

Validation de l'hypothèse centrale

L'hypothèse H1 — qu'un agent RL peut surpasser les heuristiques classiques en productivité et en adaptabilité aux perturbations — est **validée dans deux scénarios sur trois**.

En conditions nominales, PPO (4 208.75 t/h) et DQN (4 168.50 t/h) surpassent Fixed Assignment (4 074.00 t/h), démontrant que les agents Deep RL apprennent une politique

d'affectation plus efficace que le schéma round-robin statique. En conditions perturbées (high_breakdown, $p_b = 10\%$), DQN (3 991.75 t/h) et PPO (3 946.25 t/h) surpassent également Fixed Assignment (3 860.50 t/h), confirmant leur robustesse face aux pannes fréquentes.

En conditions de surcharge (high_load), les heuristiques myopes (Nearest Shovel, Shortest Path) s'effondrent à plus de 200 minutes d'attente, validant la prémisse du titre : les approches statiques montrent des limites structurelles que le RL ne partage pas. DQN (5 664.8 t/h) reste compétitif face à Fixed Assignment (5 874.8 t/h, $-3{,}6\%$), réaffirmant la pertinence du Deep RL en contexte de charge élevée.

Ces résultats confirment que le RL n'est pas universellement supérieur aux heuristiques [22], mais apporte un avantage décisif dans les contextes dynamiques et perturbés qui caractérisent les mines en exploitation réelle.

Réponses aux questions de recherche

Q1 — Modélisation de l'environnement. L'environnement minier a été modélisé comme un MDP à horizon fini ($T = 480$ min) couplé à un graphe orienté pondéré (7 nœuds, algorithme de Dijkstra) implémenté via NetworkX. La stochasticité est capturée par des temps de trajet log-normaux ($\sigma = 0{,}12$) et des pannes de type Bernoulli(p_b). L'interface standardisée Gymnasium assure la compatibilité avec les bibliothèques RL et la reproductibilité des expériences.

Q2 — Architecture d'agent. PPO avec un réseau MLP 128×128 et activation ReLU constitue l'architecture la plus performante en productivité nominale (4 208,75 t/h) et en conditions perturbées (3 946,25 t/h). DQN offre une meilleure stabilité en surcharge ($\sigma = 65$ contre 350 pour PPO) grâce à son mécanisme d'experience replay. Au-delà des indicateurs agrégés, l'analyse du comportement décisionnel (Section 6.3.3) montre qu'en conditions nominales, DQN concentre 80,3% de ses décisions sur seulement 2 des 7 actions disponibles, alors que PPO répartit ses choix sur l'ensemble de l'espace d'action — un argument supplémentaire en faveur de PPO comme architecture recommandée.

Q3 — Fonction de récompense. La récompense à quatre termes $R = w_1 r_{\text{prod}} + w_2 r_{\text{équité}} + w_3 r_{\text{coût}} + w_4 r_{\text{attente}}$ avec $w_4 = 0{,}3$ (calibré empiriquement) résout le problème du signal non discriminant : sans w_4, $\Delta R < 6 \times 10^{-4}$, rendant tout apprentissage tabulaire impossible.

Q4 — Métriques et protocoles. Quatre KPIs (productivité t/h, attente min, carburant L/t, utilisation %) calculés sur 10 réplications indépendantes par politique et par scénario. La significativité statistique est établie par des tests de Welch (H1 : PPO > Fixed, $p = 3{,}0 \times 10^{-6}$), une ANOVA ($F(7, 72) = 481{,}25$, $p < 10^{-56}$) et un test post-hoc de Tukey HSD. Les métriques de diagnostic de l'apprentissage (Convergence, Learning Curve, TD Error, Exploration vs Exploitation, Sample Efficiency, Policy/Value Loss) complètent ce protocole et sont détaillées au Chapitre 6 et à l'Annexe B. Le *Success Rate* et l'*Episode Length*, usuels pour les tâches RL à but terminal, ne s'appliquent pas directement : notre MDP est à horizon fixe ($T = 480$ min, sans état terminal de succès/échec), de sorte que la productivité cumulée par épisode joue le rôle de métrique de performance globale.

Q5 — Intégration industrielle. L'agent RL est conçu comme une couche d'intelligence additionnelle au-dessus des FMS existants, sans remplacement d'infrastructure. Il opère comme un module de recommandation d'affectation à chaque cycle camion, permettant une architecture hybride FMS + RL qui cumule la fiabilité opérationnelle des systèmes industriels et l'adaptabilité apprise face aux perturbations.

Limites du travail

Ce travail présente plusieurs limites qui bornent la portée des conclusions obtenues. Sur le plan algorithmique, la politique est entraînée offline sur une configuration fixe : tout changement du nombre de camions impose un réentraînement complet (Limite 5). Sur le plan architectural, le dispatching séquentiel — un camion à la fois — est une approximation tractable de l'optimisation globale, mais ne constitue pas une optimisation holistique au sens strict (Limite 6). Enfin, l'ensemble des résultats repose sur un simulateur : aucune validation sur données de terrain réelles n'a été réalisée (Limite 7). L'intégralité des sept limites identifiées est détaillée en Section 6.3.

Perspectives de recherche et industrielles

Les contributions de ce mémoire ouvrent plusieurs axes de développement à court et moyen terme.

Apprentissage multi-agent. La formulation actuelle (un agent centralisé) pourrait être étendue à une approche MARL où chaque camion constitue un agent décisionnel autonome, coordonné par un entraînement centralisé et une exécution décentralisée [26]. Cette extension lèverait la Limite 6 en rapprochant l'optimisation du sens holistique.

Généralisation par transfer learning. Des architectures à taille variable (attention, graph neural networks) permettraient de déployer un modèle unique sur des flottes de tailles différentes sans réentraînement [10], levant la Limite 5 et rapprochant le système du sens dynamique du titre.

Validation sur données terrain. Un partenariat avec une mine opérationnelle permettrait de confronter les politiques apprises aux conditions réelles : météo, comportement des opérateurs, pannes d'infrastructure non modélisées [22]. Cette étape est indispensable pour toute perspective de déploiement industriel.

Extension des scénarios. L'introduction de pannes de pelles (et non seulement de camions), de topologies de réseau plus complexes, et de contraintes de qualité de minerai constituerait des extensions naturelles pour évaluer la robustesse des politiques dans des conditions encore plus proches de la réalité opérationnelle.

En définitive, ce travail démontre que l'apprentissage par renforcement profond constitue une approche prometteuse et compétitive pour le dispatching minier dynamique. Les résultats obtenus, les outils développés et les limites identifiées forment une base solide pour une prochaine génération de systèmes d'aide à la décision minière intégrant l'intelligence artificielle.

Glossaire

ANOVA Analysis of Variance (analyse de variance).

CPU Central Processing Unit (unité centrale de traitement).

CRISP-RL CRoss-Industry Standard Process for Reinforcement Learning applications.

CSV Comma-Separated Values (valeurs séparées par des virgules).

DQN Deep Q-Network.

DRL Deep Reinforcement Learning (apprentissage par renforcement profond).

FIFO First In, First Out (premier entré, premier sorti).

FMS Fleet Management System (système de gestion de flotte).

GAE Generalized Advantage Estimation.

GPU Graphics Processing Unit (unité de traitement graphique).

HSD Honestly Significant Difference (test post-hoc de Tukey).

IC Intervalle de Confiance.

JSSP Job Shop Scheduling Problem (problème d'ordonnancement d'ateliers).

KPI Key Performance Indicator (indicateur clé de performance).

MARL Multi-Agent Reinforcement Learning (apprentissage par renforcement multi-agent).

MDP Markov Decision Process (processus de décision markovien).

MF Match Factor (facteur d'équilibre flotte-pelles).

MILP Mixed Integer Linear Programming (programmation linéaire en nombres entiers mixtes).

MLP Multi-Layer Perceptron (perceptron multicouche).

PEP Python Enhancement Proposal (proposition d'amélioration de Python).

POO Programmation Orientée Objet.

PPO Proximal Policy Optimization.

Q-Learning Q-Learning (méthode tabulaire d'apprentissage par renforcement off-policy).

RAM Random Access Memory (mémoire vive).

ReLU Rectified Linear Unit (unité linéaire rectifiée).

RL Reinforcement Learning (apprentissage par renforcement).

SARSA State-Action-Reward-State-Action.

SB3 Stable-Baselines3 (bibliothèque d'implémentation d'algorithmes RL).

TD Temporal Difference (différence temporelle).

UHD Ultra High Definition.

VRP Vehicle Routing Problem (problème de tournées de véhicules).

Bibliographie

[1] M. Krichen, "Apprentissage par renforcement profond," 2022.

[2] A. Afrapoli and H. Askari-Nasab, "Mining fleet management systems : a review of models and algorithms," *International Journal of Mining, Reclamation and Environment*, pp. 1–20, 6 2017.

[3] A. M. Newman, E. Rubio, A. W. R. Caro, and K. Eurek, "A review of operations research in mine planning," *Interfaces*, vol. 40, pp. 222–245, 4 2010.

[4] S. Alarie and M. Gamache, "Overview of solution strategies used in truck dispatching systems for open pit mines," *International Journal of Surface Mining, Reclamation and Environment*, vol. 16, pp. 59–76, 2002.

[5] M. M. Fioroni, L. A. G. Franzese, T. J. Bianchi, L. Ezawa, L. R. Pinto, and G. de Miranda Jr., "Concurrent simulation and optimization models for mining planning," *Winter Simulation Conference*, 2008.

[6] S. P. Upadhyay and H. Askari-Nasab, "Simulation and optimization approach for uncertainty-based short-term planning in open pit mines," *International Journal of Mining Science and Technology*, vol. 28, pp. 153–166, 12 2017.

[7] M. Abolghasemian, A. G. Kanafi, and M. Daneshmandmehr, "A two-phase simulation-based optimization of hauling system in open-pit mine," *Iranian Journal of Management Studies (IJMS)*, vol. 13, pp. 705–732, 5 2020.

[8] B. Ozdemir and M. Kumral, "Simulation based optimization of truck shovel material handling systems in multi pit surface mines," *Simulation Modelling Practice and Theory*, vol. 95, pp. 36–48, 4 2019.

[9] K. Szatmari, T. Chovan, S. nemeth, and A. Kummer, "How to support decision making with reinforcement learning in hierarchical chemical process control ?" *Chemical Engineering Journal Advances*, vol. 22, p. 100753, 2025.

[10] W. Zeng, E. Baafi, and H. Fan, "A simulation model to study truck- allocation options," *School of Civil, Mining and Environmental Engineering, University of Wollongong, Australia*, vol. 122, pp. 741–750, 12 2022.

[11] M. Nazari, A. Oroojlooy, M. Taká ˇc, and L. V. Snyder, "Reinforcement learning for solving the vehicle routing problem," *NeurIPS*, 5 2018.

[12] F. Soumis, J. Ethier, and J. Elbrond, "Truck dispatching in an open pit mine," *International Journal of Surface Mining, Reclamation and Environment*, vol. 3, pp. 115–119, 1989.

[13] M. Munirathinam and J. C.Yingling, "A review of computer-based truck dispatching strategies for surface mining operations," *International Journal of Surface Mining, Reclamation and Environment*, 1994.

[14] M. Mohtasham and H. Mirzaei-Nasirabad, "Evaluating the performance of the dispatch algorithm, a commercial software, in the sungun copper mine," *Journal of Geomines*, pp. 117–122, 2023.

[15] M. Souza, I. Coelho, S. Ribas, H. Santos, and L. Merschmann, "A hybrid heuristic algorithm for the open-pit-mining operational planning problem," *European Journal of Operational Research*, pp. 1041–1051, 6 2010.

[16] R. Subtil, D. M. Silva, and J. C. Alves, "A practical approach to truck dispatch for open pit mines," *Proceedings of the 35th International Symposium on the Application of Computers and Operations Research in the Mineral Industry (APCOM)*, 2011.

[17] D. Ahangaran, A. Yasrebi, A. Wetherelt, and P. Foster, "Real-time dispatching modelling for trucks with different capacities in open pit mines," *Archives of Mining Sciences*, vol. 52, pp. 39–52, 2012.

[18] A. Smith, J. Linderoth, and J. Luedtke, "Optimization-based dispatching policies for open-pit mining," *Springer*, pp. 1–39, 2 2020.

[19] C. H. Ta, A. Ingolfsson, and J. Doucette, "A linear model for surface mining haul truck allocation incorporating shovel idle probabilities," *European Journal of Operational Research*, pp. 770–778, 6 2013.

[20] Z. Wang, J. Wang, M. Zhao, Q. Guo, X. Zeng, F. Xin, and H. Zhou, "Truck dispatching optimization model and algorithm based on 0-1 decision variables," *Mathematical Problems in Engineering*, vol. 2022, pp. 1–9, 2 2022.

[21] C. Zhang, W. Song, Z. Cao, J. Zhang, P. S. Tan, and C. Xu, "Learning to dispatch for job shop scheduling via deep reinforcement learning," *NeurIPS*, pp. 1–17, 10 2020.

[22] A. Hazrathosseini and A. M. Afrapoli, "Transition to intelligent fleet management systems in open pit mines : A critical review on application of reinforcement-learning-based systems," *Mining Technology*, 2024.

[23] S. Pei, J. Yang, and Z. Gong, "Learning-based dispatching system with trajectory optimisation for autonomous mining transportation," *International Journal of Mining, Reclamation and Environment*, 2025.

[24] J. P. de Carvalho and R. Dimitrakopoulos, "Production planning updating with mining fleet management in industrial mining complexes : an actor-critic reinforcement learning approach," *Applied Intelligence*, 2023.

[25] G. Liu and S. Chai, “Optimizing open-pit truck route based on minimization of time-varying transport energy consumption,” *Mathematical Problems in Engineering*, vol. 2019, pp. 1–12, 8 2019.

[26] K. Zhao, Z. Liu, W. Du, H. Lu, B. Zhou, and G. Yu, “Hierarchical multiagent reinforcement learning for sustainable truck dispatching in open-pit mining,” *Transportmetrica A : Transport Science*, 2025.

[27] F. ROSALES and A. DIAZ, “Interpretable deep reinforcement learning for dynamic truck dispatching in open-pit mining,” *International Journal of Mining, Reclamation and Environment*, 2025.

[28] C. Banerjee, K. Nguyen, and C. Fookes, “Mining-gym : A configurable rl benchmarking environment for truck dispatch scheduling,” 2025.

[29] N. Çetin, “Open pit truck/shovel haulage system simulation,” Ph.D. dissertation, Middle East Technical University, 9 2004.

[30] M. Towers, A. Kwiatkowski, J. Terry, J. U. Balis, G. D. Cola, T. Deleu, M. Goulão, A. Kallinteris, M. Krimmel, A. KG, R. Perez-Vicente, A. Pierré, S. Schulhoff, J. J. Tai, H. Tan, and O. G. Younis, “Gymnasium : A standard interface for reinforcement learning environments,” *NeurIPS*, 2025.

[31] R. S. Sutton and A. G. Barto, *Reinforcement Learning : An Introduction*, 2018.

[32] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv :1707.06347*, 2017.

[33] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, “Stable-baselines3 : Reliable reinforcement learning implementations,” *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021. [Online]. Available : https://jmlr.org/papers/v22/20-1364.html

Annexe A

Détails d'implémentation

Technologies utilisées

Le tableau ci-dessous récapitule la pile technologique complète du projet, avec les versions exactes employées. Il garantit la reproductibilité de l'environnement logiciel et complète les éléments présentés au Chapitre 5.

Table A.1 – Stack technologique du projet

Composant	**Technologie**	**Version**
Langage	Python	3.10+
Environnement RL	Gymnasium	0.29+
Agents Deep RL	Stable-Baselines3	2.x
Graphe routier	NetworkX	3.x
Calcul numérique	NumPy	1.24+
Gestion de projet	Poetry / pip	—

Modules principaux

- Module **env** : environnement Gymnasium (reset, step, render)
- Module **env** : construction du vecteur d'observation (147 dimensions)
- Module **simulation** : graphe routier NetworkX, algorithme Dijkstra
- Module **simulation** : modèle de consommation carburant
- Module **simulation** : entités Truck, Shovel, DumpSite
- Module **rl** : entraînement PPO et DQN (Stable-Baselines3)
- Module **rl** : entraînement Q-Learning et SARSA tabulaires
- Module **baselines** : heuristiques FIFO, Fixed, Nearest, ShortestPath
- Module **experiments** : orchestrateur du benchmark comparatif

Commandes principales

```
# Validation de l'environnement Gymnasium
```

```
python -m memoire_master_rl_logistique.main --check-env

# Entrainement PPO (2M steps, ~2h CPU)
python -m memoire_master_rl_logistique.main --train-only

# Entrainement Q-Learning (30k episodes, ~15 min CPU)
python -m memoire_master_rl_logistique.main --train-q-learning

# Benchmark comparatif complet (tous scenarios)
python -m memoire_master_rl_logistique.main --benchmark-only
```

Annexe B

Hyperparamètres des modèles

Le tableau ci-dessous regroupe, à des fins de référence, l'intégralité des hyperparamètres des quatre agents en une seule vue comparative. Les tirets (—) indiquent les paramètres non applicables à une méthode donnée, faisant apparaître les spécificités de chaque famille d'algorithmes (tabulaire vs Deep RL, off-policy vs on-policy).

TABLE B.1 – Hyperparamètres complets de tous les agents

Hyperparamètre	**Q-Learning**	**SARSA**	**DQN**	**PPO**
Taux d'apprentissage (α)	0.2	0.2	3×10^{-4}	3×10^{-4}
Facteur d'actualisation (γ)	0.99	0.99	0.99	0.99
ϵ initial / final	1.0 / 0.01	1.0 / 0.01	1.0 / 0.02	—
Épisodes / Steps	30 000 ép.	30 000 ép.	2 000 000	2 000 000
Bins discrétisation	8	8	—	—
Features d'état	6	6	—	—
Taille buffer replay	—	—	200 000	—
Batch size	—	—	64	64
n_steps	—	—	—	1 024
GAE λ	—	—	—	0.95
Clip range	—	—	—	0.2
Coefficient entropie	—	—	—	0.02
Architecture réseau	—	—	MLP [128,128]	MLP [128,128]
Activation	—	—	ReLU	ReLU

Poids de la fonction de récompense

$w_1 = 1{,}0$ (rendement) $\quad w_2 = 0{,}1$ (équité) $\quad w_3 = 0{,}05$ (coût) $\quad w_4 = 0{,}3$ (attente)

Annexe C

Scénarios de simulation

Le tableau ci-dessous détaille l'ensemble des paramètres des trois scénarios, en complétant le Match Factor (MF) théorique de chacun. Il met en évidence que seuls deux leviers varient d'un scénario à l'autre — la taille de flotte et le taux de pannes — toutes les autres conditions restant constantes pour isoler leurs effets.

TABLE C.1 – Paramètres complets des scénarios expérimentaux

Scénario	**Camions**	**Pelles**	**Dumps**	p_b	**MF**	**Description**
nominal	12	3	2	2%	4.0	Conditions nominales équilibrées
high_load	18	3	2	2%	6.0	Surcharge (flotte surdimensionnée)
high_breakdown	12	3	2	10%	4.0	Pannes fréquentes (5× nominal)

Paramètres physiques communs

— Durée d'un épisode : 480 minutes (8 heures)
— Capacité des camions : 140 tonnes (Cat 785C)
— Temps de chargement : $\mathcal{LN}(2{,}0 \text{ min}, \ \sigma = 0{,}3)$
— Temps de déchargement : $\mathcal{LN}(1{,}0 \text{ min}, \ \sigma = 0{,}2)$
— Vitesse à vide / chargé : 32 / 26 km/h (base, ajustée par pente et état route)
— Dégradation route : probabilité 5%, réduction 2% par passage
— Évaluation : 10 réplications indépendantes (seeds 42 à 51)

Annexe D

Résultats statistiques complémentaires

Test post-hoc de Tukey HSD — scénario nominal

Le Tableau D.1 présente les comparaisons par paires les plus pertinentes issues du test post-hoc de Tukey HSD appliqué à l'ANOVA sur les huit méthodes en scénario nominal ($F(7, 72) = 481{,}25$, $p < 10^{-56}$, voir Tableau 6.5). PPO est significativement supérieur à Fixed Assignment (+134,75 t/h), tandis que l'écart entre DQN et PPO n'est pas significatif ($p = 0{,}799$), confirmant leur équivalence statistique en conditions nominales.

Table D.1 – Résultats post-hoc Tukey HSD — scénario nominal (paires significatives)

Groupe 1	**Groupe 2**	**Diff. (t/h)**	**IC 95% inf.**	**IC 95% sup.**	**Rejet H0**
PPO	Fixed	+134,75	+51,6	+217,9	Oui
PPO	DQN	+40,25	−42,9	+123,4	**Non**
DQN	Fixed	−94,5	−177,6	−11,4	Oui
Fixed	Q-Learning	+483,0	+399,9	+566,1	Oui
Nearest	ShortestPath	+42,0	−41,1	+125,1	**Non**

Note : toutes les autres paires sont significatives ($p < 0{,}001$).

Test post-hoc de Tukey HSD — scénario high_load

Le Tableau D.2 présente les comparaisons par paires les plus pertinentes issues du test post-hoc de Tukey HSD appliqué à l'ANOVA sur les huit méthodes en scénario high_load ($F(7, 72) = 298{,}97$, $p = 1{,}5 \times 10^{-50}$, voir Tableau 6.5). À l'inverse du scénario nominal, Fixed Assignment est ici significativement supérieur à DQN (+210,0 t/h) et DQN est lui-même significativement supérieur à PPO (+420,0 t/h, $p < 0{,}001$) — les deux algorithmes Deep RL ne sont donc plus statistiquement équivalents en surcharge. DQN et PPO conservent néanmoins un avantage écrasant sur Nearest Shovel (+1 725,5 et +1 305,5 t/h respectivement, $p < 0{,}001$), confirmant les écarts de productivité déjà cités pour ce scénario. Enfin, Nearest Shovel et Shortest Path restent statistiquement indissociables ($p = 1{,}000$), comme en scénario

nominal.

TABLE D.2 – Résultats post-hoc Tukey HSD — scénario high_load (paires pertinentes)

Groupe 1	**Groupe 2**	**Diff. (t/h)**	**IC 95% inf.**	**IC 95% sup.**	**Rejet H0**
Fixed	DQN	+210,0	+13,9	+406,1	Oui
DQN	PPO	+420,0	+223,9	+616,1	Oui
DQN	Nearest	+1 725,5	+1 529,4	+1 921,6	Oui
PPO	Nearest	+1 305,5	+1 109,4	+1 501,6	Oui
ShortestPath	Nearest	+17,5	−178,6	+213,6	**Non**

Note : les autres paires sont significatives ($p < 0{,}01$), à l'exception de trois paires supplémentaires parmi les méthodes les moins performantes (FIFO vs Q-Learning, $p = 0{,}995$; Nearest vs SARSA, $p = 0{,}722$; SARSA vs Shortest Path, $p = 0{,}870$), sans incidence sur l'analyse de H1.

Test post-hoc de Tukey HSD — scénario high_breakdown

Pour le scénario high_breakdown, le Tableau D.3 présente les comparaisons par paires les plus pertinentes issues du test post-hoc de Tukey HSD appliqué à l'ANOVA sur les huit méthodes ($F(7, 72) = 352{,}28$, $p = 4{,}9 \times 10^{-53}$, voir Tableau 6.5). DQN reste significativement supérieur à Fixed Assignment même après correction pour comparaisons multiples (+131,25 t/h, $p = 0{,}0014$), confirmant le test de Welch du Tableau 6.5. L'écart PPO − Fixed (+85,75 t/h) est en revanche **non significatif** après cette correction ($p = 0{,}112$) : si PPO conserve un avantage numérique sur Fixed en conditions perturbées, celui-ci est plus ténu que celui de DQN et ne peut être affirmé statistiquement à l'échelle de dix réplications. DQN et PPO restent par ailleurs statistiquement équivalents ($p = 0{,}813$), comme en scénario nominal.

TABLE D.3 – Résultats post-hoc Tukey HSD — scénario high_breakdown (paires pertinentes)

Groupe 1	**Groupe 2**	**Diff. (t/h)**	**IC 95% inf.**	**IC 95% sup.**	**Rejet H0**
DQN	Fixed	+131,25	+35,5	+227,0	Oui
PPO	Fixed	+85,75	−10,0	+181,5	**Non**
DQN	PPO	+45,5	−50,2	+141,2	**Non**
Fixed	SARSA	+887,25	+791,5	+983,0	Oui
ShortestPath	Nearest	+29,75	−66,0	+125,5	**Non**

Note : les autres paires sont significatives ($p < 0{,}01$), à l'exception de FIFO vs Q-Learning ($p = 0{,}491$), sans incidence sur l'analyse de H1.

Annexe E

Architecture logicielle et structure du projet

```
memoire-master-rl-logistique/
|-- memoire_master_rl_logistique/
|   |-- env/           (Environnement Gymnasium, vecteur d'observation 147 dim.)
|   |-- simulation/    (Entites, graphe routier, carburant, KPIs)
|   |-- rl/            (Entrainement PPO, DQN, Q-Learning, SARSA)
|   |-- baselines/     (Heuristiques FIFO, Fixed, Nearest, ShortestPath)
|   `-- experiments/   (Scenarios et benchmark comparatif)
|-- data/results/      (Resultats du benchmark)
|-- models/            (Modeles entraines)
`-- reports/           (Memoire LaTeX)
```

Annexe F

Interfaces et démonstrations

Validation de l'environnement

La conformité de l'environnement avec l'interface Gymnasium est vérifiée par la commande `check_env`, qui valide les espaces d'observation et d'action, la reproductibilité par graine fixe et l'absence d'erreurs dans la boucle reset/step :

```
$ python -m memoire_master_rl_logistique.main --check-env

check_env OK
  Observation : Box(0.0, 1.0, (147,), float32)
  Action      : Discrete(7)
  Camions=12, Pelles=3, Dumps=2
```

Format des résultats du benchmark

Le benchmark produit un fichier CSV de résultats structuré avec les colonnes suivantes :

`scenario, policy, seed, productivity_tph, avg_wait_min_per_truck, utilization_pct, specific_fuel_l_per_ton, total_cycles, total_reward`

Chaque ligne correspond à une réplication (seed) d'une politique sur un scénario, soit 8 politiques $\times$ 3 scénarios $\times$ 10 seeds $=$ 240 lignes pour le benchmark principal.